Mind It! v1.69 — QC Report (builds 121–144)

Date: 12 October 2026 · **Version:** 1.69 (unchanged) · **Android versionCode:** 121 · **Source:** git tag `checkpoint-build121`

Where the testing happened: everything in this report was run on the build machine (Linux sandbox): the real Windows laptop app (Electron, the same app shell as `Mind It!.exe`) under a virtual screen, with the tablet and phone pages opened in Chromium at real sizes and touch emulation. **No physical Windows PC, Galaxy Tab A9+, phone or TV was used.** The live acceptance test is **PENDING USER LIVE TEST** (section J).

Overall readiness: Ready for your live acceptance test — not yet cleared for unrestricted live use. Every automated check, audit, stress test and the 60-minute soak passed in the sandbox, and the rollback was tested; what's missing is a real game on your real laptop, tablet, phones and TV (section J). Keep build 119 until that passes.

Build 144 addendum (21 October 2026)

What you asked for (21 Oct, 13:14): "Do 5,6" — ↔ Move a whole category, and the docs brought up to date — from the list offered at 13:09 ("any upgrade suggestions?"); "show me 3" (category colours on the TV board), then **13:18:** "3 is rejected, it makes it look tacky". Before/after picture of 5 sent at about 13:19; **13:20:** "approved". Also settled: "For 7 it stays on the square", "wizarding name ok", "Gaming theme stays", "Loading is fine". Estimated time given: about 17:30 Dubai.

Same version: v1.69 · versionCode 144 · `APP_BUILD=144`. New page module `app138.js` (`build.py` order `…,136,137,138,6`). This is a minor change under your rule of 20 Oct (no stress phases or soak; next due: build 146).

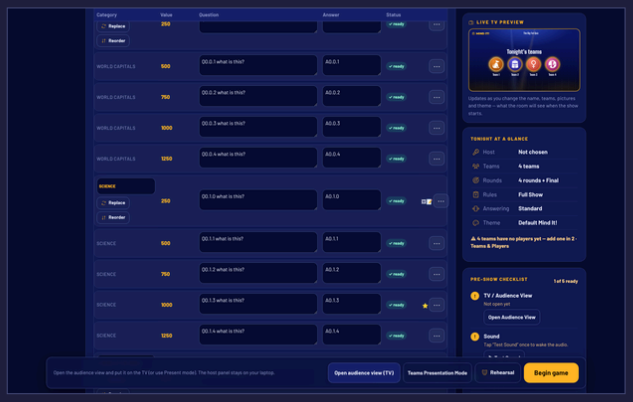
A · ↔ Move a whole category (`app138.js`)

- **Where:** Setup → 4 · Rounds & Questions → Review & Edit Clues, on the first row of each category, after 🔄 Replace and ↕ Reorder.
- **What it does:** every round is a fixed grid, so the category swaps places with the one picked — in another round, or in the same round (its column changes). It asks first ("Swap "A" and "B"?"; Back changes nothing).
- **What moves:** the category name (and any other category-level field) and each question's contents — answer, alternates, host note, pictures and video, Question Database link. **What stays:** the values, the Daily Double squares and the played marks (your rule, 21 Oct: a Daily Double stays on its square).
- **Can't be picked:** the category itself, one with a different number of questions, one with a played question in a paused game (that category gets no ↔ Move either).
- ↩ **Undo move** in the toolbar of both rounds straight after; kept in memory only, never in the save. No change to the save format.

Build 144 · 5 · Move a whole category to another round

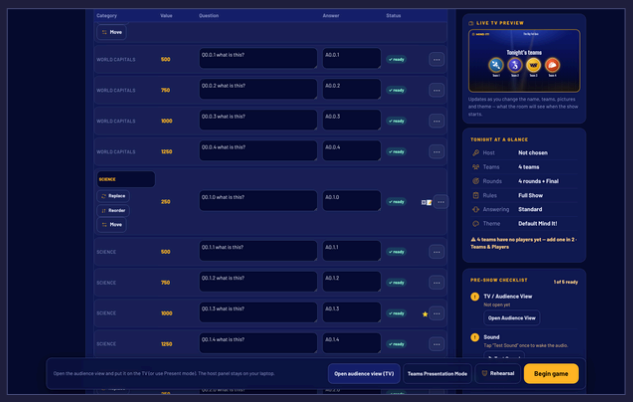
Setup → 4 · Rounds & Questions → the clue list. ↔ Move swaps a category with the one you pick (another round, or the same round to change its column). The questions travel with their answers, notes and pictures; values and Daily Double squares stay on their board.

BEFORE · build 143

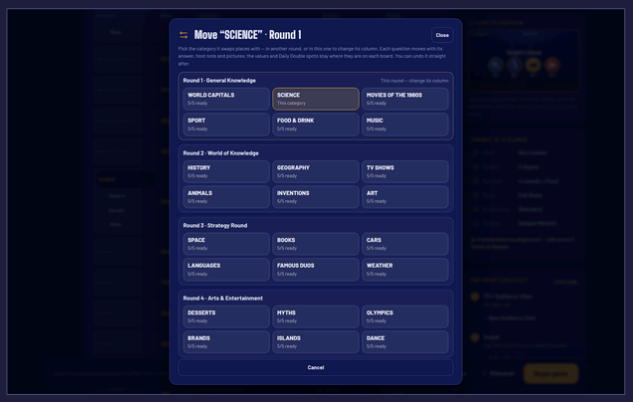


Each category has Replace and Reorder. Moving a category to another round means retyping it.

AFTER · build 144

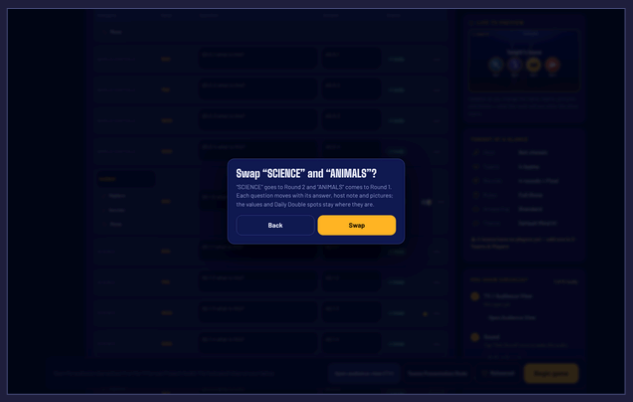


New ↔ Move button under Replace and Reorder.



Pick the category to swap with — every round is listed; played or wrong-sized ones are greyed out.

AFTER · build 144



It asks first (Back changes nothing).

Done: ANIMALS is now in Round 1, SCIENCE in Round 2. An Undo Move button appears at the top of the list.

B · Docs brought up to date (item 6)

- Places where the docs and the code or repository disagreed: CLAUDE.md (git covered only part of the project; the Android build through the retired `build.sh`; the passwords "in build.sh"), CURRENT_STATUS §4/§8 (build 110, the MindIt-Downloads public/private question), MASTER_CONTEXT §2/§16.3/§17/§18 (build 110, the build-108 source zip), ARCHITECTURE header/§2/§9/§13 (the build-108 source zip, `build.sh`), and the Daily Double question that MASTER_CONTEXT called unanswered while GAME_RULES §25-I had it settled on 10 Oct. All rewritten to match the code; no code changed for this.

C · Tests

- **New suite:** `fix144` (webpage) 31/31. `run_all.py` now has 150 suites.
- **Full run** on the final page (md5 `1c8b2474…`), with the build-144 tablet app inside the laptop app: full run on the shipping page (md5 `1c8b2474…`, with the build-144 tablet app inside the laptop app): 150 suites, 3,321/3,321 checks (102 + 3 min). In the first run 147 suites passed first time and 3 couldn't run their comparisons with older builds because the new workspace was missing two things — the repository's git history (fix130c, fix133m: build-129/132 code) and the older published pages (fix140s: the build-139 page; run_all.py drops the web proxy, so it can't download them). Both were fetched, the three suites re-run on the same page and passed (fix140s 36/36, fix130c 7/7, fix133m 32/32), and `setup_workspace.sh` now fetches both. No retries.
- **Stress phases and 60-minute soak:** not run — a minor change under the owner's rule of 20 Oct (stress phases and the 60-minute soak only for a major change or every 5th build). Last run: build 141; next due: build 146
- **Reproducible build:** identical (tools/verify_build.py: a fresh clone of GitHub main ef86314 builds the tested page byte for byte, sha256 0fd85b27…); the tablet APK from a fresh clone: a fresh clone of HEAD builds the tested page and an APK carrying it, signed with the original key (5f4819e4…), apk_check.py OK.
- **Code map:** modules 132 · changed functions 206 · hooks 244 · wrappers 298 · replacements 51 · issues 0 (new 0, known 0, fixed since baseline 0) · parse errors 0.

D · Problems found while building

- **The Move dialog's cards** sat 4 + 2 per round at first (uneven); they now sit 3 + 3.

Build 143 addendum (21 October 2026)

What you asked for (21 Oct, 09:28): "7" — the tablet offers its own update — from the list offered at 09:26 ("any more upgrades or improvements?"). Before/after picture sent at about 09:37; **09:39:** "approved". Estimated time given: about 2:30 pm, then about 12:45 pm Dubai.

Same version: v1.69 · versionCode 143 · `APP_BUILD=143`. New page module `app137.js` (`build.py` order `…,135,136,137,6`). This is a minor change under your rule of 20 Oct (no stress phases or soak; next due: build 146).

A · The Galaxy Tab app (Java)

- `MainActivity.Bridge.appBuild()` → the app's versionCode.
- `MainActivity.Bridge.openTabletAppPage(host)` → `Intent.ACTION_VIEW` on `http://<host>/tablet-app`. The host must be a local IPv4 address (10.x, 172.16–31.x, 192.168.x, 169.254.x) with an optional port; the path is fixed. Anything else is refused (`false`).
- **Checked:** the APK compiles with both methods (present in `classes.dex`); the address rule run in Java on ten cases: `192.168.1.23:8765`, `10.0.0.5`, `172.20.1.1:80`, `192.168.137.1:8765` accepted; `8.8.8.8`, `example.com`, `192.168.1.23:8765/evil`, `javascript:alert(1)`, `172.32.0.1`, `192.168.1.23:8765?x` refused. Opening Chrome from the app has not been tried on a real tablet.

B · The tablet's Main Menu (`android/assets/start.html`)

- When a laptop is found (`probe` → `FOUND[host]`), `checkUpd(host)` asks it once for `/api/tablet-app` (3 s limit). If the laptop's tablet app is newer than `MindItAndroid.appBuild()`, the **Update Available** card appears under the found card: "Galaxy Tab app build N is on · This tablet has build M. Keep the laptop and the tablet on the same build." **Update** → `openTabletAppPage(host)`; if that is refused, a message gives the address to open in Chrome. Apps before build 143 (no `appBuild`) and quizzes hosted on another tablet get nothing.

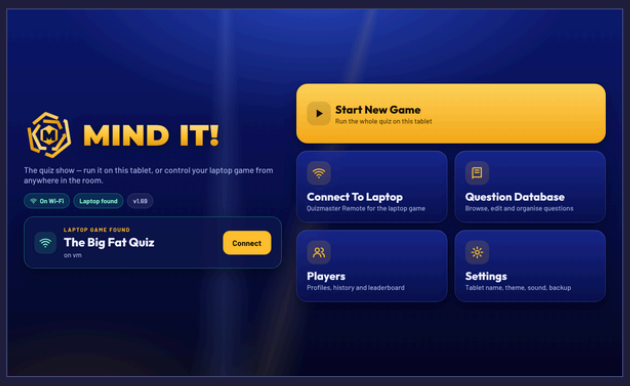
C · The tablet remote (`app137.js`)

- Only in the Galaxy Tab app (`IS_LINK_REMOTE` with `MindItAndroid.openTabletAppPage`). Once connected (`live`), it asks the laptop once for `/api/tablet-app`; a newer build gives the `.lr-upd` banner under the top bar with **Update** and **Later**, in place of app125's `.lr-build` "Different builds" line (that line comes back after Later). The build compared is the app's own (`appBuild()`, else `APP_BUILD`). **Later** is remembered for the app session (`sessionStorage`). A refused open puts the address in the banner itself (a remote waiting for control shows no toasts — found by the new test).
- **Checked:** `fix143d` 13/13.

Build 143 · 7 · The tablet offers its own update

When the Galaxy Tab finds or connects to a laptop that carries a newer tablet app, it says so and offers Update: the tablet's browser opens the laptop's download page (over your Wi-Fi), then Download and Update. Here the laptop has build 143 and the tablet build 142.

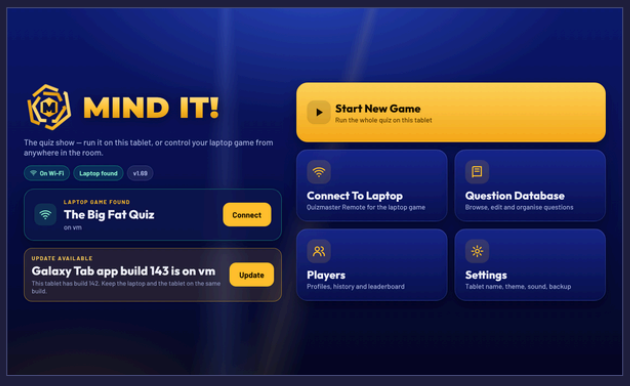
BEFORE · build 142



Home screen: the laptop is found; nothing says the tablet is out of date.

Connected: 'Different builds … update the older one from the download page' — no way to do it from here.

AFTER · build 143



Home screen: an Update Available card under the laptop it found, with Update.

Connected: the same offer as a banner, with Update and Later (it replaces the plain Different builds line).

D · Tests

- **New suite:** `fix143d` (laptop app + a tablet browser with a stand-in bridge) 13/13. `run_all.py` now has 149 suites.
- **Full run** on the final page (md5 `d8ecccf0…`), with the build-143 tablet app inside the laptop app: full run on the shipping page (md5 `d8ecccf0…`): 149 suites, 3,288/3,288 checks, all on the first try (104 min). (qc2 counts 270 checks this time — its random whole games vary the number of checks from run to run; 0 failures.)
- **Layout audit** (`avaudit`): 270 screens at 9 sizes, 0 issues on the final page
- **Stress phases and 60-minute soak:** not run — a minor change under the owner's rule of 20 Oct (stress phases and the 60-minute soak only for a major change or every 5th build). Last run: build 141 (14/14); next due: build 146
- **Reproducible build:** identical (SHA-256 c541e502…abaa, commit 1c6cced); the tablet APK from a fresh clone: yes (versionCode 143, v2 + v3, certificate 5f4819e4…; `appBuild` and `openTabletAppPage` in classes.dex; the APK built with the private build.sh from the same files has identical contents).
- **Code map:** modules 131 · changed functions 206 · hooks 244 · wrappers 297 · replacements 51 · issues 0 (new 0, known 0, fixed since baseline 0) · parse errors 0.

E · Problems found while building

- **The first version compared with the page's build**, so a tablet page and a laptop on the same build never showed the offer even when the app itself was older (seen in the after picture). It now compares with the app's own versionCode.
- **The tablet showed two warnings** when the builds differed: the old Different builds line and the new banner. The banner now replaces the line while it is shown.
- **The "can't open the browser" message** used the remote's toast, which a tablet waiting for control doesn't show. It is now written in the banner.

Build 142 addendum (20 October 2026)

What you asked for (20 Oct, 17:33): "1,4" from the eight ideas offered after build 141 — 1 · update the Galaxy Tab app from the laptop, 4 · an automatic Safety Backup at the end of every game. **17:35:** "all pics are approved for now". **17:41:** "i still have to enter my pin to connect laptop and tab, fix that too pls". Estimated time given: about 10:30 pm, then about 10:45 pm Dubai with the PIN fix.

Same version: v1.69 · versionCode 142 · `APP_BUILD=142`. New page module `app136.js` (`build.py` order `…,134,135,136,6`).

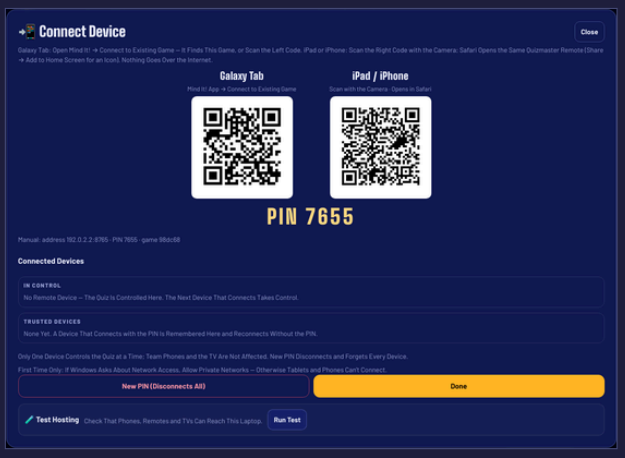
A · 1 · The Galaxy Tab app from the laptop

- **Where it comes from:** `desktop/build_desktop.sh win|mac [page] [tablet.apk]` copies the APK of the same build into `desktop/app/tablet/` with `tablet-app.json` (`build` = the versionCode in `android/AndroidManifest.xml`, `sha256`). The release EXE and Mac app were built after the APK, from it. Not in the repository (`.gitignore`).
- **The laptop app (`main.js`):** `GET /api/tablet-app` → `{ok, build, size, sha, path}` ; `GET /tablet-app` → a small page (title, "Build N · the tablet app that matches this laptop", one Download button, three steps); `GET|HEAD /tablet-app/Mind-It-Android-Galaxy-Tab.apk` → the file (`application/vnd.android.package-archive`, `Content-Disposition: attachment`, `Content-Length`, `nosniff`). Any device on the network may fetch them — it is the same public app as on the download page; nothing else is served under `/tablet-app/`. Without a bundled APK the details say `ok: false`, the page and the file answer 404 ("isn't included in this copy"), and the Quizmaster page offers nothing. Port 80 serves them too (the same handler).
- **The page (`app136.js`):** only in the laptop app (`LIVE.on`, not the Galaxy Tab's own page) it fetches `/api/tablet-app` once. 📲 **Connect Device** gets a line under the codes ("Galaxy Tab app build 142 is in this laptop app" · 📲 **Update the Tablet App**); the button opens 📲 **Update the Galaxy Tab App**: the QR code of `http://<Wi-Fi address>:<port>/tablet-app`, the address, three steps (scan; Download anyway; Update) and the Auto Blocker note. Connected Devices' **Different builds** warning (app124) gets the same button and says an iPad or iPhone should reload instead.
- **Checked:** `fix142d` 17/17 — the details match the bundled file and this laptop's build; the page and the APK from the Wi-Fi address (byte for byte), HEAD, three other addresses refused; the line, the panel (its code for the right address, three steps), the warning's button; a tablet browser opening the address and downloading the identical file. `fix142` — away from the laptop app (a browser, the Galaxy Tab's quiz page) nothing is offered and nothing is fetched.

Build 142 · 1 · Tablet app from the laptop

The laptop app now carries the Galaxy Tab app of the same build and offers it over your Wi-Fi: scan a code on the laptop with the tablet's camera and it downloads straight from the laptop — no GitHub, no internet. Only in the laptop app; the tablet app itself is unchanged.

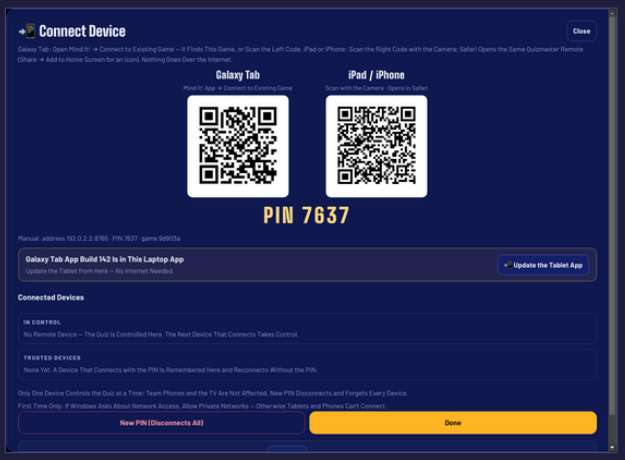
BEFORE · build 141



Connect Device: the codes to connect a tablet, nothing to update it.

A tablet on another build: 'Update it from the download page' — on the tablet, that meant GitHub.

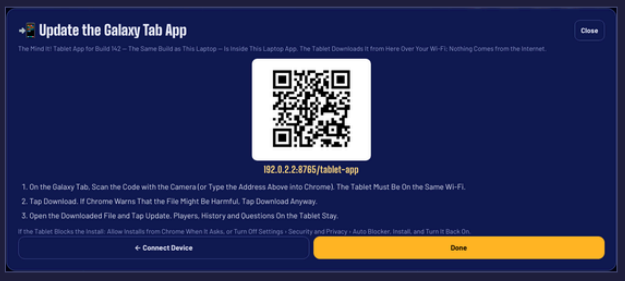
AFTER · build 142



A new line under the codes: the tablet app build 142 is in this laptop app, with an Update the Tablet App button.

The Different builds warning gets the same button.

AFTER · build 142 — the code and what the tablet sees



Scan with the tablet's camera (or type the address into Chrome). Three steps, including Chrome's 'Download anyway'.

On the tablet: one Download button (from the laptop),

B · The PIN between the laptop and the Galaxy Tab

- **The cause (the code is the source of truth; the guide said otherwise):** since build 128 a remote that pairs with the PIN gets its own key (`mindit.trust[hostId]`) and the laptop lets it in with that key alone (`trustCheck` , no PIN or game number). But the tablet's Main Menu (`android/assets/start.html` , `connectTo`) asked for the PIN whenever the laptop's game number (`SESSION` , new each time the laptop app starts) differed from the saved one — so every quiz night. The guide (5.1) said "after a laptop restart it reconnects by itself, without the PIN"; that was only true inside an open remote page.
- **The fix:** `connectTo` reads the laptop's `hostId` from `/api/link/info` ; if this tablet holds a key for it, it opens the remote page straight away (no PIN), and the remote page signs in with the key as before. If the laptop has forgotten the tablet (New PIN, Forget), the remote page says so ("isn't on the laptop's list of trusted devices any more"), removes the key, and the next Connect asks for the PIN. Laptops before build 130 (no `hostId`) still get the PIN.
- **Checked:** `fix142d` — pair with the PIN from the code; change the saved game number (as after a laptop restart); **Connect To Laptop → Connect** goes straight in (connected, listed on the laptop); without the key the PIN is asked.

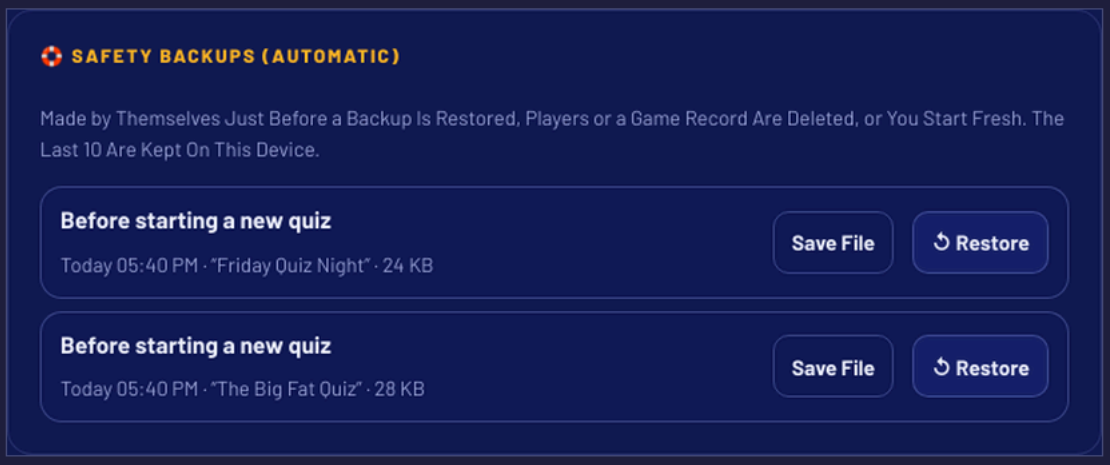
C · 4 · A Safety Backup after every game

- **How:** a `finalizeGame` wrapper (outermost, after app41's Rehearsal rule): when this call records the game (`recordedThisGame()` false before, true after), 1.5 s later `safetySnapshot("After the game — <winners> won")` (app114; `exportFull()` , so it holds the game, its record in Historic Games, players and the Question Database). Once per game id; never for a Rehearsal (never recorded). Same list and same limit (the last 10). The Safety backups card now says "after every game". On the laptop, the tablet's own quiz page and any browser.
- **Checked:** `fix142` 12/12 — one backup named after the winner, not again on the standings / winner screens, the record inside, a second game's own (joint winners both named), none for a Rehearsal, the card text; the Galaxy Tab's quiz page too. `fix142d` — in the laptop app.

Build 142 · 4 · Backup after every game

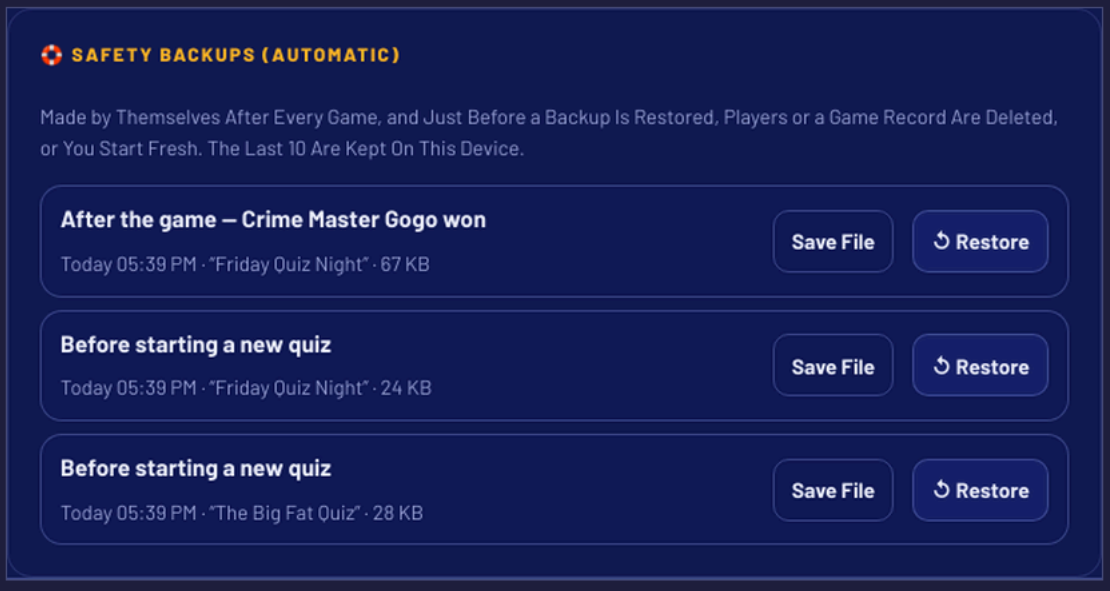
Right after a game is recorded in Historic Games, a Safety Backup is made by itself (never for a Rehearsal), in the same list as the others: Backups & Restore → Safety backups, the last 10 on this device. Nothing else changes on screen.

BEFORE · build 141



A game has just finished: only the backups made before a change — the finished game is in none of them.

AFTER · build 142



The same game: 'After the game — Crime Master Gogo won' at the top, with Save File and Restore. The card says they are now made after every game too.

D · Tests

- **New suites:** `fix142` (browser) 12/12; `fix142d` (laptop app + a tablet browser) 17/17. `run_all.py` now has 148 suites. `fix141` / `fix141d` accept build 141 or later.
- **Full run** on the final page (md5 `6d8b34cd…`), with the build-142 tablet app inside the laptop app: full run on the shipping page (md5 `6d8b34cd…`): 148 suites, 3,296/3,296 checks, achphx (Phoenix achievements) passed on its automatic retry — a one-off timing failure under load; run 4 more times on its own afterwards: 7/7 each (107 min).
- **Layout audit** (`avaudit`): 270 screens at 9 sizes, 0 issues on the final page

- **Stress phases:** not run (the same rule; last run: build 141, 24/24) · **60-minute soak:** not run — the owner's rule since 20 Oct, 17:56 ("lets limit the stress tests and 60 minute soaks to major changes or after every 5 builds"): build 142 is a minor change (new download addresses on the laptop, the tablet's Main Menu, one more kind of Safety Backup). Last run: build 141 (14/14); next due: build 146
- **Reproducible build:** the final page rebuilt from a fresh clone of GitHub `main` is identical: identical (SHA-256 04902d56…facd, commit 322e9d8); the tablet APK built from a fresh clone with `build_apk.sh` : yes (versionCode 142, v2 + v3, certificate 5f4819e4…; the APK built with the private build.sh from the same files has identical contents).
- **Code map:** modules 130 · changed functions 206 · hooks 243 · wrappers 297 · replacements 51 · issues 0 (new 0, known 0, fixed since baseline 0) · parse errors 0.

E · Problems found while building

- **The Different builds check in `fix142d`** first read the warning 1.2 s after drawing it, when the laptop's next device update had already redrawn the list without the made-up tablet. It now reads it at once (a test timing issue; the app was right).
- **The documents and the code disagreed** about the PIN (see B): the guide promised no PIN after a laptop restart. The code now does what the guide said; the guide adds when that changed.

Build 141 addendum (20 October 2026)

What you asked for (20 Oct, 09:36): "any upgrades or improvements?" Nine were suggested, with the places where the documents and the code disagreed. **09:39:** "do them all". Estimated time given: pictures about 1:30 pm, the release about 9 pm Dubai.

Approval: before/after pictures of the five you can see were sent at about 10:00 (Pre-show checklist, ⚑ Fix Later, the early-reveal question, the Night Recap, the Storage line). **10:04:** "ok all five". The other four change nothing on screen (a smaller complete copy, two release checks, two unused functions removed).

Same version: v1.69 · versionCode 141 · `APP_BUILD=141` . New page module `app135.js` (`build.py` order `…,133,134,135,6`).

A · 1 · Storage the browser keeps

- **Asking:** 2.5 s after the host page opens, `navigator.storage.persisted()` ; if not yet, `persist()` once. Never on the TV, phones or remotes. The browser decides; nothing is shown when it says no.
- **Backups & Restore:** `afterHook("backupsDialog")` adds two rows to the last status box:
  - **Complete copy (pictures, videos, music):** "Saved HH:MM" (from `FS140.lastSave`), "Couldn't be saved — back up now" (`FS140.failing`) or "Not needed yet";
  - **Storage:** laptop app "Kept in the Mind It! folder on this computer"; Galaxy Tab "Kept in the tablet app's own files"; a browser "Protected — the browser keeps it" / "Not protected — the browser may clear it if space runs low, so back up regularly" / "This browser can't protect it — back up regularly".
- **Checked:** `fix141` (a browser that agrees, and one that refuses) and `fix141d` (laptop app: the folder line).

B · 2 · The complete copy keeps Question Database pictures once

- **How:** `fsRefSlots(g)` finds game pictures identical to `PROF.images[imgIdOf(v)]` and writes them into the state record as `imgref:<id>` (the pointer the quick save has used since build 111), restoring the real picture in memory straight after packing. They are no longer copied into the media record.
- **Coming back:** on Resume, `normalizeState` → `imgRefsResolve` (app99, since build 111) puts the picture back from the profiles file; `fsMergeMedia` also carries the pointers when a resumed game is filled in from the complete copy. Build 140 resolves the same pointers the same way, so a copy made by 141 opens in 140.
- **Checked:** `fix141` — a quiz with a Question Database picture: the media record no longer holds it, the state record has the pointer, and after a restart Resume shows the picture.

C · 3 · The release check saves and reopens a game in the tablet app

- **The page reports** (Android only, once each, `console.info` → logcat): "MINDIT_FS page android build N", "saved android state+media N chars" / "failed android …", and at start-up "found android state N chars, media yes/no" / "found android nothing".
- **The check** (`.github/scripts/apk_save_test.sh` , run by `publish-release.yml` after the launch check): opens the app, taps **Start New Game** through the accessibility tree, confirms the home screen has gone, waits for "saved", force-stops the app, opens it again and expects "found android state". It answers the emulator's "isn't responding → Wait", "Viewing full screen → Got it" and "Allow notifications?" interruptions.

- **ok** → a line in the release notes; **fail** → nothing is published; **skip** (the emulator couldn't drive the screen) → a warning, the release goes ahead.

- **Tried on build 140's published APK** (which doesn't report, so the expected result is a skip): five try-outs on build 140's APK (which doesn't report, so 'skip' is the expected answer): tries 1–3 stopped before the quiz page by the emulator's 'Process system isn't responding' dialog, Android's 'Viewing full screen' note and a quoting mistake in the script; tries 4 and 5 opened the quiz page (the home screen gone) and, as expected, found no report — try 4 also met the 'Allow notifications?' question, granted up front from try 5. In the browser, a fresh quiz page with a stand-in for the tablet's bridge saved its complete copy within 15 s without any tap and found it after a restart.
- **On the build-141 APK:** the release run on GitHub does it before publishing and adds its result to the release notes there; nothing is published if it fails.

D · 4 · The 60-minute soak plays a tiebreaker

- `stress110.py soak` : `SOAK_TIE_GAME` (default 2) levels the top two after the Final reveal; the tablet then plays ⚡ **Play Tiebreaker** to a winner, and the game record must show `suddenDeath` . In game 1 the tablet puts 📊 **Night Recap** on the TV (checked on the Audience window), then 🏆 **Winner Screen**.
- **Result:** 14/14 — 3 whole games to the winner screen in 56 minutes (game 1 with ⏱ One More Question and the 📊 Night Recap from the tablet on the real Audience window; games 2 and 3 with ⭐ Go to the Final; game 2's forced tie for first decided by ⚡ the sudden-death tiebreaker played from the tablet and recorded in Historic Games), Final wagers from the phones, 64 questions, 315 tablet actions, 0 false disconnects, no script errors. The first 60-minute run gave 13/14: the script pressed the tablet's next step (Show final standings) before the tablet had seen the forced tie, so no tiebreaker was played (the app recorded joint winners, correct for a skipped tiebreaker). Script fixed (wait, then ⚡ Play Tiebreaker), checked in a 6-minute run (8/8, tiebreaker won and recorded), then the whole soak was repeated

E · 5 · Two unused functions removed

- `remoteBoot` in app110 (the phone remote's start-up from build 121, superseded by app116's) and the `wordmark` assignment in app34 (superseded by app36's). Both were replaced before they could ever run; the code map had them in its reviewed baseline.
- **Now:** `tools/codemap.py --check` : modules 129 · changed functions 206 · hooks 240 · wrappers 296 · replacements 51 · issues 0 (new 0, known 0, fixed since baseline 0) · parse errors 0. The baseline is empty.
- **Checked:** `fix141` (the logo still comes from app36 and the phone remote page from app116) and the full run.

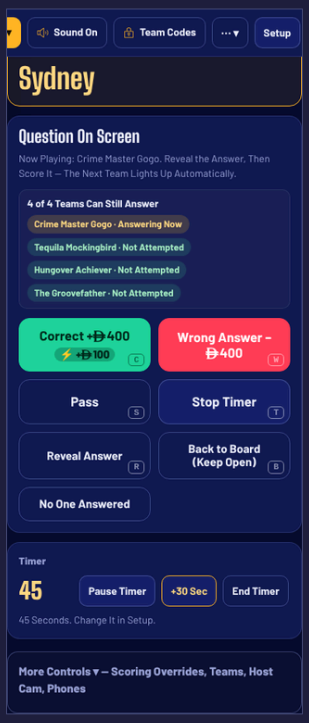
F · 6 · ⚑ Fix Later

- **The action:** `primaryActions` gains "⚑ Fix Later" (ghost, key **F**) on the question, answer, Final question and Final answer screens; "⚑ Flagged — Remove Flag" when the question is already flagged. Remotes (tablet and phone) get it through the same actions.
- **The record:** `PROF.fixLater` (newest first, at most 300): `{id,k,q,a,cat,where,quiz,gameId,libId,at}` , matched by `qKey(q)` , saved with `profSave()` (the players and Question Database file, so it is in backups too).
- **The list:** `afterHook("questionLibrary")` adds the "⚑ Fix Later (N)" tab while anything is flagged: question, answer, quiz · round · category · flagged date and time; ✎ **Edit** (`editLibQuestion`) when the question is in the database, otherwise ➕ **Add To Question Database** (new item, source "Fix later"), and ✓ **Fixed**.
- **End of the game:** a dock card "⚑ N question(s) flagged to fix" with ⚑ **Open Fix Later List** on the final standings, winner, awards and recap screens (flags from this game only).
- **Checked:** `fix141` (button, F key, toggle, the Final, saved and reloaded, the tab and its three buttons, the card) and `fix141d` (from the tablet and from the phone remote).

Build 141 · 6 · ⚑ Fix Later

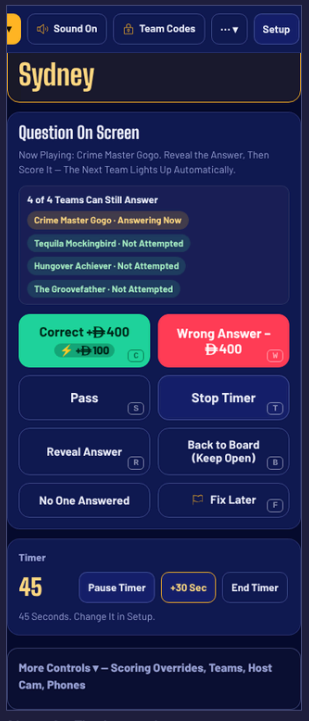
Spot a wrong question or answer during the game? Tap ⚑ Fix Later (or press F, or tap it on the tablet) and carry on. After the game the Question Database lists every flagged question to fix.

BEFORE · build 140

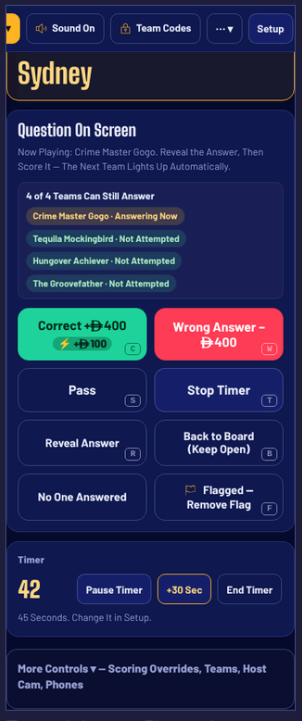


The question screen: no way to mark a question to fix — you had to remember it.

AFTER · build 141 — during the game

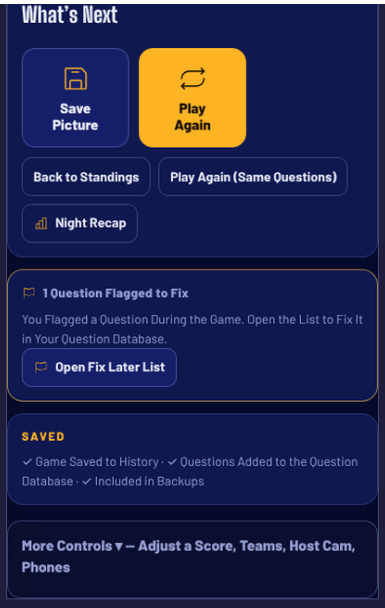


New ⚑ Fix Later button (keyboard F; also on the tablet).

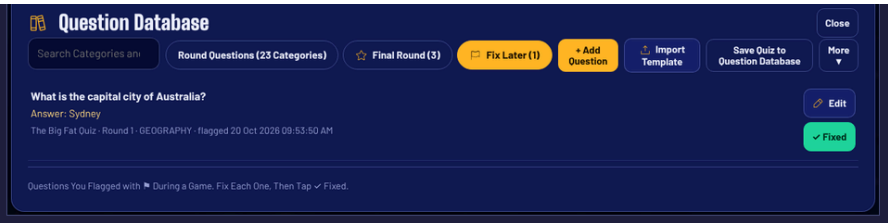


Tapped: it says Flagged — tap again to remove the flag.

AFTER · build 141 — after the game



End of the game: a reminder card with Open Fix Later List.



Question Database → ⚑ Fix Later tab: Edit the question (or Add To Question Database if it isn't in it), then ✓ Fixed.

G · 7 · Reveal before anyone has answered asks first

- **When:** `b141Early()` — the question screen, not resolved, a Daily Double or rotate (taking turns) mode, nobody has tried or passed, no buzz winner. Classic turns never ask (revealing first is how Classic works).
- **Laptop:** the "Reveal answer" action's `fn` asks (`ask` : "Reveal the answer now?" · Keep It Hidden / Reveal Answer) when the press came from this computer — a click event, or a key within 150 ms (capture-phase keydown, so **R** asks too).
- **Remotes:** a remote command runs `fn()` without an event, so the laptop doesn't ask. The link snapshot marks the action `early`, and the tablet remote asks with its own confirm (app25 and app92). The phone remote reveals at once, as before.
- **Checked:** `fix141` (click, R key, Keep It Hidden, after a wrong answer, Classic turns, a Daily Double, a remote) and `fix141d` (the tablet: cancel keeps it hidden, OK reveals it at once; the phone remote reveals at once).

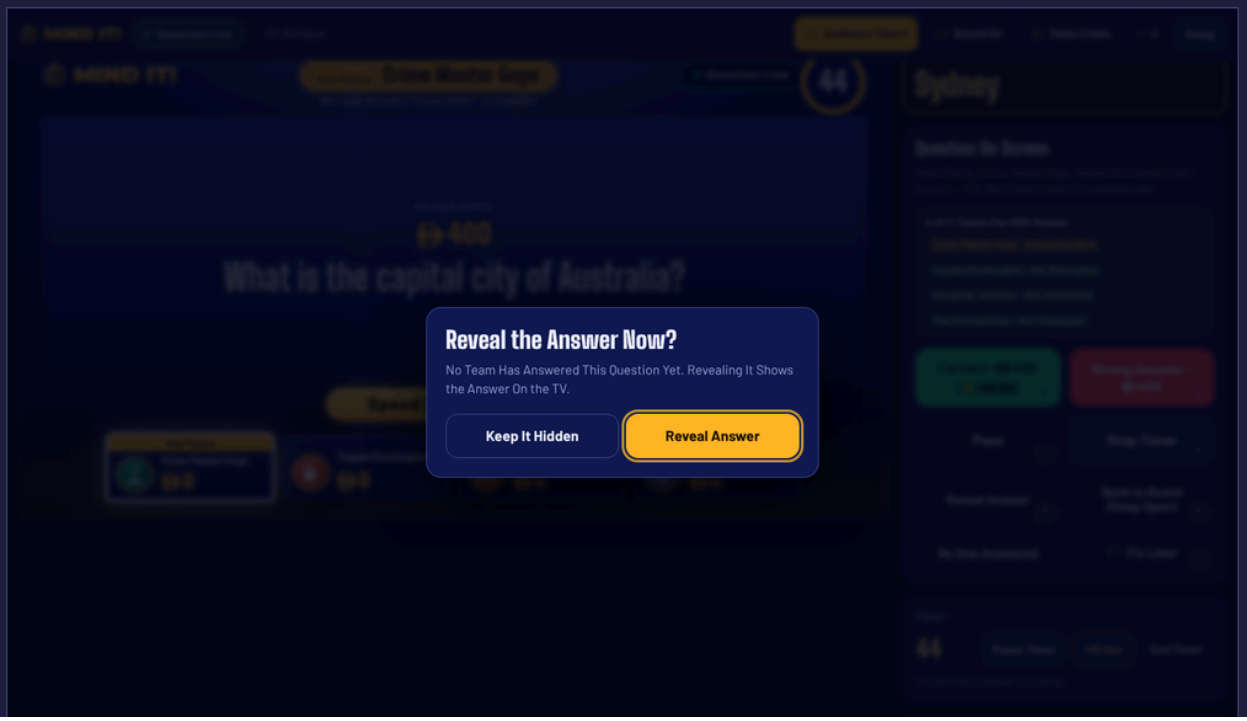
Build 141 · 7 · Confirm before an early reveal

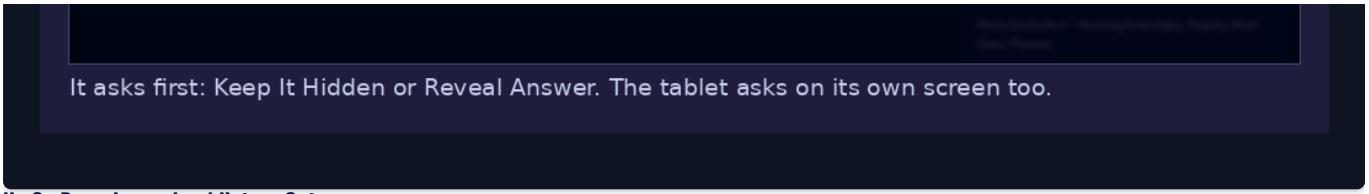
Revealing the answer before any team has answered (taking turns, or a Daily Double) now asks first, so a mis-tap can't spoil the question. Once a team has answered, Reveal works straight away as before. In Classic turns (reveal first, then score) nothing changes.

BEFORE · build 140

Reveal Answer (button or R) showed the answer on the TV at once, even if nobody had answered yet.

AFTER · build 141





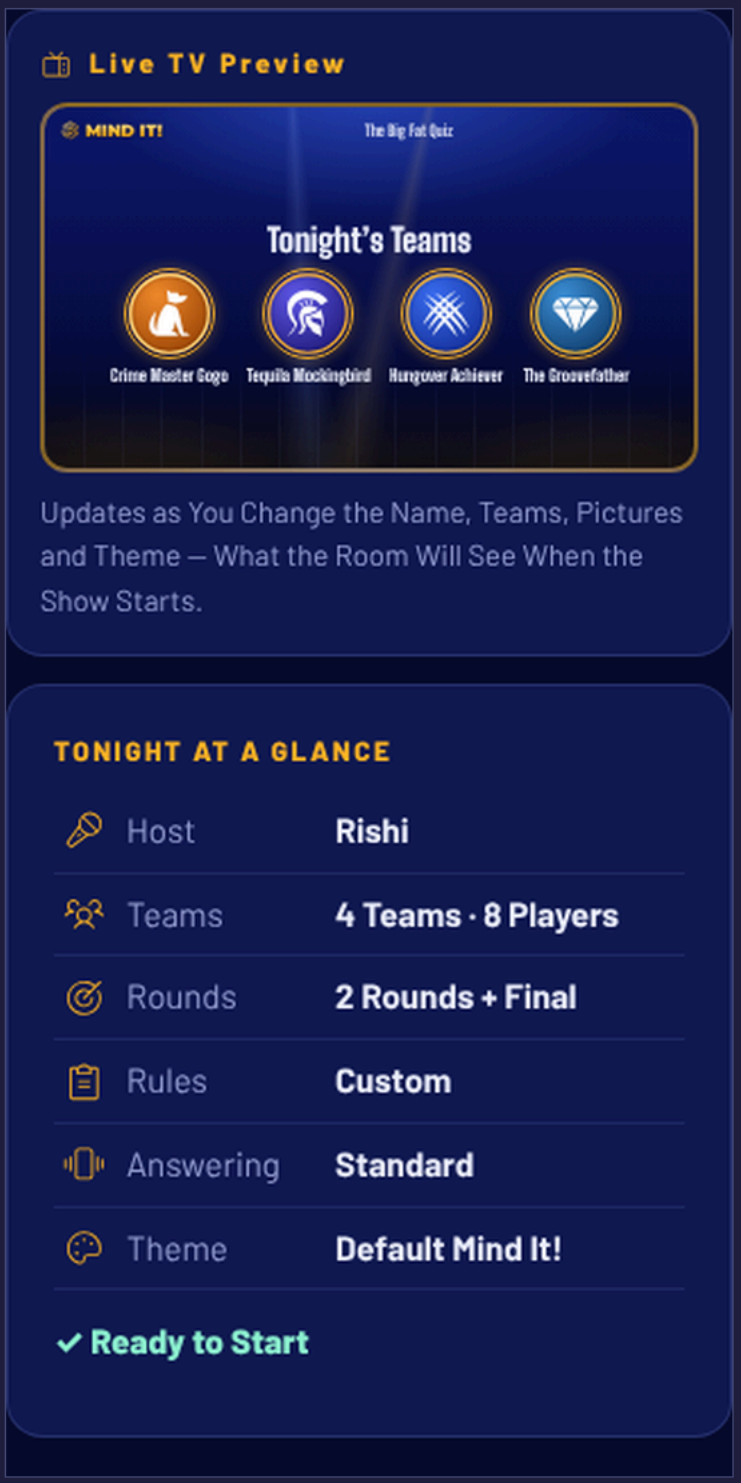
H · 8 · Pre-show checklist on Setup

- **Where:** `afterHook("arrangeSetup")` appends `.pre-check` to Setup's right-hand column; it is rebuilt every 3 s while Setup is on screen and no dialog is open.
- **Rows:** 📺 TV / Audience View (the Audience window, or TV screens on the Wi-Fi; **Open Audience View**) · 📱 Team phones and 🎛 Tablet remote (laptop app only; phones required only in phone-buzzer mode; 🔐 **Team Codes**) · 🔊 Sound (`audioStatus()`, the same check as Teams Presentation Mode; ▶ **Test Sound** plays the reveal sound) · ❓ Questions (empty questions in the rounds being played, and the Final; 📚 **Question Database**) · 💾 Backup file (made today; ⬇ **Back Up Now**) · 🛡 Storage (as in A).
- **The count:** "N of M ready" over the required rows; optional rows show "–".
- **Changed after the pictures:** the class names became `psc-*` — the first version's `.pc-row` collided with app18's Presentation Check. The action button moved under the value so the text isn't squeezed.
- **Checked:** `fix141` (the rows, an empty quiz, a browser that won't protect its data, questions filled in and Back Up Now, Test Sound, Open Audience View) and `fix141d` (the laptop app's phone and tablet rows; Open Audience View ticks the TV).

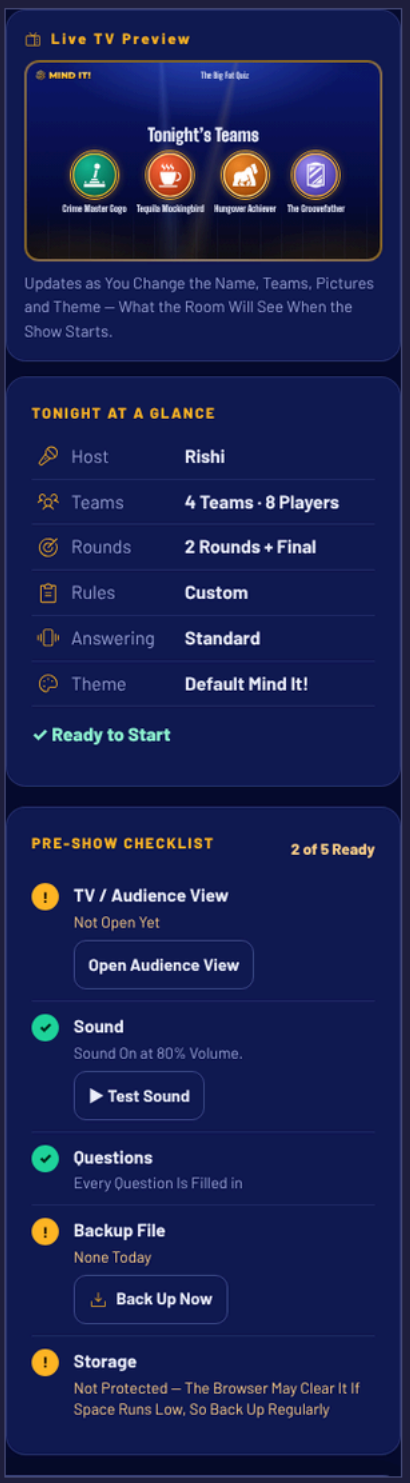
Build 141 · 8 · Pre-show checklist

On Setup, under Tonight at a glance: what's ready for the show, ticked by itself as things connect, with a button for each thing still to do. Team phones and the tablet only show when the laptop app is running (they're optional unless you use phone buzzers).

BEFORE · build 140 / AFTER · build 141



BEFORE: Tonight at a glance only.



AFTER: + Pre-show checklist — TV / Audience View, Sound (Test Sound), Questions filled in, Backup file made today

I · 9 · 📊 Night Recap on the TV

- **The screen:** `G.screen="recap"` from "📊 Night Recap" (ghost) on the winner, final standings and awards screens. `renderStage` draws `renderRecap` from `recapData()` (app90): title, date, host; champion or joint champions with score and players; up to 8 standings rows (ties shown "="); questions played, Daily Doubles, biggest swing, game time; up to four highlights (the tiebreaker first, then chips, then achievements); the footer. If drawing fails it falls back to the winner screen and logs the error.
- **Laptop actions on it:** 🏆 Winner Screen · Back to standings · 💾 Save Standings Picture · 🎲 Play Again (New Questions). `gameOverNow()` is true on it, so phones and remotes see the game as over; `nowTitle` / `nowHint` describe it.
- **Layouts:** landscape and portrait (`@container (orientation: portrait)`), 4 to 8 teams (`rc-many` above 6). On an upright screen the champion and the whole table always show (`grid-template-rows:auto minmax(0,1fr)`), and the highlights get the rest: `b141FitOne` leaves out any highlight that doesn't fit whole, checked again by a `ResizeObserver` whenever the box changes size (TV window resized, Present mode) and once the fonts are in.
- **Checked:** `fix141` (1920×1080, 1080×1920, 1280×720 and 800×1280 with 8 teams, a two-line champion name and four highlights: every row shown and clear of the numbers, no highlight cut or under the footer, nothing outside the TV picture; a game saved on it reopens on it), `fix141d` (from the tablet and the phone remote, on the real Audience window) and `avaudit` (new scenarios `recap_4` and `recap_8`).

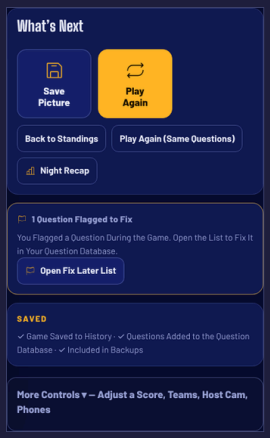
Build 141 · 9 · Night Recap on the TV

After the game, 🏆 Night Recap shows the whole night on one TV slide: the champion, the final table, the night's numbers and its highlights. It's a button on the standings, awards and winner screens; nothing else changes.

BEFORE · build 140

The TV ended on the winner screen; the recap existed only as a saved picture.

AFTER · build 141



Laptop: 🏆 Night Recap on the winner screen.

TV (landscape).

TV (portrait).

J · Tests

- **New suites:** `fix141` (browser) 49/49; `fix141d` (laptop app + tablet remote + phone remote) 12/12. `run_all.py` now has 146 suites.
- **Full run** on the final page (md5 `a2689b23…`): full run on the shipping page (md5 `a2689b23…`): 146 suites, 3,264/3,264 checks, all on the first try (101 min).
- **Layout audit** (`avaudit`): 270 screens at 9 sizes (including the new `recap_4` and `recap_8`), 0 issues on the final page. The first audit (on the page of the first full run) found 12 overlaps, all in `recap_8` at 800×1280 — fixed (see K)
- **Stress phases** (`stress110.py` , laptop app, 8 teams, 10 phone pages, a tablet remote): idle 2/2, net 6/6, load 4/4, lock 2/2, rapid 4/4, health 6/6 — 24/24 (laptop app, 8 teams, 10 phone pages, a tablet remote)
- **Guide:** 7 new pictures (the Pre-show checklist, the ⚑ Fix Later button and Question Database tab, the early-reveal question, the Night Recap on the TV, the winner screen's What's Next card with the Night Recap and the Fix Later card, the Backups & Restore storage lines); new text in Step 12, 6.4, 15.3, 15.10, 15.12, 19.1 and Chapter 22; a "New in build

141" list at the front; the cheat sheet gains ⚑ Fix Later (F) and 📊 Night Recap; the Complete Idiot's Guide is now 168 pages

- **Reproducible build:** the final page rebuilt from a fresh clone of GitHub `main` is identical: identical (SHA-256 0a424588…f795, commit d989f44); the tablet APK built from a fresh clone with `build_apk.sh` passes the certificate and manifest checks: yes (versionCode 141, v2 + v3, certificate 5f4819e4…; the APK built with the private build.sh from the same files has identical contents).

K · Problems found and fixed while building

- **The Night Recap on an upright screen with many teams (found by `avaudit` on the final page, after the first full run).** At 800×1280 with 8 teams and a two-line champion name, standings rows 7–8 sat under the four numbers and the footer on the last highlight (12 overlaps; every other size was clean). `fix141`'s size check had passed because it looked only for things outside the TV picture, with no highlights in its game. Fixed in `style.css` and `app135.js` as described in I; `fix141` now also checks 800×1280, overlaps and cut highlights, with a long name and four highlights (on the earlier page it fails at 1080×1920 and 800×1280). The recap screens were audited again at all 9 sizes (18 screens, 0 issues), and the full run was repeated on the fixed page.

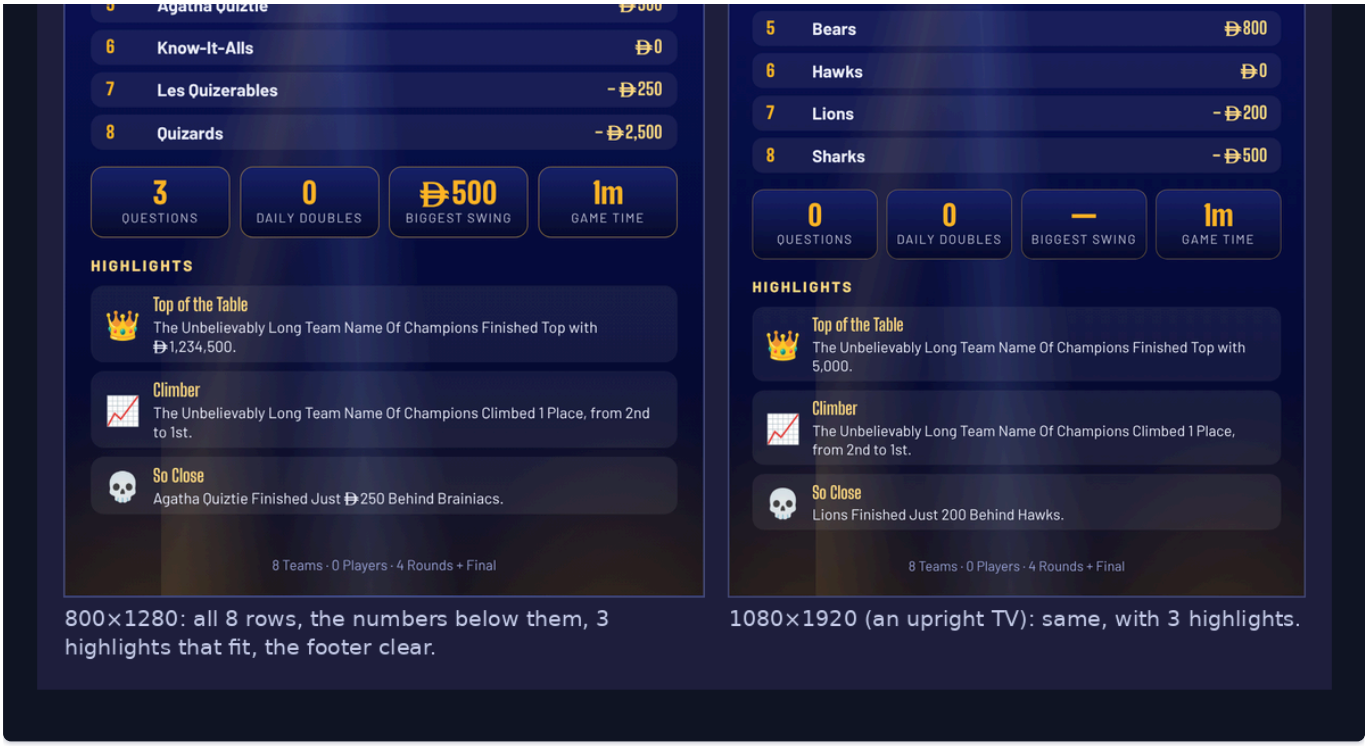
Build 141 · Night Recap, upright screen, 8 teams

Found by the layout check on the final page: on an upright tablet-sized screen (800×1280) with 8 teams and a long champion name, the standings ran under the numbers and the footer sat on the last highlight. Now the champion and the whole table always show; a highlight that doesn't fit whole is left out instead of being cut. Landscape TVs are unchanged.

BEFORE · this morning

800×1280: rows 7 and 8 hidden under Questions / Daily Doubles; the footer on top of 'Comeback Kings'.

AFTER · now



800×1280: all 8 rows, the numbers below them, 3 highlights that fit, the footer clear.

1080×1920 (an upright TV): same, with 3 highlights.

- **The soak didn't play its forced tiebreaker the first time.** After forcing the tie in game 2 the script pressed the tablet's next step — Show final standings, from a state that didn't show the tie yet — so the tiebreaker was skipped and the game recorded joint winners (right for a skipped tiebreaker). Fixed in the script (wait for the tablet, then ⚡ Play Tiebreaker); a 6-minute run played and recorded it (8/8), then the whole 60-minute soak was repeated (see D).
- **One old check expected build 140 exactly** (`fix140s` , G · a build-139 save opens in 140). It now accepts 140 or later, like `fix140` .
- **The checklist's first styling** reused `.pc-row` , already used by app18's Presentation Check, which boxed every row. Seen in the before/after pictures; renamed to `psc-*` .
- **The checklist's buttons squeezed the text** at the narrow Setup width. Moved under the value; the emoji were taken off the button labels.
- **The emulator save check** first gave "skip" three times: the emulator's "Process system isn't responding" dialog, Android's "Viewing full screen" note, and a shell quoting mistake in the script. All three are handled; it now also confirms the quiz page really opened, and keeps a screenshot every time.
- **The docs and the code disagreed** in four places (the code is the source of truth):
  - the master context's "Latest build" line still said versionCode 111 (and "it is 111 now") — the code is at 141;
  - the master context still listed the Undo pop-up, Go to the Final, One More Question and the sudden-death tiebreaker (all built in 140), and now the early-reveal question, ⚑ Fix later, the pre-show checklist and a recap TV slide (141), as "not approved" ideas — moved to "built";
  - "Mac (dropped)" in the master context and the changelog — the Apple Silicon app has been built for every release since build 123; corrected to "Intel dropped";
  - the status file's open question 3 still said the web link is republished every alternate build — since 18 Oct it is every build.

Build 140 addendum (19 October 2026)

What you asked for (19 Oct, 10:15): "3,4,5,6" from the list offered after build 139:

- 3: the full-game soak test;
- 4: refresh the Complete Idiot's Guide pictures;
- 5: the Final reveal on portrait screens with 8 teams;
- 6: the four ideas never approved before — the Undo pop-up, Go to the Final, "time for one more question" and a sudden-death tiebreaker.

Your answers for the tiebreaker: "Quizmaster's choice", "Out loud, Quizmaster marks", "Random from my Question Database". **For Go to the Final:** "Round summary, then Final".

Approval: before/after pictures of all five were sent at about 11:10, and you answered "OK all five". Estimated time given: about 7–8 pm Dubai.

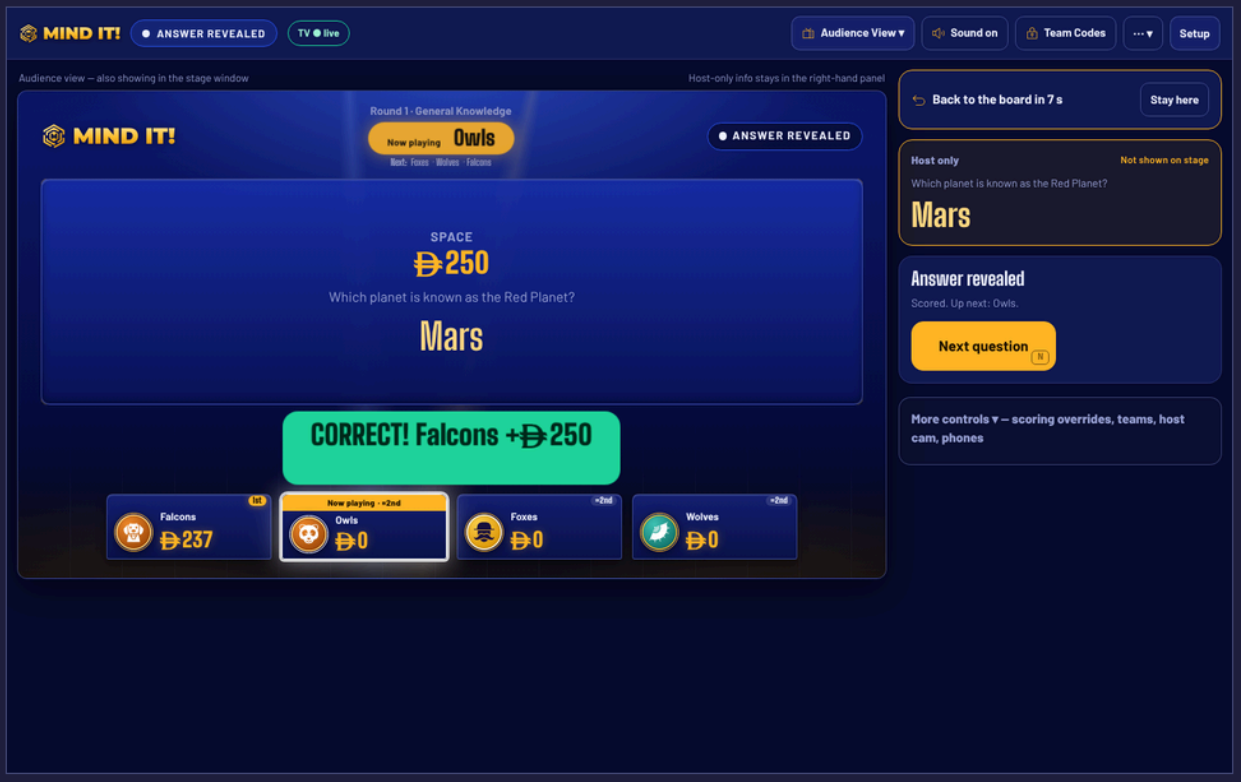
A · ↩ Undo pop-up

- **How it works:** after every score (`applyScore`), the Quizmaster's page shows the journal entry's label at the bottom centre, with **↩ Undo**, for 8 seconds. Undo calls `doUndo` on that entry directly, with no `ask` dialog.
- **When it goes:** the next score replaces it, and it disappears once its entry is no longer in the journal (undone elsewhere, reset, new game). It is hidden in Present mode (`body.present`). It is never added on the TV, team phones, phone remote or tablet remote pages.
- **Changed after the pictures:** the pop-up no longer waits while the mouse is over it. A pop-up that appeared under a resting mouse would never have gone away.
- **Checked:** `fix140` , 78/78.

Build 140 · 1 · Undo pop-up

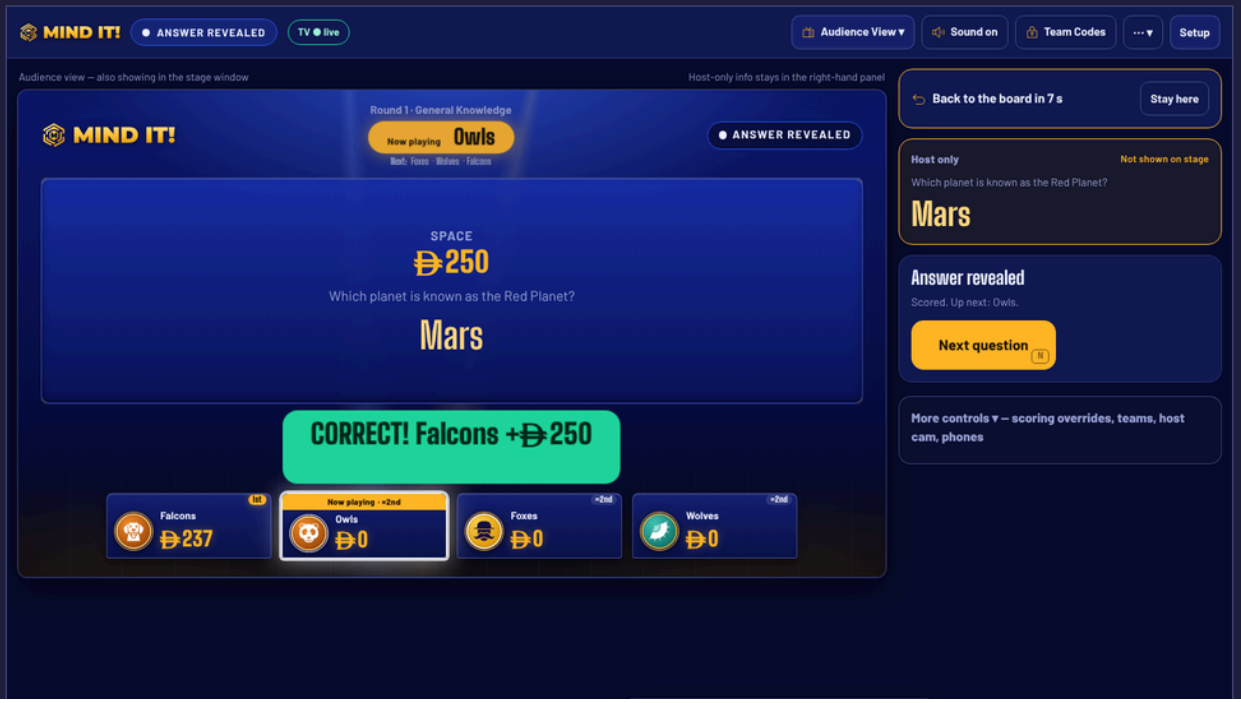
After every score, the Quizmaster's screen shows what was scored with an Undo button for 8 seconds. One tap undoes it, with no question asked. The old Undo button and the tablet's Undo are unchanged.

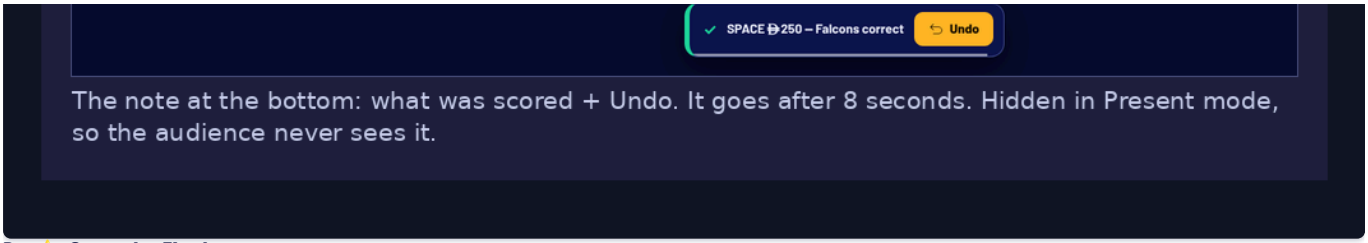
BEFORE · build 139



Falcons scored. To undo: More controls → Undo last score → confirm.

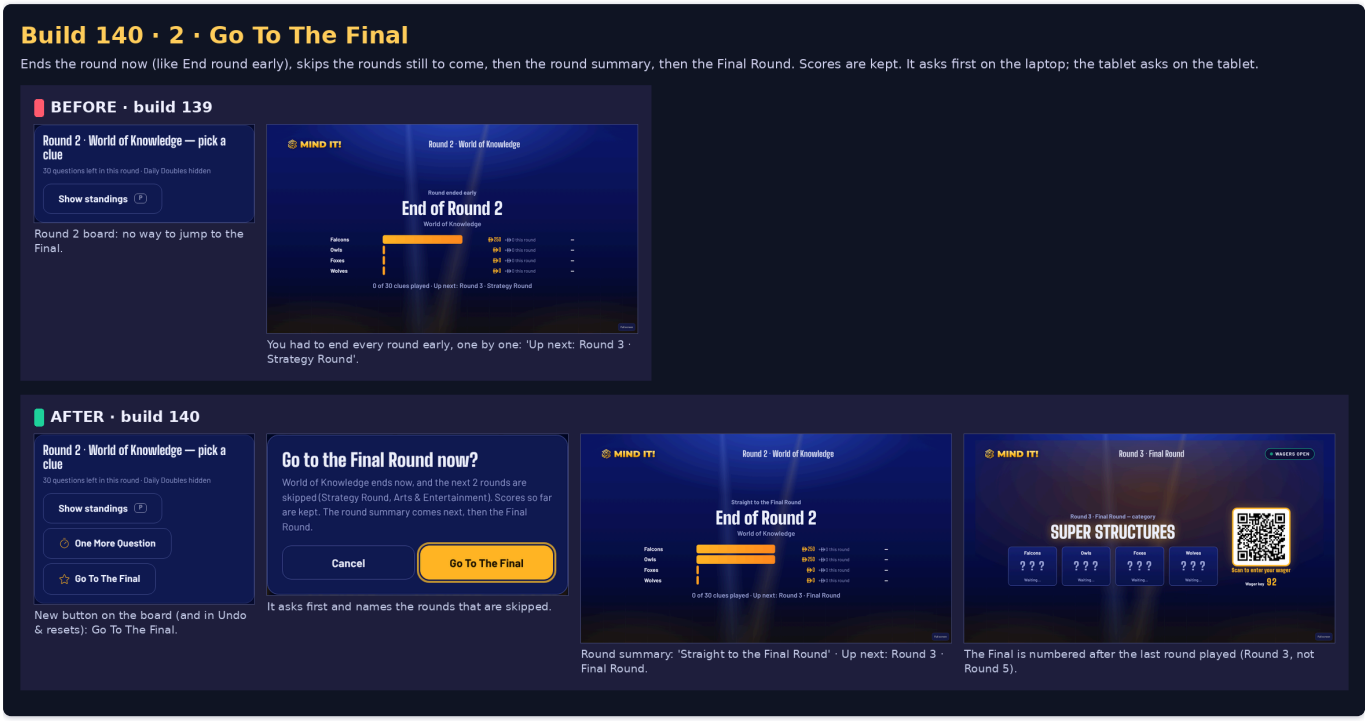
AFTER · build 140





B · ⭐ Go to the Final

- **Where it is offered:** a ghost action on the board, and a button in Undo & resets next to End round early. Only when `plan().final` is set and rounds are still to come.
- **Asking first:**
  - A laptop click asks: the dialog names the skipped rounds and says scores are kept and that an open question or wager is closed.
  - A remote runs it straight away (no click event). The tablet asks with its own confirm: the `^⭐ Go to the Final` rule is added to both confirm rules.
- **What it does:** sets `G.toFinal={from,at}` and calls `goRoundSummary(true)`. On the round summary:
  - the next step is "Go to the Final Round" and `advanceRound` goes straight to `startFinal`;
  - "Up next" names the Final, and the TV kicker says "Straight to the Final Round";
  - `roundLabel("final")` becomes "Round N · Final Round" with N = the last round played + 1.
- **Undoing it:** Resume cancels it, and so do a new game and `enterRound`.
- **Checked:** `fix140`.



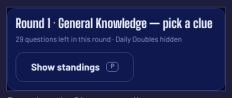
C · ⏱ One More Question

- **Arming it:** `G.lastQ={ri,ref}`. The next clue opened, or the one on screen ("Make This the Last Question" in Undo & resets), becomes the last one.
- **Ending the round:** once that clue is played and the board would come back (`backToBoard`, `closeClueNoScore`, or any render on the board), `goRoundSummary(true)` runs. The kicker reads "That was the last question".
- **On screen:** the TV shows "⏱ LAST QUESTION OF THE ROUND" in place of the Next strip. The laptop hint says so too. The tablet's big button says "🏁 Round Summary →".
- **Limits:** Back to board (keep open) leaves it armed. Cancel Last Question clears it. It is not offered with one question left.
- **Checked:** `fix140`.

Build 140 · 3 · One More Question

Tells the game the next question is the last of the round. After it, the round summary comes up by itself. The TV says so at the top. You can cancel it until then.

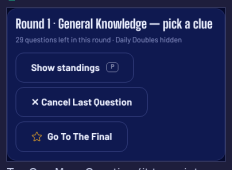
BEFORE · build 139



Board: only Show standings.

The TV doesn't know time is up; you end the round early after the question.

AFTER · build 140



Tap One More Question (it turns into Cancel Last Question).

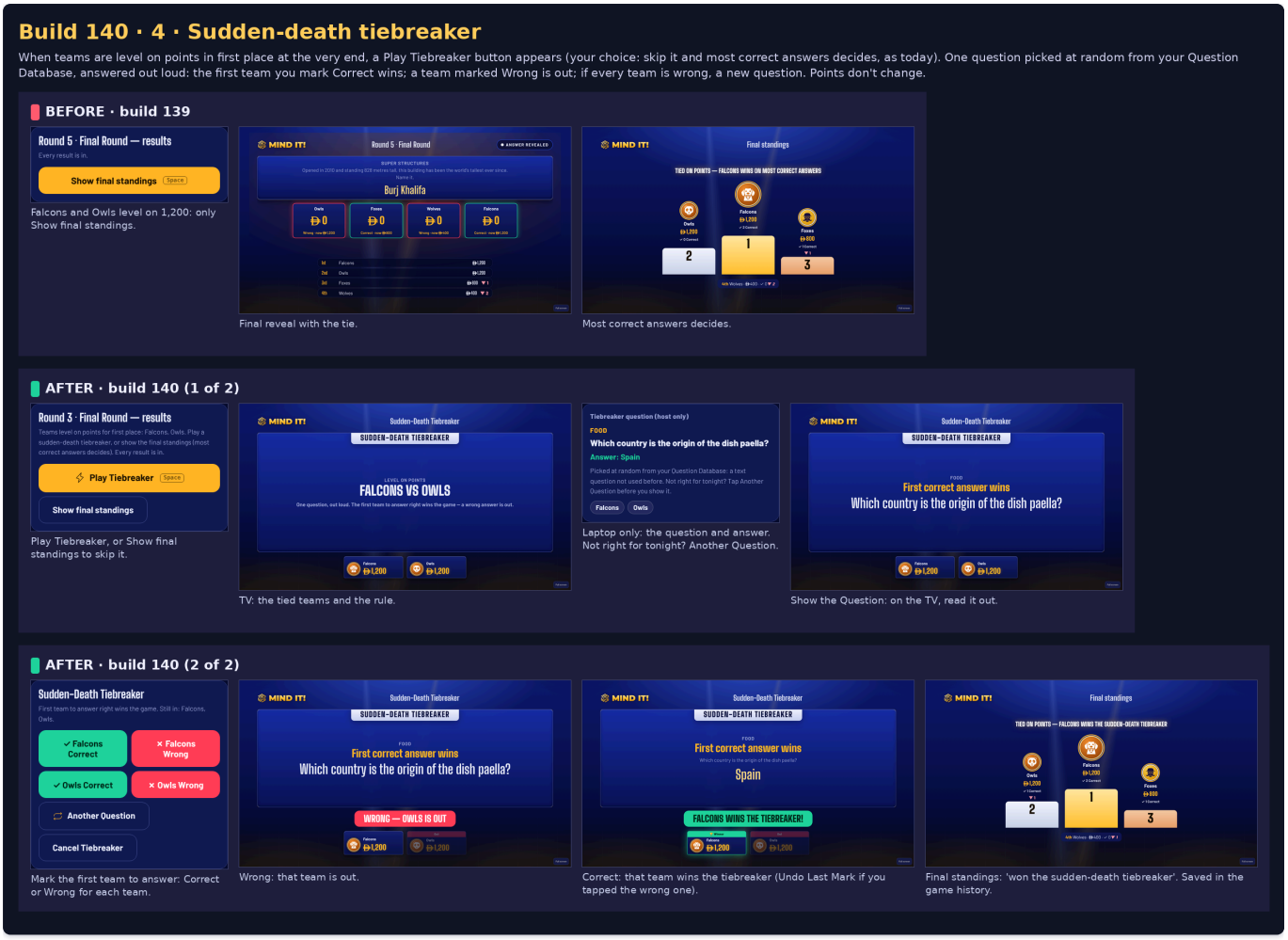
The TV: 'LAST QUESTION OF THE ROUND' under the team in play.

Close-up of the TV.

After that question: the round summary, 'That was the last question'.

D · ⚡ Sudden-death tiebreaker

- **When it is offered:** only when at least two teams that aren't marked away share the top score, and either the Final reveal is complete or this is the last round summary with no Final. ⚡ Play Tiebreaker is the primary action, and "Show final standings" (skip) stays.
- **Choosing the question:** a random library item that is a text question (no picture, video, audio or choices), not in this game's rounds or Final (by `libId` and by question text), and not already asked in this tiebreaker. Questions never used in earlier games come first. If none is left, the Quizmaster asks one of their own.
- **Screens:**
  - **TV:** a new screen `tiebreak` with the ribbon, the tied teams' scoreboard only, and per phase the rule, the question, or the answer with a banner.
  - **Laptop:** a host-only question/answer card, plus Correct and Wrong for each team.
  - **Tablet remote:** a tiebreaker card with the question and the answer behind a tap, and a Correct / Wrong row for each team.
- **Ranking:** no points change. The winner ranks first among the teams level with it, through wrappers on `rankTeams`, `cmpTB`, `champions`, `tbText`, `tbLine` and `tbPlaces`. They apply only while the winner still holds the top score and shares it.
- **Records:** the game record gets `suddenDeath`, the winner's `tbWon` (so a history edit's `reRank` keeps it) and the qKeys of the tiebreaker questions shown.
- **Saved games:** a game saved during a tiebreaker reopens on it. A saved `tiebreak` screen with no tiebreaker state falls back to the Final reveal.
- **Checked:** `fix140`, including the 💾 standings picture line "Tiebreak: won the sudden-death question" and the tablet card.



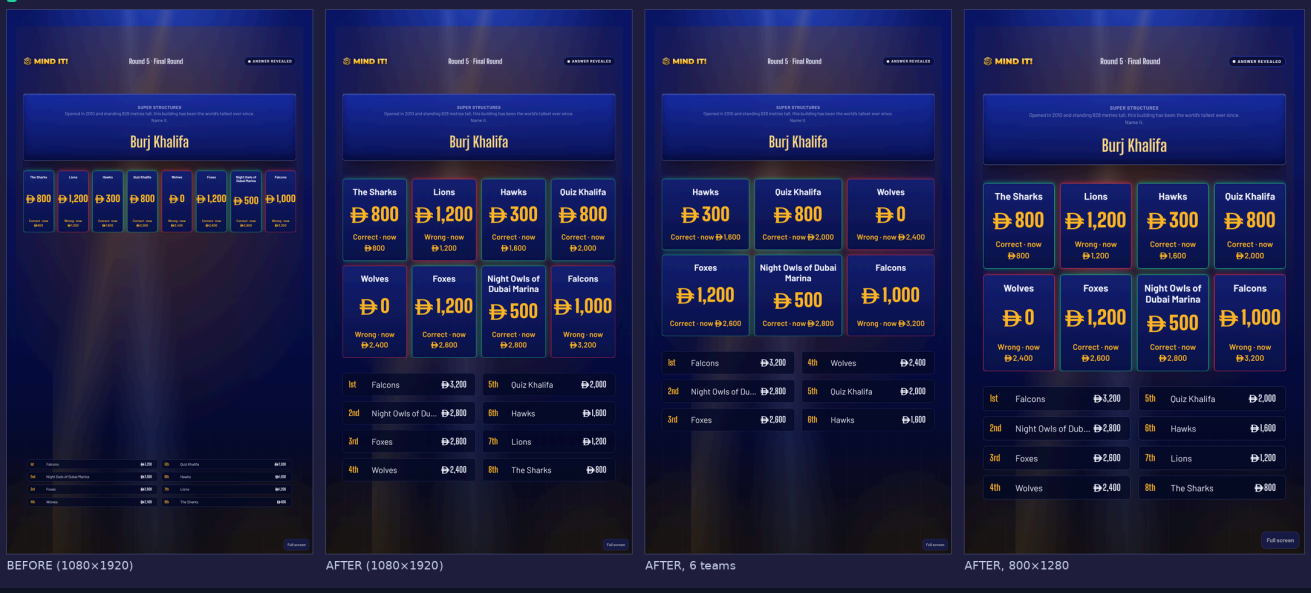
E · Final reveal on a portrait screen, 6–8 teams

- **Before:** at 1080×1920 with 8 teams, the build-139 one-row rule (`fr-one`) squeezed the cards to 108 px wide. The standings were absolutely placed at the bottom, with about 800 px of empty screen between them.
- **Now:** a container query (`orientation: portrait`) for `.sx.fin-look.fr-two` :
  - the cards wrap in rows of four (three for six teams), centred, about 227 px wide at 1080×1920;
  - the standings sit in the grid flow straight under them, with larger rows.
- **Landscape:** unchanged.
- **Checked:** `fix140` at 1080×1920 and 800×1280 (two rows, standings within 80 px of the cards, nothing overlapping or off screen), and landscape 1920×1080 (still one row).

Build 140 · 5 · Final reveal on a portrait screen, 8 teams

With 6–8 teams on a portrait (upright) TV, the answer cards were squeezed into one row and the standings sat tiny at the very bottom, with a big empty middle. Now the cards go in rows of four, larger, with the standings straight under them. Landscape screens are unchanged.

BEFORE · build 139 / AFTER · build 140



BEFORE (1080×1920) | AFTER (1080×1920) | AFTER, 6 teams | AFTER, 800×1280

F · The soak plays whole games (test only)

- **What changed in `stress110.py soak`:**
  - Every game is now driven to the winner screen. Each team sends its Final wager from its own phone page (the wager pad: a quick pick, then Lock In). The tablet marks every Final answer and reveals them. If a tiebreaker comes up, the tablet plays it.
  - Then each game is checked: the winner on the TV; the record in Historic Games with the same positions as the final standings; the record in the laptop's `player-profiles.json`; and the recap card, the Historic Games recap and the standings picture.
  - Reading time is now 15–35 s per question (was 20–45 s).
  - So that whole games fit the hour, game 1 uses ⏱ One More Question late in Round 2, and game 2 uses ⭐ Go to the Final after six questions.
- **Result:** 12/12 — 3 whole games to the winner screen in 55 minutes (game 1 with ⏱ One More Question, games 2 and 3 with ⭐ Go to the Final), Final wagers from the phones, every game record and picture checked, 67 questions, 310 tablet actions, 0 false disconnects, no script errors

G · Guide pictures

- 22 pictures refreshed and 7 new (5 for the four new buttons, 2 for the complete save: the new line in Undo & resets and the red card); section 22.1 explains the complete save; the Complete Idiot's Guide is now 165 pages

H · Tests

- **Full run** on the final page (md5 `0dc63aea…`), including the four new audit suites: full run on the shipping page (md5 `0dc63aea…`): 144 suites, 3,188/3,188 checks, all on the first try (98 min).
- **Layout audit** (`avaudit`): 252 screens, 0 issues (253/253)
- **Stress phases** (`stress110.py`, laptop app, 8 teams, 10 phone pages, a tablet remote): idle 2/2, net 6/6, load 4/4, lock 2/2, rapid 4/4, health 6/6 — 24/24 (laptop app, 8 teams, 10 phone pages, a tablet remote)

I · Not tried for real

- The Galaxy Tab itself, a Windows PC, a Mac and real smart TVs. The page is the same on all of them; the laptop app, the tablet remote page and the phones were tested on the build machine.
- The tablet's new save files (`gameWrite` / `gameRead`) were compiled and their page side tested against a stand-in bridge; the Java code itself has not run on a real tablet.

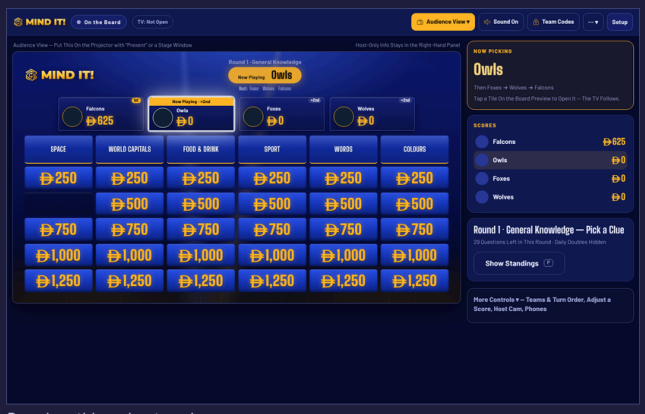
J · Code audit — your approval (19 Oct, 12:50) and how it was done

You approved all four improvements from the code audit (sent 12:34), to be built into **this build 140** — same build number, no separate build or release. Your instructions: don't stop the work in progress; ask before any change with substantial risk; run the complete regression afterwards, including laptop, Android, phone and TV connectivity; don't publish until everything is verified. No change needed a risk stop. One visual change, the red "isn't completely saved" card, was sent as before/after pictures at about 14:05 (you chose option 1 at 14:09: the card plus the one-time message, as built).

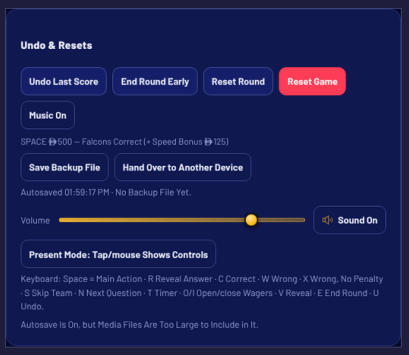
Build 140 · Code audit 1 · Complete save

A quiz with pictures, videos, music or team photos is now also saved complete (in the laptop app's folder, the tablet's storage and the browser's database), so they come back after a crash or restart. You only see something new in two places:

BEFORE · build 139

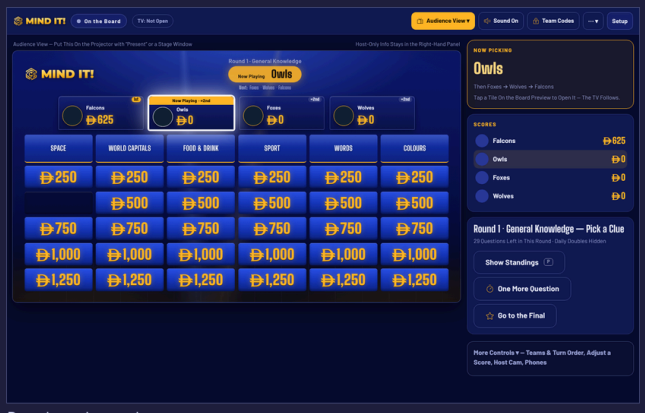


Board: nothing about saving.

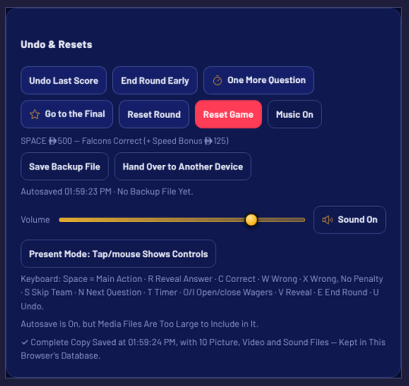


Undo & resets: 'Autosave is on, but media files are too large to include in it' — after a crash the game came back WITHOUT its pictures, videos and music.

AFTER · build 140 · saved normally (almost always)

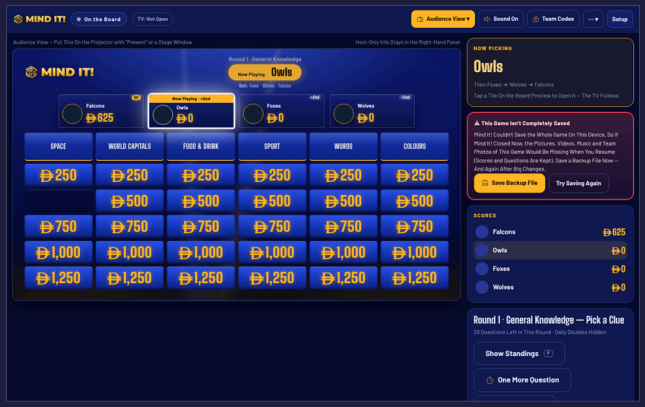


Board: unchanged.

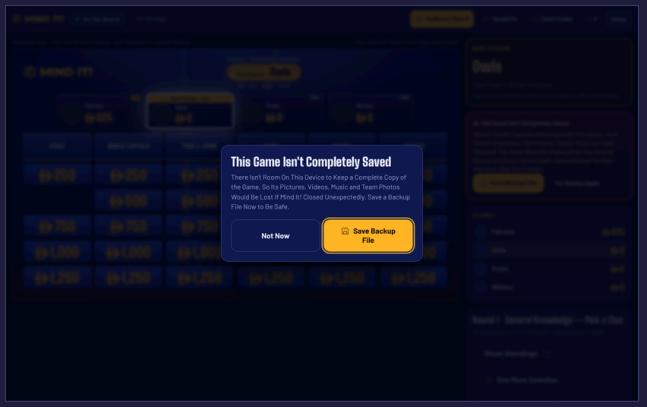


Undo & resets: one new line — '✓ Complete copy saved at …, with 10 picture, video and sound files — kept in …'.

AFTER · build 140 · only if there is no room anywhere for the complete copy



A red card under Now Picking: what would be missing after a crash, with Save Backup File and Try Saving Again. It goes away by itself once a complete copy is saved.



Shown once per game as a message too (never in Present mode).

K · Audit 1 — the complete save (`app134.js` , `main.js` , `MainActivity.java`)

- **The gap:** `saveNow` (app1) keeps the game in localStorage (about 5–10 MB). A quiz with media that doesn't fit got the cut-down "lite" save: question pictures and videos, music, intro video, team photos and Final media set to null. After a crash, those were gone. Profiles, the Question Database and history were never affected (separate stores, app126).
- **Now:** after every quick save (debounced 1.5 s, at most 6 s), a complete copy is written as two records:
  - **media:** every embedded `data:` string of 8 KB or more, found anywhere in the game, keyed by its path;
  - **state:** the game with a marker at each path.

  - Media are written first, and only when they change (signature check), so a state never points at media not yet stored.
- **Where the copy goes:** every place the device has.
  - **Galaxy Tab:** `MindItAndroid.gameWrite/gameRead/gameClear` : files `game-complete-state.json` and `…-media.json` in the app's storage, same pattern as `profWrite` . Out of memory returns false instead of closing the app. Above 40 M characters, the browser database takes it.
  - **Laptop app:** `/api/host/fullsave` , host only, written atomically with fsync, previous copy kept as `.prev` . The files don't depend on the port.
  - **Every browser:** IndexedDB `mindit-game` with strict durability.
- **Start-up (`fsRecover`):** the newest copy that opens is used; damaged copies are passed over.
  - **Waiting to Resume:** a game offered for Resume is replaced by its complete copy when the copy is the same game (`saveId` , or `gameId` for saves from build 139) and at least as new. If the cut-down save is newer, the copy's media are put into it.
  - **Setup:** a Setup quiz loaded from a cut-down save is reloaded complete.
  - **Already resumed:** a game already resumed from a cut-down save gets its media back.
  - **No quick save at all:** if the quick save is gone but a complete copy exists, Resume is offered.
  - **Never overwritten:** the fresh Setup shown while a game waits for Resume is never copied over it (like app103's copy).
- **Warning:** only when no complete copy could be stored **and** the quick save is incomplete. It shows as a red card straight under the panel's first card, with 💾 Save Backup File and Try Saving Again, plus a message once per game (never in Present mode). Undo & resets says where the last complete copy went.
- **Start fresh:** the Safety Backup holds the complete game, and app114's cut-down one is skipped, so there is one backup, not two. New quiz while a game waits for Resume keeps that game's copy.
- **Checked:**
  - `fix140s` 36/36 (browser + tablet bridge): a SIGKILLed browser restarted; Setup reload; resumed without media; a state record cut short; media cut short; media newer than state; no media record; the browser database at its quota (100 KB) → card + message, then quota back + Try Saving Again; localStorage completely full; Start fresh; build 139 → 140 → 139 → 140 saves; the tablet's files, a restart, and a refused write.
  - `fix140sd` 19/19 (laptop app): SIGKILL and restart on another port (8765 taken), a state file cut short (.prev used), the data folder refusing writes, a phone address refused (403), bad requests (400), Start fresh.

L · Audit 2 — reproducible builds

- **`build.py` :**
  - no machine paths: inputs next to `build.py` , `--out` / `MINDIT_OUT` (default `dist/mindit.html`);
  - UTF-8 with `\n` line ends;
  - team badges in `assets/team-icons` ;
  - gzip without a timestamp;
  - the tightening step uses `node_modules` from `npm ci` and stops the build if it fails.
- **Locked tools:** `package.json` + `package-lock.json` (terser 5.51.2, csso 5.0.5, acorn 8.19.0).
- **`tools/verify_build.py` :** clones, runs `npm ci` and `build.py` , and compares fingerprints.
- **Laptop and tablet apps in the repository:**
  - `desktop/` : `main.js` , electron-builder settings with exact versions + lock file, `build_desktop.sh win|mac` ;
  - `android/` : Java, manifest, resources, home page, checks, `build_apk.sh` . The passwords come from the environment only; it never creates a keystore and stops unless the certificate is `5f4819e4…637b` ;
  - `.gitignore` keeps out keystores, `*.jks` , `release.pass` and `build.sh` .
- **Checked:**
  - two builds of the same commit are byte-identical;
  - the final page rebuilt from a fresh clone of GitHub `main` is identical: identical (SHA-256 979cf55b…3248, commit 09a4684);
  - the tablet APK built from a fresh clone with `build_apk.sh` passes the certificate and manifest checks: the release APK itself was built from a fresh clone (commit 09a4684): original certificate 5f4819e4…637b, v2+v3, apk_check OK, its page = the tested page, and its 22 entries are identical to the same build through the private build.sh;
  - the release laptop app and APK were built from the repository's `desktop/` and `android/` files: the EXE's and the Mac app's main.js equal desktop/app/main.js; the APK came from android/ in the clone.

M · Audit 3 — secure random codes

- **What uses it:** `secureRandom()` (app1) uses `crypto.getRandomValues` , with Math.random only if that is missing. It feeds:
  - `uid` and team codes;
  - the game key (`newGameKey` , `newState` , app12);
  - the remote PIN, the TV id, the tablet id and the Safety Backup id.
- **Formats unchanged:** 4 letters from `ACDEFHJKMNPRTUVWXY` , a 4-digit PIN, a 2-digit key.
- **Servers unchanged:** the laptop server already used `crypto.randomBytes` and the tablet server `SecureRandom` . Rate limiting and trusted devices are unchanged.
- **Checked:** `fix140r` 14/14 — calls counted; a zero-forced generator proves every value follows it; formats; the spread over 36,000 letters and 18,000 keys; the fallback when the generator is missing or throws; no Math.random in either server.

N · Audit 4 — code map and lint

- **The tools:** `tools/codemap.py` + `tools/codemap.js` parse the page's 128 source files in page order (acorn).
- **What it maps:** every hook, wrapper, replacement and temporary override (`CODE_MAP.md`): modules 128 · changed functions 205 · hooks 235 · wrappers 291 · replacements 53 · issues 2 (new 0, known 2, fixed since baseline 0) · parse errors 0.
- **What it flags:**
  - duplicate declarations (hoisting: the last one wins everywhere);
  - replacements that drop earlier wrappers or hooks, at load time or while the page runs;
  - double replacements;
  - wrappers that run before their function exists;
  - hooks on missing functions;
  - names defined nowhere, unless guarded by `typeof` (inside `try` they are marked as failing silently).

- `--check` : fails on anything not in the reviewed baseline.
- **Found on today's code:** no duplicate declarations, dropped wrappers or undefined names. Two double replacements:
  - `remoteBoot` : app110 is superseded by app116;
  - `wordmark` : app34 is superseded by app36.
  - The earlier versions are dead code. They were left in place (no refactor) and recorded in the baseline with the reason.
- **Checked:** `lint140` 12/12 — the page's sources, plus made-up modules containing each kind of problem, all found, with no false alarms.

0 · Problems found and fixed while integrating

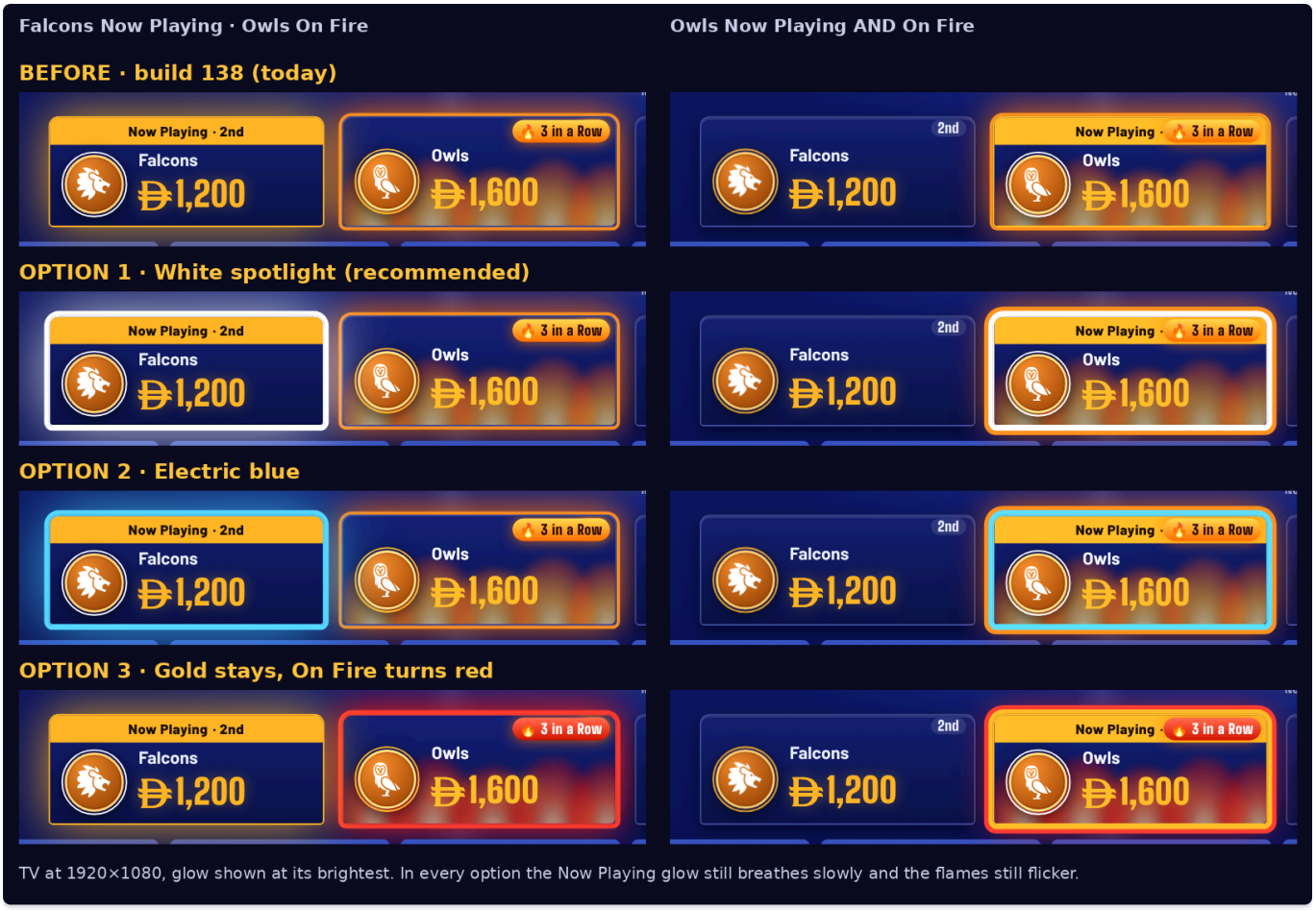
- **The complete copy could be overwritten while a game waited for Resume.** At start-up, the laptop shows an empty Setup while it offers the saved game, and that Setup is saved. Found while testing a killed browser. Fixed: no complete copy is written while Resume is offered, the same rule app103 already used for its copy of the save.
- **The quiz was not always recognised after a crash.** The quick save on disk could be older than the one in memory and missing the new quiz id. Fixed: every new quiz gets its id at once (`newState`), and saves from build 139 or earlier are matched by game id.
- **Start fresh backed up the wrong thing.** Build 125's Safety Backup copies the quick save, which by then can be the empty Setup on screen (a gap since build 125) or the cut-down save. Fixed for 140: Start fresh backs up the complete unfinished game, once.
- **New quiz could delete a waiting game's copy.** New quiz while a saved game still waits for Resume keeps that offer (app110). Fixed: it now keeps the game's complete copy too.
- **The warning card first landed in the wrong place.** It went under "More controls", where it couldn't be seen, or took the round's status line. Seen in the before/after pictures. Fixed: it is added after the panel's layout, straight under the first card, in red.
- **Stray requests from other local web servers (found by the full run, `qc2`).** A page opened from some other local web server sent a request meant only for the laptop app; that server answered 501, which showed in the console. Fixed: the page talks to the laptop's save files only after the laptop app's own server has answered. The full run was restarted on the fixed page.
- **Three old checks expected build 139 exactly** (`fix139` , `fix139t` , `fix139d`). They now accept 139 or later.
- **Old statements in the docs.** The build instructions and architecture notes said the page is about 2.7 MB (it is about 3.6 MB). They also said the build uses this machine's paths, the order from `docs/BUILD_ORDER.txt` , and the badges from the laptop app's folder, and that a failed tightening step carries on. All of that is corrected: the code (`build.py`) is the source of truth.

Build 139 addendum (19 October 2026)

Your requests (18 Oct, 23:20): "do them all" (the seven upgrades offered after build 138: back-pressure for TVs on weak Wi-Fi, a media memory limit on the Galaxy Tab, a stuck-device safeguard, a team-code guessing limit, forgetting screens that have left, a smaller request limit on the laptop, and the 60-minute soak test), "make the default timer for final question at 45 seconds", "check the layouts especially when there is more than 7 players", and "the gold pulsing light is very similar to on fire player". At 23:21 you asked to see the glow options and picked **1 · White spotlight**. Estimated time given: about 8 hours, done around 7:30 am.

A · The "now playing" light: a white spotlight (your pick)

- **Before:** the team in play had a gold ring and a gold glow that pulsed, and 🔥 Streak Flames have an orange ring, an orange glow and flames, so the two looked alike. When the team in play was also On Fire, the flames' `!important` shadow replaced the Now Playing glow completely; only the gold tag and the lift showed it.
- **Options shown with before/after pictures** (`Build_139_glow_options_before_after.png`): 1 white spotlight, 2 electric blue, 3 gold stays and On Fire turns red. You picked 1.
- **Now:** a white ring and a soft white glow for Now Playing on every layout (`.stage/.sx/.sx.zoned/.sx.q-look/.ts-scores .lectern.control`); the two halo layers from build 138 carry it (calm and peak are both white), so the light still breathes slowly without repainting. The gold "Now Playing" tag is unchanged.
  - Now Playing **and** On Fire: the white ring round the orange ring and glow (5+ in a row: round the yellow "blaze" ring).
  - A Returning Champion in play: the white ring round its gold one.
  - Comic Book draws its own black outline on every lectern; the white ring still shows there, and On Fire keeps the comic outline, as before.
- **Checked (`fix139` , 20/20):** the computed border and shadows on the TV and the Quizmaster's preview; On Fire without white; the double ring; the other teams without a ring; all 12 themes.



B · The Final Round timer defaults to 45 seconds

- `newState()` sets 45 (was 60), and every place that fell back to 60 now falls back to 45 (the laptop's timer, the TV's number, Setup, the "Timer Ready" check, the tablet's preview).
- **Saved games:** `normalizeState` now moves a saved game whose Final timer is still the old default 60 to 45, once (defaults version 8), like the 30 → 45 s question timer in an earlier build. Any other value is kept, a Final already running keeps its time, and 60 chosen after the change stays.
- **Checked (`fix139`):** a new game, the Setup box, the running Final on the laptop and the TV, the five save cases, and Online Mode. The older checks `ext` and `rules35` expected 60 and were updated to 45.

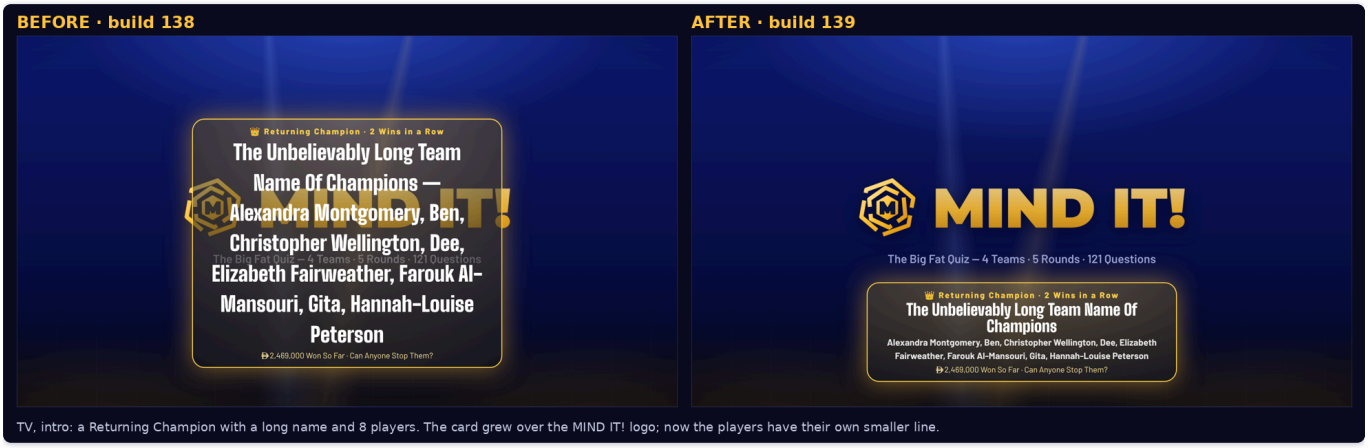
C · The layout check with more than 7 players

- **How:**
  - a copy of `avaudit.py` with every Audience View screen at 8 teams with long names, at 9 sizes: with no players, with 8 players a team, in Online Mode, and with 4 teams × 12 players;
  - the laptop at 1900×975 and 1366×768;
  - the tablet remote at 1280×800, 800×1280 and 768×1024;
  - team phones at 390 and 360 wide;
  - the 💾 standings and recap pictures.

  8 teams is the most a game allows. Done by a second agent in parallel, on build 138's page; nothing was changed while it looked.
- **Found and fixed** (before/after pictures: `Build_139_final_reveal_…` , `…_champion_intro_…` , `…_side_glow_buzz_…` , `…_pictures_…` , `…_tablet_laptop_before_after.png`):
  1. **Final reveal, 6–8 teams:** the live standings (absolutely placed at the bottom) covered the answer cards by up to about 200 px at 1920×1080. With 6+ rows the standings now use two columns; with 6+ cards the cards stay on one row, a little smaller (`app132#frfit` , decided by the counts, because the cards zoom in and the screen is arranged after the hook runs). 2–5 teams: unchanged.
  2. **Intro, Returning Champion:** with several players the card grew upwards over the logo (up to 432 px). Now the players are on their own smaller line (at most 2 lines), the team name is at most 2 lines and smaller when long, and the card sits a little lower.
  3. **Side scoreboard (7–8 teams):** the Now Playing glow sat 47 px to the right (31 px at 720p): `.sx.sc-side > *{grid-column:1}` also placed the halo layers in grid column 1. Now `grid-column:auto` for them, and the glow sits exactly on the lectern.
  4. 💾 **Recap card, 7–8 teams:** the chip and achievement boxes pushed the stats onto the footer and past the bottom. The boxes now stop where they fit.
  5. 💾 **Standings picture:** more than 6 champion players were drawn off-centre (centred on all, 6 drawn). Now centred on what is drawn, with a "+N" circle.
  6. **"Team Buzzed In!" with 7–8 teams:** cut off at both ends with a long name in the side layout. Smaller there, with an ellipsis if it still doesn't fit.
  7. **Tablet remote, Final marking:** rows shrank and centred, so Correct/Wrong sat at a different place on every row, 40 px tall. Rows now span the card; buttons are 48 px.
  8. **Laptop at 1366×768:** the 8 status chips took 7 lines and pushed the action buttons below the fold. Two columns with 7–8 teams.
- **Looked fine:** every other TV screen at 8 teams on all 9 sizes (board, questions, answers, Daily Double, Triple Gamble, Phoenix, Bottoms Up, round summary, standings, Final wagers/question/standings, winner, awards, How To Play, category reveal, buzzers, matchup), Online Mode, the laptop's Setup with 8 × 8, Final wagers card, Adjust Score, the tablet's question screen with 8 statuses, and team phones (they never list the teams; a 44-letter name wraps).

- **Checked (`fix139l` , 16/16):** no overlap in the Final reveal with 6 and 8 teams at 1920×1080 and 1280×720 (and 4 teams unchanged), the glow rectangle equal to the lectern's in the side layout, the buzz banner on screen, the champion card clear of the logo with a 44-letter name and 8 players, two-column chips. The layout audit on the final page: 234 screens, 0 issues (235/235); and with 8 teams × 8 players at all 9 sizes, 234 screens, 0 issues (235/235) — on build 138's page that audit flagged the intro, the Final reveal and Buzzed In!.

TV, Final Round reveal with 8 teams: the standings covered the answer cards; now they sit in two columns and the cards stay on one row (6+ teams).

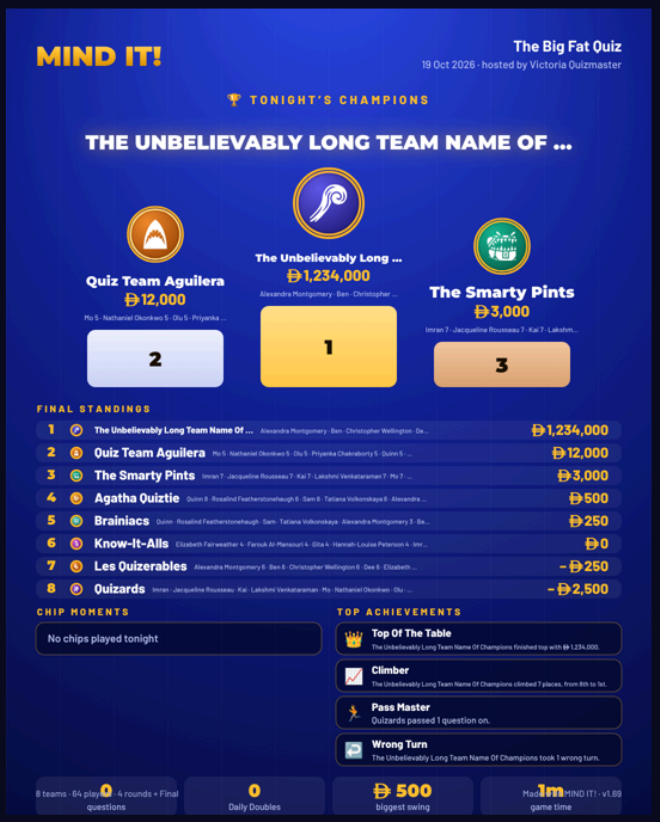


TV, intro: a Returning Champion with a long name and 8 players. The card grew over the MIND IT! logo; now the players have their own smaller line.

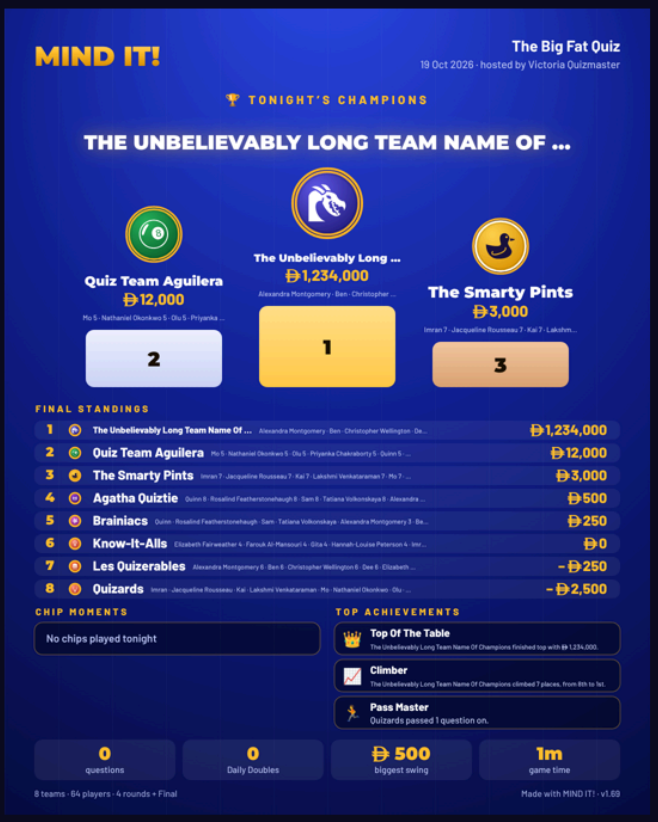
Top: the glow sat 47 px to the right of its lectern; now it is on it (and white). Bottom: the banner was cut off at both ends; now it fits.

Recap card picture (Save Picture), 8 teams

BEFORE · build 138



AFTER · build 139



Standings picture, a champion with 8 players

BEFORE · build 138

AFTER · build 139

Recap card: the stats ran into the footer and off the card; now the boxes stop where they fit. Standings: 6 of the 8 players were shown off-centre; now they are centred, with “+2”.

D · Reliability on the Wi-Fi

- **Found during this work, fixed:** in build 138 the laptop app had no `error` listener on its WebSocket connections. A message over the size limit (a TV: 32 KB) or a broken frame from **any** device on the Wi-Fi threw in the main process — Electron's error box — and the app stopped answering everything. Reproduced on build 138 (a test TV sent 40 KB: no answer afterwards). Now that connection closes and the app carries on (`fix139d`).
- **TV back-pressure** (laptop `main.js` and tablet `MindItHost.java`): a TV gets the next picture only when the previous one has gone out (laptop: under 256 KB still buffered; tablet: its send queue empty) and it has confirmed all but the last two; otherwise it is marked owed and gets the newest picture when it catches up (on its confirmation, when its queue empties, or at most 4–5 s later). Each picture is the whole screen, so skipping the ones in between loses nothing. Sound packets are skipped for a TV with over 1 MB waiting.
  - Tablet server (`fix139j`): a slow TV (it takes 0.35 s a picture) got 7 of the 40 pictures sent in 2.1 s, ending with the newest (#41) — the ones in between were skipped.
  - Laptop app (`fix139d`): a TV that keeps up got 80 pictures; a slow TV (0.4 s a picture) got 23 and still ended on the newest (#83 = #83).
- **Stuck-device safeguard:**
  - **Tablet:** every connection has its own send queue and writer thread; sends used to write to the socket directly, usually while holding the server's global lock, so one device that stopped taking data could freeze the server. A closing message ("blocked", "moved", an error) still goes out before the socket closes. The heartbeat closes a device whose writing has made no progress for 20 s; more than 96 MB waiting closes it at once; never the quiz page on the tablet.
  - **Laptop:** a phone remote, tablet or TV with more than 1 MB waiting that doesn't go down for 30 s is closed (it reconnects by itself); over 96 MB closes it at once; never the quiz page.
  - Checked: while a TV takes no data, the tablet server answered within 0.05 s and the other TV kept getting pictures; the stuck TV was disconnected. Same in the laptop app.
- **Media memory on the tablet:** pictures and videos for Wi-Fi TVs are kept within 120 MB of data in all, 40 MB each (about a 30 MB file); the ones not on screen go first, least recently shown first. A file over the limit, or with no room next to what is on screen, isn't kept: the TVs show the screen without it, the quiz page is told once (`aud-skip` , a note on the tablet), and it is never asked for again — so it can't loop (the page sends each file once). Checked with a 41 MB file, thirteen 10 MB files one after another, and all thirteen on one screen (`fix139j`), and the note on the tablet's quiz page (`fix139t` , 9/9).
- **Team-code guessing limit** (laptop and tablet): 20 **different** wrong codes from one network address in 10 minutes → 5 minutes' wait, during which even the right code is refused with `wait` and the phone says "Too many wrong team codes from this phone. Wait 5 minutes, then try again." A phone repeating the same mistyped code every 1.5 s counts

once (the phone page keeps checking with the code it was given); other addresses are unaffected.

- **Request and message limits:**
  - **Laptop:** other devices may send 2 MB (was 30 MB), checked against the announced size before anything is read (the Galaxy Tab's hand-over routes keep 30 MB; the quiz page on the laptop keeps 150 MB). Link messages from other devices: 2 MB (was 64 MB), through a second WebSocket server for non-local connections.
  - **Tablet:** link messages from a remote after its hello: 2 MB (was 64 MB).
- **Screens that have left** (laptop and tablet): an offline screen that isn't named, blocked or the sound screen is forgotten after 7 days away, and at most 12 such screens are kept (pruned at start-up and when a screen connects).
- **Checked:** `fix139j` (tablet server on the JVM, 26/26), `fix139d` (laptop app, 23/23), `fix139t` (tablet quiz page + server, 9/9).

E · Tests

- **Full run** on the final page (md5 `b1098ad2…`): full run on the shipping page (md5 `b1098ad2…`, 94 min): 139 suites. 136 passed first time, none needed a retry. Three failed because the tests were out of date, not the app, and passed once the test was corrected: `fix130c` 6/7 → 7/7 (its CSS check counted build 139's approved restyles as build-130 rules), `desk` 17/18 → 18/18 (it expected the 60-second Final) and `fix139j` stopped at its last step → 26/26 (the test program didn't expect the tablet to close a connection while it was still sending 3 MB). In all 3,053 checks, all passing.
- **Stress phases** (`stress110.py`, laptop app, 8 teams, 10 phone pages, a tablet remote): idle (1 min each, question and board) 2/2, net 6/6, load 4/4, lock 2/2, rapid 4/4, health 6/6: 24/24 on the final build
- **60-minute soak:** 2/2 — a 60-minute real-time quiz with a tablet remote, the TV window and 10 phones: 60 questions, 377 tablet actions, 93 samples, the Quizmaster window hidden and shown 5 times, 0 false disconnects, no script errors; laptop server memory 223 → 233 MB, the Quizmaster page's JS heap 15 → 16 MB (no game was finished inside the hour)

F · Not tried for real

- The Galaxy Tab itself (its server ran on a computer's Java; the release check opens the APK on an Android 15 emulator), a Windows PC, a Mac, real smart TVs and real weak Wi-Fi (slow and stuck TVs were test programs that read slowly or not at all).

Build 138 addendum (18 October 2026)

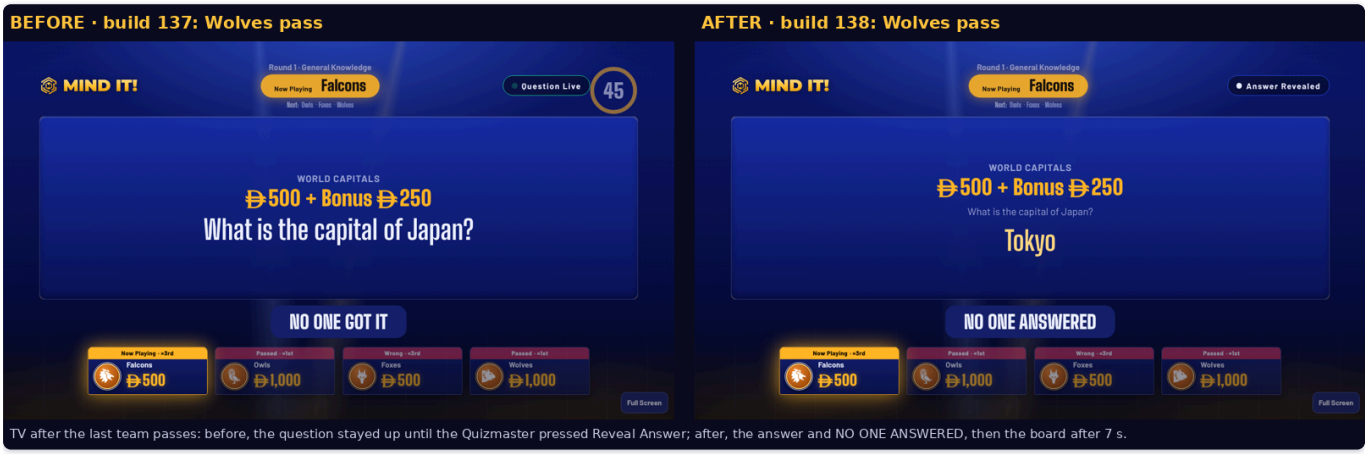
Your requests: "do a code check across all and optimize it if needed" (16:59), then the passing and automatic-return specs (19:07–19:22, section A), all in this build. Estimated time given: published by about midnight.

A · One go per team, the automatic No One Answered and the 7-second return (your three specs this evening, merged into this build)

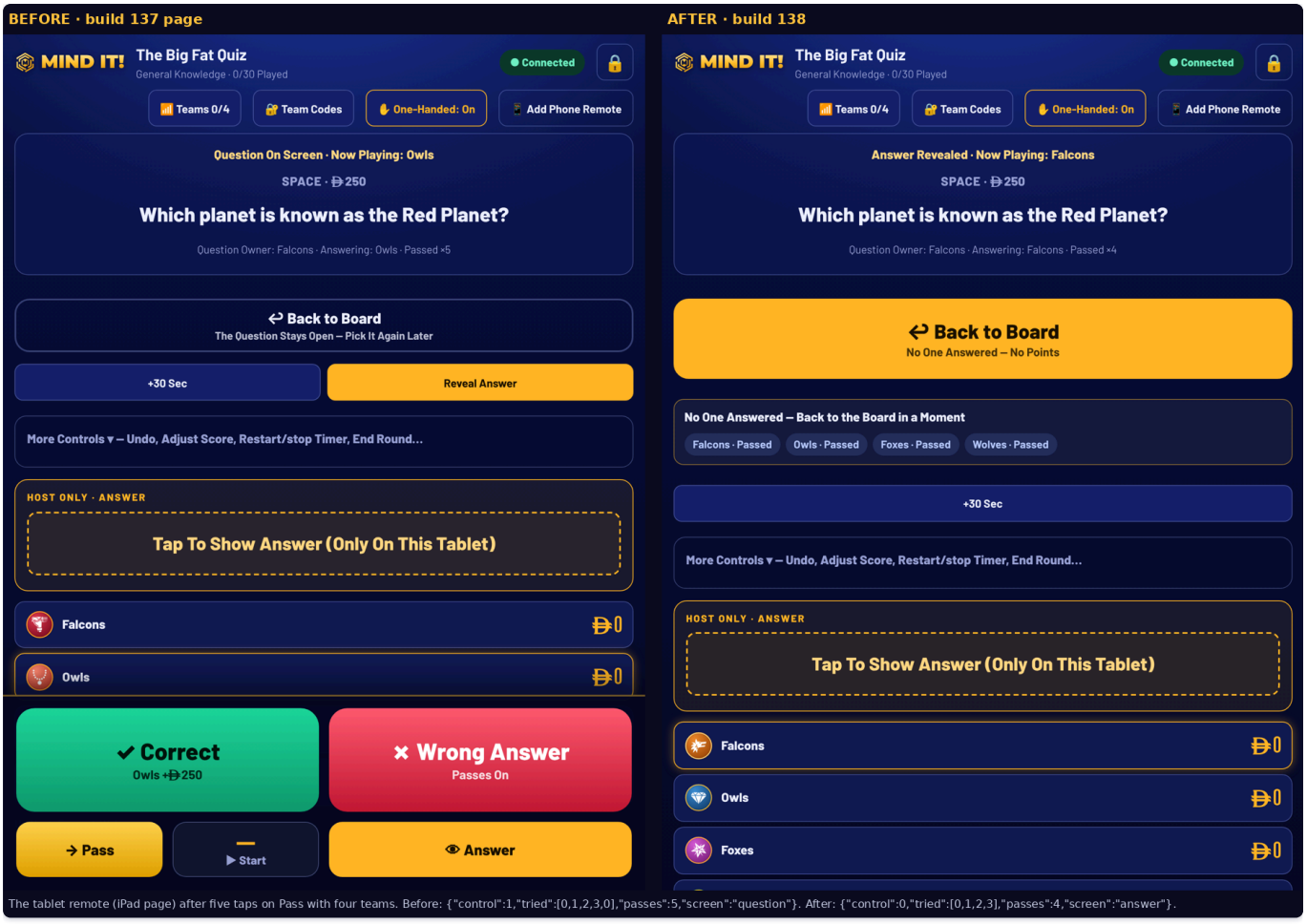
- **Your requests:**
  - "Fix Unlimited Passing & Add 'No One Answered' Option";
  - its correction "Automatic No One Answered Flow";
  - "combine it all and put it in build 138";
  - "Automatic Return to Board After Answer Reveal" (merged into this build).
- **What the code did before** (the code is the source of truth, and it differed from the spec's starting point):
  - the laptop's own buttons already stopped once every team had had a go: "No one got it — next question", and "NO ONE GOT IT" on the TV;
  - the **Galaxy Tab / iPad / iPhone remote** kept showing ✓ Correct, ✗ Wrong and → Pass after that, and the laptop accepted them (`linkHandle` → `passTurn` / `answerWrong` had no such check). This made the "unlimited passing" real;
  - `nextTeam()` didn't skip teams that had already had a go, which mattered after the team was changed by hand;
  - the host override ("A different team answered?") could mark a team wrong twice, taking the penalty twice;
  - Classic turn order allowed any number of goes per team.
- **Now** (`app131.js`, plus small patches to `app3`, `app4`, `app25` and `app116`), on normal board questions:
  - **One go per team:**
    - `cu.tried` (passes and wrong answers, existing) decides who is eligible: available and not tried (`qEligible`);
    - `answerCorrect`, `answerWrong` and `passTurn` refuse a team that has had its go;
    - `setActive` refuses it, and Make Active is greyed;
    - the host override lists only eligible teams;
    - remotes get `no_go` / `tried` refusals, with a message on the tablet;
    - after a pass or a wrong answer the question goes to `nextEligible()`, the next eligible team in the usual order;
    - Daily Double, Triple Gamble and the Final keep their own rules.
  - **Automatic close:**
    - when no eligible team is left (`qExhausted`, checked after every redraw), `noOneAnswered()` stops the timer and shows the answer screen;
    - the TV's banner under the answer now reads **NO ONE ANSWERED** (it read "NO ONE GOT IT", and only in turn order);
    - after **7 s** (`NOONE_MS`), `closeClueNoScore()` marks the question played, gives no points and keeps the deductions. The next team after the owner is highlighted in turn order; in Classic the picking team stays, as before;
    - the countdown is stored in the game (`cu.noOne.until`), so a reload or a hand-over finishes it on time.
  - **No One Answered button:**
    - in the action bar and the Scoring card, replacing "No one got it — close clue";
    - it asks first on the laptop; the tablet and the 📱 phone remote ask on their own screen.
  - **The 7-second return.** The code already returned to the board by itself after a correct answer: `app80`, 3½ s, on by default, with a "Stay here" button and a setting in 5 · Audio & Presentation. Your spec described the old flow as manual, so this was reported to you at 19:33. Rather than adding a second timer:
    - that feature now waits **7 s** (`AUTO_BOARD_MS`);
    - it also starts when a Daily Double or Triple Gamble ends with a wrong answer (`app131#rv`, once per question);
    - its setting is now called "Back to the board automatically after each answer".
    - No One Answered always closes after 7 s (`NOONE_MS`), whatever the setting.
    - Next Question, Back To Board Now, "Stay here" or Undo before then cancel it. Each timer checks that the same question is still showing, so the board comes back once.
    - Only the quiz device runs these timers (not remotes, phones or TVs).

    - The team in control was already set when the question was scored (unchanged).
  - **Statuses:** "Answering Now · Passed · Wrong Answer · Not Attempted · Away" and "N of M teams can still answer" on the Scoring card. The tablet remote shows the same, and when none are left: "All teams have attempted or passed on this question." In that case its big buttons go.
  - **Phone Buzzer Mode:** Reopen Buzzers keeps the teams that have had their go out.
- **Choices made where the specs left room** (stated to you at 19:18–19:25):
  - **7 s for No One Answered too.** The first correction said about 5 s; the newest message says 7 s for every revealed answer.
  - **The existing back-to-board setting and "Stay here" stay.** If the setting is off, correct answers wait for Next Question as before; No One Answered still closes by itself.
  - **The tablet shows the No One Answered close as its big "↩ Back to Board"** (`app92`): "No one answered — no points".
  - **A manual "Reveal Answer" doesn't start the 7 s** while teams still have their go, because the question isn't decided yet.
  - **Reopen Buzzers** reopens only for teams that haven't had a go.
- **Checked (`fix138p` , 30/30):**
  - two teams both wrong; four teams all pass; mixed wrong and passes;
  - a correct answer after two passes, with the real 7 s measured (7.1 s after the correct answer);
  - one team left; No One Answered early (Cancel and OK);
  - tablet commands after every go (refused);
  - Daily Double and Triple Gamble;
  - a reload in the middle; the next question; a manual adjustment;
  - Next Question before 7 s (no second jump); Undo during the countdown;
  - Classic.
  - **In the laptop app with a real tablet remote:** `fix138pd` 9/9 — an iPad remote on the laptop's link: three Passes moved the question 0 → 1 → 2 → 3, the statuses showed on the tablet, a Pass after every team's go was refused with "Every team has had its go on this question.", and laptop and tablet were back on the board 7.3 s after the last wrong answer; a Correct from the tablet returned once.

Laptop, the Question On Screen card: Falcons wrong, Owls passed, Foxes wrong — Wolves answering.



TV after the last team passes: before, the question stayed up until the Quizmaster pressed Reveal Answer; after, the answer and NO ONE ANSWERED, then the board after 7 s.



0 · How the check was done

- **Read:**
  - every module the page is built from (`app0` … `app129` , `reuse.js` , `starter_book.js` , `style.css`);
  - the laptop app's main process (`desktop/app/main.js`);
  - the Galaxy Tab's built-in server (`MindItHost.java` , `Json.java`).
- **Ran:**
  - a whole simulated game with every error hook watched: no hook, app or page errors; about 12.6 ms per redraw; the page's memory went from 10.1 to 11.2 MB; 27 timers, all cheap;
  - CPU profiles of the Quizmaster page with the Audience View open, in Chromium and in the laptop app (all of its processes);
  - the tablet server on a computer's Java (as the other tablet suites do), fed malformed and oversized requests.
- **What came out:** the fixes and speed-ups below. The code check itself changes nothing about how the game plays; section A, your specs, does.

1 · Bugs fixed

1 · Six buttons did nothing in the laptop app (Windows and Mac).

- The laptop app (Electron 33) has no `prompt()` . It throws "prompt() is not supported." These buttons used it:
  - ▪ New Category, Move To Category, Rename and Delete Category (when the category has questions) in the Question Database;
  - ⇄, which moves a question to another round;
  - ✎ Edit Answer on Mark Each Team.
- Build 137's addendum noted that `prompt()` does nothing in the laptop app, but only replaced it for Online Mode's chat answers.
- **Fix:** `askText()` in the new module `app130.js` is the app's own text box. It opens with the current value selected; Enter or OK saves; Escape, Cancel or a click outside closes it without saving; Escape doesn't also close the Question Database underneath. It offers the existing categories as suggestions. `app27` and `app32` use it.
- **Checked:**
  - in the laptop app, `prompt('x')` throws "prompt() is not supported.", and + New Category works through the box;
  - on the published build-137 page, with `prompt()` throwing as it does in the laptop app, + New Category does nothing (25 → 25 categories);
  - on build 138, all six buttons work.

2 · ✎ Edit Answer was undone within a second when team phones were connected.

- The laptop app's phone link replaces `G.finalAnswers` twice a second with what the phones sent, so a correction written there was overwritten.
- **Fix:** a correction is kept in `G.finalAnswersEdited` (the phone's answer stays as the original in `G.finalAnswersOriginal`). `finalAnswerFor()` reads the correction first, so the suggestion, the TV reveal and the game record use it. The link snapshot, which phones and Wi-Fi TVs read, carries it too. A new Final Round starts with no corrections.
- **Small change:** saving an empty box now leaves the answer as it was. Build 137 blanked it.

- **Checked in the laptop app:** three phones answered the Final; "burj kalifa" was corrected to "Burj Khalifa"; 3.5 s later, with the phone link on, the correction was still there, the suggestion said correct, and the TV reveal showed “Burj Khalifa”.
- Online Mode is unchanged: there, answers are corrected in the Mark Each Team boxes, and it has no ✎ button.

3 · Laptop app: a large save could damage accented letters and emoji.

- The laptop app decoded each 64 KB piece of an incoming request on its own. A character whose bytes fell across two pieces became "��".
- It was reproduced with build 137's `main.js` : a 940 KB save came back with 14 damaged characters.
- Your 10 October backup has no damaged characters (no U+FFFD).
- **Fix:** `readBody()` decodes the whole request as one UTF-8 stream (`req.setEncoding("utf8")`).
- **Checked:** a 1,826 KB save full of é ’ — 😀 東京 comes back identical, with 0 damaged characters. The laptop's own profiles were then put back.

4 · 📶 Devices always said "Tablet not connected".

- It read `LINK.devices` and `linkDeviceCount()` , which haven't existed since the connection rework.
- **Fix:** it reads the trusted-device list that the laptop sends to the page (`window.DVSTATE`). Phone remotes don't count as the tablet.
- **Checked in the laptop app:** with the tablet connected it says "● Connected · ✓ Tablet connected". With only a phone remote online it says "Tablet not connected".

5 · Galaxy Tab hosting: Wi-Fi TVs were disconnected every 15 seconds.

- The tablet's heartbeat pinged every socket every 15 s and closed any that hadn't answered the previous ping. TV sockets have their own 5-second beat (`tvBeat()`), so the heartbeat closed them every time, and they reconnected.
- **Fix:** `heartbeat()` skips TV sockets.
- **Checked on the tablet server:** a TV stays connected for 35 s and answers 7 pings. Build 137's server closed it after 12.8 s.

6 · Unexpected or malformed data.

- **Galaxy Tab** (an uncaught error there closes the app):
  - deeply nested JSON overflowed the stack. Nesting now stops at 256 levels: 120,000 `[` get an HTTP answer and the server keeps answering;
  - a WebSocket frame claiming a negative length reached a 2 GB allocation. It is refused now. Before its hello, a device may only send messages up to 64 KB (TVs: 32 KB, as before);
  - requests from other devices are limited to 2 MB, refused before anything is read. The hand-over routes allow 30 MB and the tablet itself 150 MB, as before;
  - a request with more than 100 header lines is closed;
  - a request's body is read as it arrives. Before, a buffer of the announced size (up to 30 MB from any device, or a negative size that failed) was set aside first.
- **Laptop app:**
  - a "null" message on the link socket threw in the main process, which shows Electron's error box. Such messages are now ignored, as the TV socket already did;
  - a malformed address threw outside the error handling. It now gets "400";
  - the "try the next port" listener stayed attached after the server started, so a later network error would have started another server. It is removed once the server is listening.
- **Both: remote PIN guessing.** 20 wrong PINs from one network address within 10 minutes make that address wait 5 minutes; even the right PIN waits. Before, any device on the Wi-Fi could try all 9,000 PINs. A remote that used to be trusted, or the wrong session, doesn't loop: the remote stops and asks for the PIN.
- **Checked:**
  - tablet server: `fix138j` , 13 checks; build 137's server fails 7 of them;
  - laptop app: `fix138d` . Five malformed messages caused no error in the main process, and the Quizmaster page's own link stayed connected.

7 · Laptop app: the "isn't responding" box.

- Each hang of the page opened another box.
- Now there is one at a time, and it closes by itself when the page answers again (`AbortController`).
- Not tested (making a real hang takes 30 s or more). Checked by reading the code.

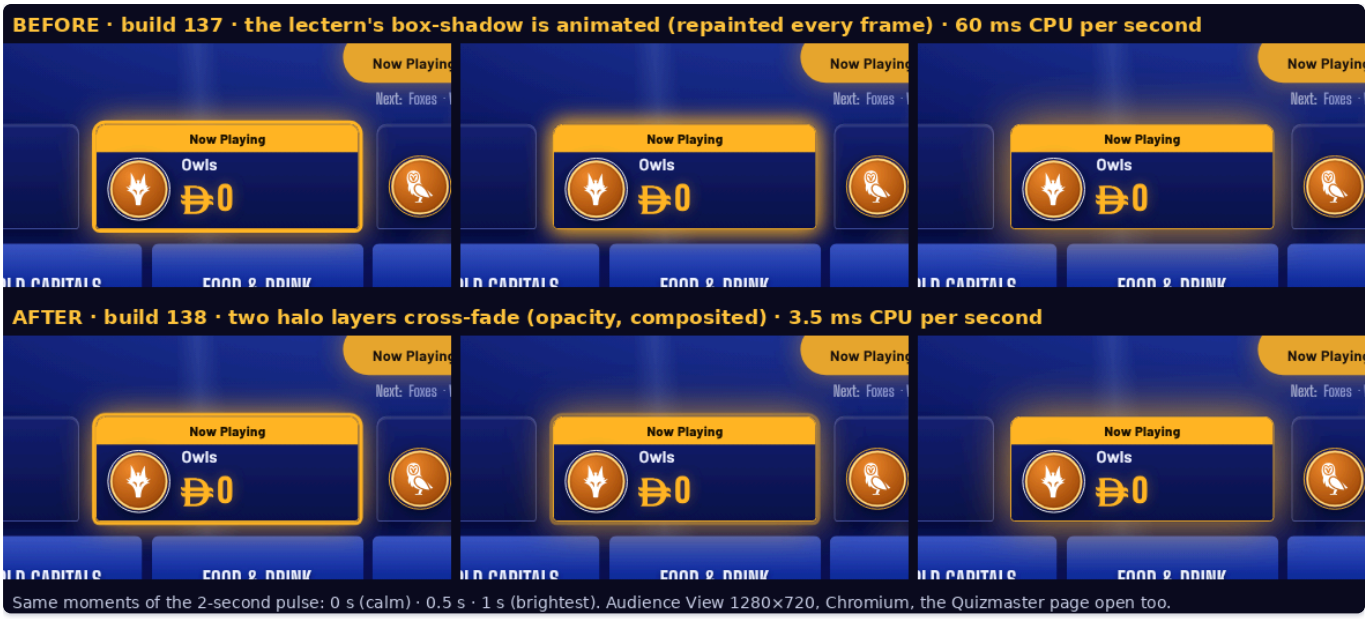
8 · Laptop app: handing the quiz to the tablet.

- **Cancel during the last step:** pressing Cancel while the final snapshot was downloading no longer carries on with the hand-over.
- **A late confirmation:** after the 2-minute wait ran out, a confirmation that the tablet sent late now restarts the hand-over. Before, it was left waiting.
- The hand-over suites in the full run cover the normal path (see Tests). These two edge cases were checked by reading the code.

2 · Optimisations

9 · The "now playing" glow.

- The pulse animated the lectern's `box-shadow` , which the browser repaints every frame, all the time a team is in control.
- It is now two layers on the lectern whose opacity the compositor cross-fades:
  - the calm layer carries the lectern's own outer glow (read from the theme, and re-read after a resize, because the glow is sized in `cqw`);
  - the peak layer is the old brightest keyframe (`0 0 3.4cqw rgba(255,182,39,.85)`).
- The lectern keeps its inset highlight. Streak Flames keep their own glow. The layers follow the lectern exactly (within 0.02 px), with and without the TV safe area, after a resize, in the Quizmaster's preview and in Wi-Fi TV frames.
- **Measured** with the Audience View open, CPU of the Quizmaster page and the Audience View, in milliseconds per second:
  - Chromium: 59.9 → 3.5;
  - laptop app, all its processes: 819 → 224 ms per second for all its processes (the page's renderer 734 → 86; the graphics process 59 → 111, because the build machine has no GPU and draws the moving layers in software — on a laptop with a graphics chip that part is close to free).
- **The look** (picture): the calm and brightest moments are the same as before. Halfway through, the amber ring fades out, where before it melted into the glow.



Same moments of the 2-second pulse: 0 s (calm) · 0.5 s · 1 s (brightest). Audience View 1280×720, Chromium, the Quizmaster page open too.

10 · The page is sent compressed.

- Phones, tablet remotes, Wi-Fi TVs and the tablet's own page used to download the 3.5 MB page in full every time. The laptop app re-encoded it on every request.
- **Now:**
  - the laptop app keeps the page gzip-compressed: 3,575,122 → 1,811,855 bytes (gzip level 9, made once at start);
  - so does the tablet server: 3,575,122 → 1,821,031 bytes (made once, on the first request);
  - both send an ETag, so a reload gets "304 Not Modified";
  - a browser that doesn't ask for gzip gets the page as before.

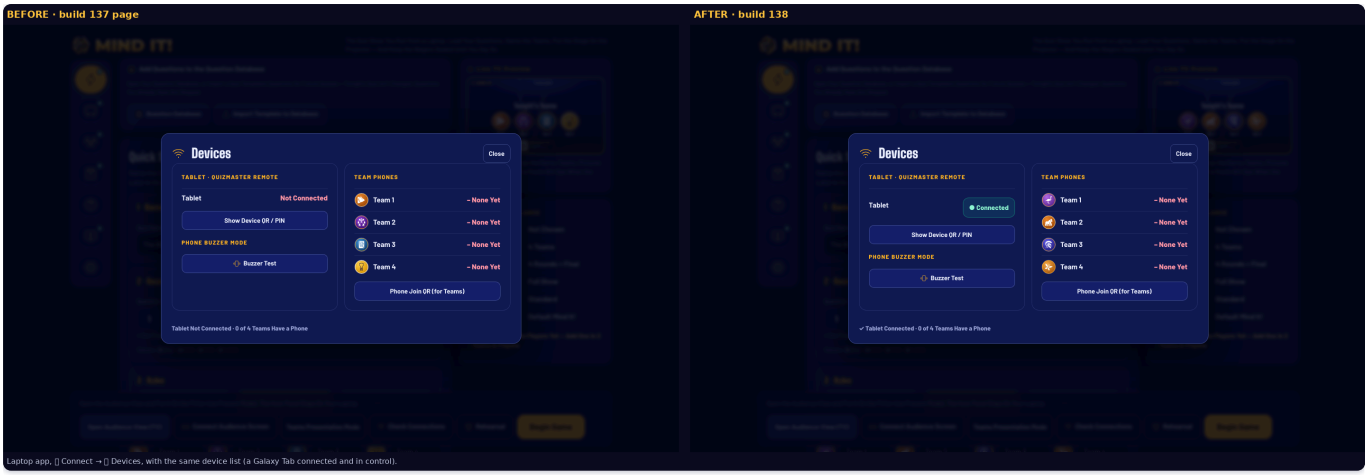
11 · Fewer writes to `trusted-devices.json`.

- A paired phone remote polls twice a second, and each poll rewrote the file.
- It is now written at most once a minute per device, or at once when the device's name changes. The time it was last seen is still kept in memory and saved with the next write.
- **Checked:** 12 polls in 6 s gave 1 write.

12 · Dead code removed.

- `app4` : two lines after a `return` .
- `app6` : an `if(false){…}` block.
- `app26` : an always-true `||true` .
- `app129` : two unused variables.

Laptop app, Question Database → ⋯ → + New Category. Before: “prompt() is not supported.”, no box (25 → 25 categories). After: the box, cursor in it.



3 · Looked at and left as it is (suggestions for later)

- **TVs on weak Wi-Fi:** a slow TV can fall behind; there is no back-pressure.
- **Picture and video rounds hosted on the tablet:** no memory limit for very large media.
- **Tablet server:** socket writes happen under the server's global lock, so one stuck device can slow the others (needs a watchdog).
- **Team codes:** there is no guessing limit (PINs now have one).
- **Audience displays:** the list of displays that have left isn't pruned.
- **Laptop app:** the limit for requests from other devices is still 30 MB (the tablet's is now 2 MB).

4 · Tests

- `fix138` (webpage, Chromium, with `prompt()` throwing as in the laptop app): 32 checks.
  - The text box; the six buttons; build 137 doing nothing.
  - Edit Answer surviving the phone refresh, in the snapshot, reveal and record.
  - Both glow layers: position, shadows and animations.
  - The rule presets are build 137's, and there are no hook or page errors.
- `fix138d` (laptop app): 21 checks. The large save; malformed messages; the compressed page and ETag; + New Category; Edit Answer with three phones; Devices; trusted-devices writes; the PIN limit; no main-process errors.
- `fix138j` (tablet server): 13 checks.
- `qlib` **:** now types into the new box.
- **Full run:** full run on the shipping page (md5 `a5b16b1c…`): 134 suites, 2,953/2,953 checks, `fix127d` passed on its automatic retry (a 4.5-second reconnection check; 22/22 again when run on its own) (90 min). New: `fix138` 32/32 and `fix138p` 30/30 (browser), `fix138j` 13/13 (tablet server), `fix138d` 22/22 (with the Wi-Fi TV check) and `fix138pd` 9/9 (laptop app + tablet remote); `qlib` 26/26.

Build 137 addendum (18 October 2026)

Your requests:

- "any upgrades? anything you would suggest for the online teams mode": six proposals with before/after pictures.
- "do them all, and from now on weblink gets republished everytime".
- Also in this build: item 1 from the build-136 list, the Android emulator launch test in the release workflow ("i will do 1 in a bit"). It was offered to you as optional.

0 · Two build-136 statements were wrong (found while making the proposals)

- **How to Play.** The build-136 addendum ("How to Play says the same") and the guide ("How to Play on the TV explains it the same way") were wrong.
  - Build 136 changed `howToLines()`, but the TV hides that list and shows the rule cards of `afterHook("renderStage","app72#1")` instead.
  - Its Final Round card still said "answers on their phone".
- **The cursor.** The build-136 release notes said "the wager card puts the cursor in the first box".
  - In the webpage version, the first box was a sealed-code box (`.wg-input.code-row`).
  - The laptop app has no code boxes, so it was right there.
- **Why the tests passed.** `fix136o` checked `howToLines()`, and its focus check accepted any `.wg-input`.
  - Both checks now look at what is shown: the TV's `.ht-card` texts, and `.wg-input:not(.code-row)`.
  - On the published build-136 page the corrected suite fails exactly these two checks (25/27); on build 137 it passes (27/27).

1 · Online Mode for Teams calls, part 2 (`app129.js`; only while Online Mode is on)

- **Teams Presentation Mode on pick.**
  - The `applyRulePreset` wrapper calls `setTeamsMode(true)`, which opens the Audience View and, 150 ms later, `presentationCheck()`.
  - It does this right away at Setup. Mid-game it waits: `ask` gets a temporary wrapper around its callback, so nothing happens until **Apply**.
  - Classic and Full Show never switch it on.
  - Not in the Galaxy Tab app (`window.MindItAndroid`): it has no second window (`window.open` returns nothing there), so the Presentation Check would only show a blocked Audience View.
- **Speed Bonus window.**
  - Picking Online Mode sets `G.speedBonusSecs=15` when it is 10, and marks it with `G.olSpeed15`. Another preset puts back 10 only if it is still 15.
  - A new quiz with Online Mode remembered starts at 15 (`newState` wrapper).

- Checked with `speedBonusFor`: an answer 12 s in earns the bonus at 15 s and not at 10 s.

- **How to Play.** `afterHook("renderStage","app129#howto")` replaces the texts of two cards:
  - **Pick & Answer:** "Teams take turns choosing a category and value. Unmute and answer out loud on the call before the timer runs out."
  - **Final Round:** "Everyone sends a wager, then their answer, to the Quizmaster in a private chat message — the scores can change completely."

- **Final answers by private chat.**
  - **TV:** `.ol-fnote` on the Final question ("📩 Send your answer to the Quizmaster in a private chat message"), re-fitted by `fitClueText`.
  - **Where answers are kept:** `G.finalAnswersChat` (team id → text), apart from `G.finalAnswers`. In the laptop app the phone link replaces `G.finalAnswers` twice a second; checked, a typed answer stays.
  - **Reading them:** `finalAnswerFor(i)` returns the phone answer, else the chat answer. So the existing suggestion (app32), the reveal line on the TV (app3) and the game record's `finalAudit` use it unchanged.
  - **Mark Each Team** gets an input per team (not revealed, no phone answer):
    - Enter or Save calls `olSetChatAnswer`, then the cursor moves to the next team without an answer;
    - the first box gets the cursor once per Final;
    - drafts survive a redraw (`OL_DRAFT`);
    - a changed answer clears that team's mark and audit;
    - the `prompt()`-based "✎ Edit Answer" is removed for these answers, because the box replaces it (`prompt()` does nothing in the laptop app).
  - **Reset:** `startFinal` and `resetProgress` clear the chat answers.
  - **Quizmaster remote:** `linkSnapshot` gives the remote the merged answers.

- **Wagers.** In `wagerCard` / `ddWagerCard` (Online Mode only):
  - removed: `.keybox`, `.code-row`, the booth button and "· N phones connected";
  - the auto banner says the wagers come by private chat;
  - **All in · Half · 0** (`Min` when the minimum isn't 0) call `hostSetWager` like Save, so the same checks and auto-reveal apply;
  - the Daily Double card gets **Half** (at least the minimum) between Minimum and Maximum;
  - the first-box focus in `app128` now skips `.code-row`.

- **Copy picture for the chat.**
  - A tile (or button in narrow layouts) after **Save picture** on the standings and winner screens, in Online Mode.
  - `navigator.clipboard.write([new ClipboardItem({"image/png": recapPNG() promise})])`: the promise keeps the click's permission while the picture is drawn. If that fails, it retries with the finished picture, then falls back to `offerDownload` with "Picture Saved Instead".
  - **Laptop app:** `main.js` now grants `clipboard-sanitized-write` in its permission handlers. It was refused before, which was measured: `NotAllowedError`. It is write only; reading stays refused. `BUILD` (the hand-over protocol, 132) is unchanged.

- **Styles:** `.ol-fnote`, `.ol-fans`, `.ol-quick`, `.end-tiles.ol-four` (2 × 2 tiles when there are four).

Build 137 · Online Mode on the TV — BEFORE = build 136 · AFTER = build 137

How to Play: the cards say how a call works (build 136's Final Round card still said "answers on their phone")

BEFORE (build 136)

AFTER (build 137)

The Final question asks for the answer by private chat

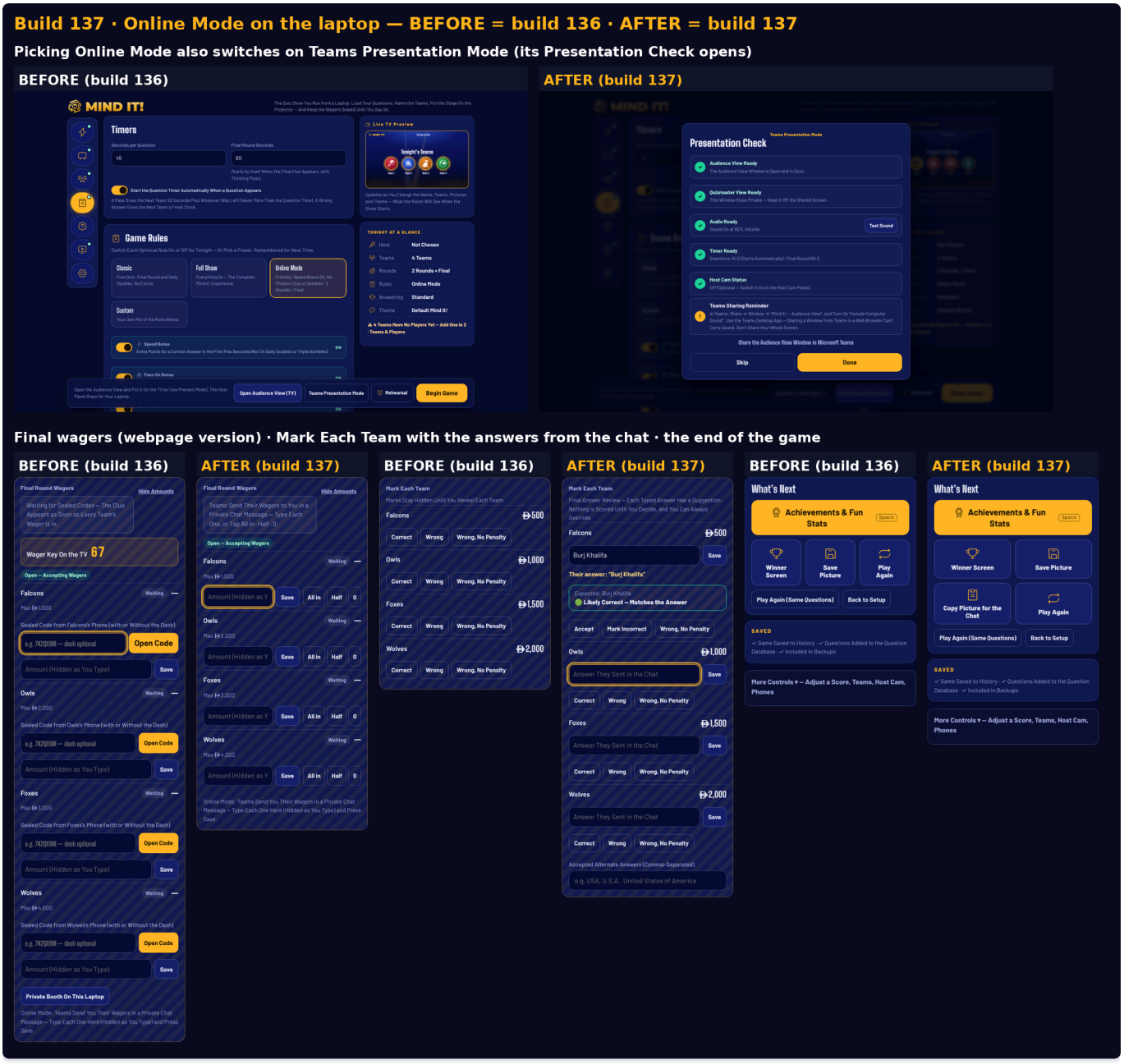
BEFORE (build 136)

AFTER (build 137)

The reveal shows the answer the team sent (typed in by the Quizmaster)

BEFORE (build 136)

AFTER (build 137)



2 · The release workflow opens the tablet app on an Android emulator

- **Scripts in MindIt-Downloads:**
  - `apk_launch_test.sh` : install, find the launcher activity, tap the icon (`monkey`), wait 20 s; the process must be alive, the resumed activity must be in `com.mindit.app` , and there must be no `FATAL EXCEPTION` ;
  - `apk_launch_result.sh` : ok → notice, plus a line in the release notes; fail → error; skip / no result → warning.
- **In `publish-release.yml` :** after validate, when the build has a new APK, `reactivecircus/android-emulator-runner@v2` runs (API 35, google_apis, x86_64).
  - An app that doesn't install or doesn't stay open stops the release.
  - An emulator that doesn't start only warns.
  - Both workflows pass `actionlint` .
- **Tried on the published apps** (runs 39117358542 and 39117817701, try-out branch `apk-launch-test`):
  - build 134 fails: "Unable to instantiate activity ... com.mindit.app.activity: ClassNotFoundException", and the launcher stays on screen;
  - build 136 opens: `com.mindit.app/.MainActivity` on screen, process alive.
- **By hand:** Actions → "Test the tablet app on an Android emulator" → tags.
- Build 137's own APK goes through the same check when the release workflow publishes it; the result is shown on the release (the last line of its notes) and in the workflow run.
- **Branches left behind:** deleting branches is refused from the build machine's session, so you can delete these on GitHub; neither holds anything that isn't in `main` .
  - In MindIt-Downloads, the try-out branch `apk-launch-test` (merged into `main`).
  - In the source repository, a branch `master` that I pushed by mistake (the same commit as `main`).

3 · The web link is republished every build

- **New rule** (owner: "from now on weblink gets republished everytime"), now in CLAUDE.md.
- Build 136's page was republished straight away (version 173). republished with the build-137 page (version 174; the saved copy holds exactly this page)

Tests: full run on this page: 129 suites, 2,837/2,837 checks, all first try. `fix137` 31/31 · `fix137d` 10/10 · `fix136o` 27/27 (25/27 on the published build-136 page) · `fix136od` 6/6.

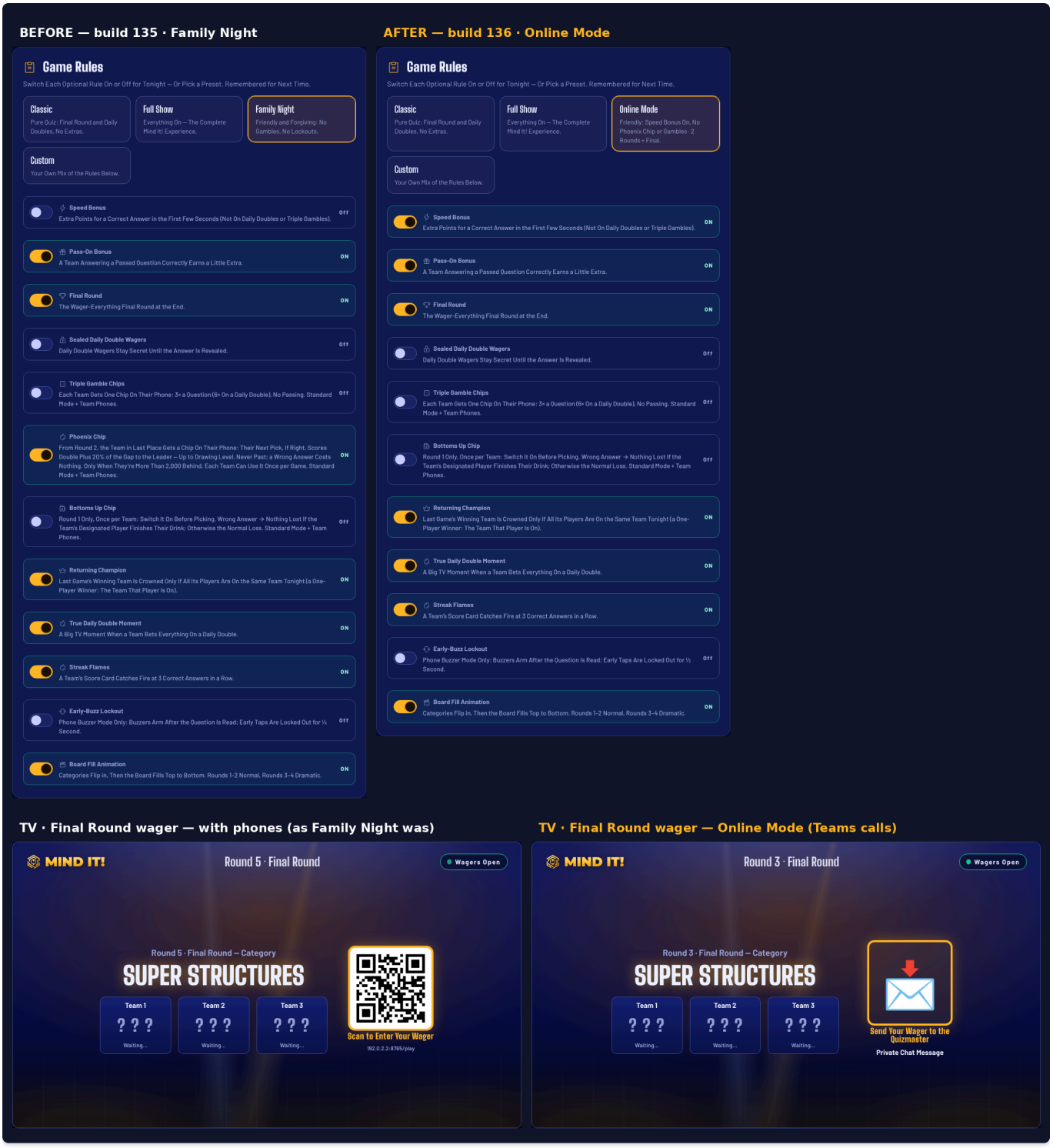
Build 136 addendum (18 October 2026)

Your requests:

- "Do 2,3,4,5" (from the build-136 upgrade list), and "make mindit-downloads public".
- **Online Mode:** "Mind It! — Replace Family Night with Online Mode", then "i will use it on Teams calls" and "do those changes, combine everything and put it in build 136".

0 · Online Mode replaces Family Night (`app128.js`)

- **Presets:** Classic · Full Show · **Online Mode**. Family Night is gone, not kept alongside.
- **Settings:** Online Mode keeps every Family Night setting except your seven:
  - Phoenix Chip **off**;
  - Speed Bonus, Streak Flames, Returning Champion, Board Fill Animation, Pass-On Bonus (25%) and the Final Round **on**.
- **Sticky:** while it is selected (`G.ruleMode="online"` , remembered for the next quiz), it stays selected when other switches change. The Phoenix switch is hidden, and the Phoenix Chip stays off even if an old save says otherwise (`ruleGet` and `udEnabled`).
- **When picked in Setup:** 2 question rounds + the Final Round (automatic values: Round 1 200–1,000, Round 2 400–2,000) and Standard answering.
- **For Teams calls** (players can't reach the laptop's Wi-Fi):
  - The Final Round wager screen shows "📩 Send your wager to the Quizmaster · private chat message" instead of the QR code.
  - The Daily Double screen asks the team to tell the Quizmaster.
  - No "Scan to join" code.
  - How to Play says the same.
  - The laptop's wager card says so and puts the cursor in the first empty box.
- **Unchanged:** Classic and Full Show (compared with the build-135 page). The same Final Round screen in Full Show still shows the QR code in the laptop app.
- **Checked:**
  - the TV shows no QR code in Online Mode, in the browser and in the laptop app;
  - no Phoenix Chip at the start of Round 2 (Full Show gives one);
  - an Online Mode game starts with the Speed Bonus.



1 · Lighter backups (item 2)

- **What was in a backup:** `exportFull()` wrote a copy of the game with every question picture in full, next to the player pictures, where the same pictures already are. Measured on your 10 October backup: 7 question pictures (13 places in the quiz), 3.4 of the 10.5 million characters a build-135 backup of that data takes.
- **Now (`app127.js`):**
  - The copy of the game in the file points at the identical picture in the same file, with the `imgref:` pointer the autosave has used since build 111.
  - The undo copy of a replaced round is left out once the game has started. A restore drops it then anyway; since build 111 it is usually gone already.
  - The game on screen is never changed.
- **Applies to:** Back Up Now, Hand Over, Save Backup File, Safety Backups and Google Drive backups. All of them use `exportFull()`.
- **Restoring:** a restore merges the pictures first, then puts them back into the quiz (`importAny` → `mergeProfiles` → `normalizeState` → `imgRefsResolve`).
  - Tested on the build-119 page from the rollback zip, the published pages of builds 132, 133, 134 and 135, and 136: every picture is back in the quiz, byte for byte.
  - **Build 111 had decided the opposite** ("backup files stay complete … so they restore on any build"). Build 136 backups still hold every picture, but a build older than 111 can't put a shared one back. None of those builds is in use, and the rollback build 119 restores them (tested).
  - A build-135 backup restores in 136.
- **Your data:** the 10 October backup, restored into build 136, writes 7.1 million characters instead of 10.5 million.

2 · 🗜 Make photos smaller (item 3)

- **What it shrinks:** only the *original* kept for re-cropping a player photo (`photoOrigId`), when it is bigger than 800 px. That is the size new photos have been kept at since build 111. Encoded like a new photo (`picEncode` : WebP where the browser can make it, else JPEG).
- **Same picture IDs:** pictures move to the tablet as "add if missing", so a tablet keeps its own copy. Restoring an older backup brings the larger copies back. Both are harmless.
- **Never touched:** the photos shown on screens (512 px), question pictures (they fill the TV), GIFs, see-through pictures, and anything that changed while it was being checked.
- **The steps:** it checks first and asks with the numbers. Then it makes a safety backup ("Before Make Photos Smaller"), replaces the pictures and saves. If the safety backup can't be made, nothing is changed.
- **Your data:** on the 10 October backup, 13 of the 14 originals shrink, 2.5 MB → 1.2 MB. A backup of that data then goes from 10.5 to 5.7 million characters.

3 · Where the data is (item 4)

- **Backups & Restore → Right now** has two new rows:
  - **Player Data:** this laptop's file / the tablet's file / this browser's database / this browser, then size and last save. "Not saved yet" or "Loading…" when that is the case.
  - **Pictures:** how many there are and their size.

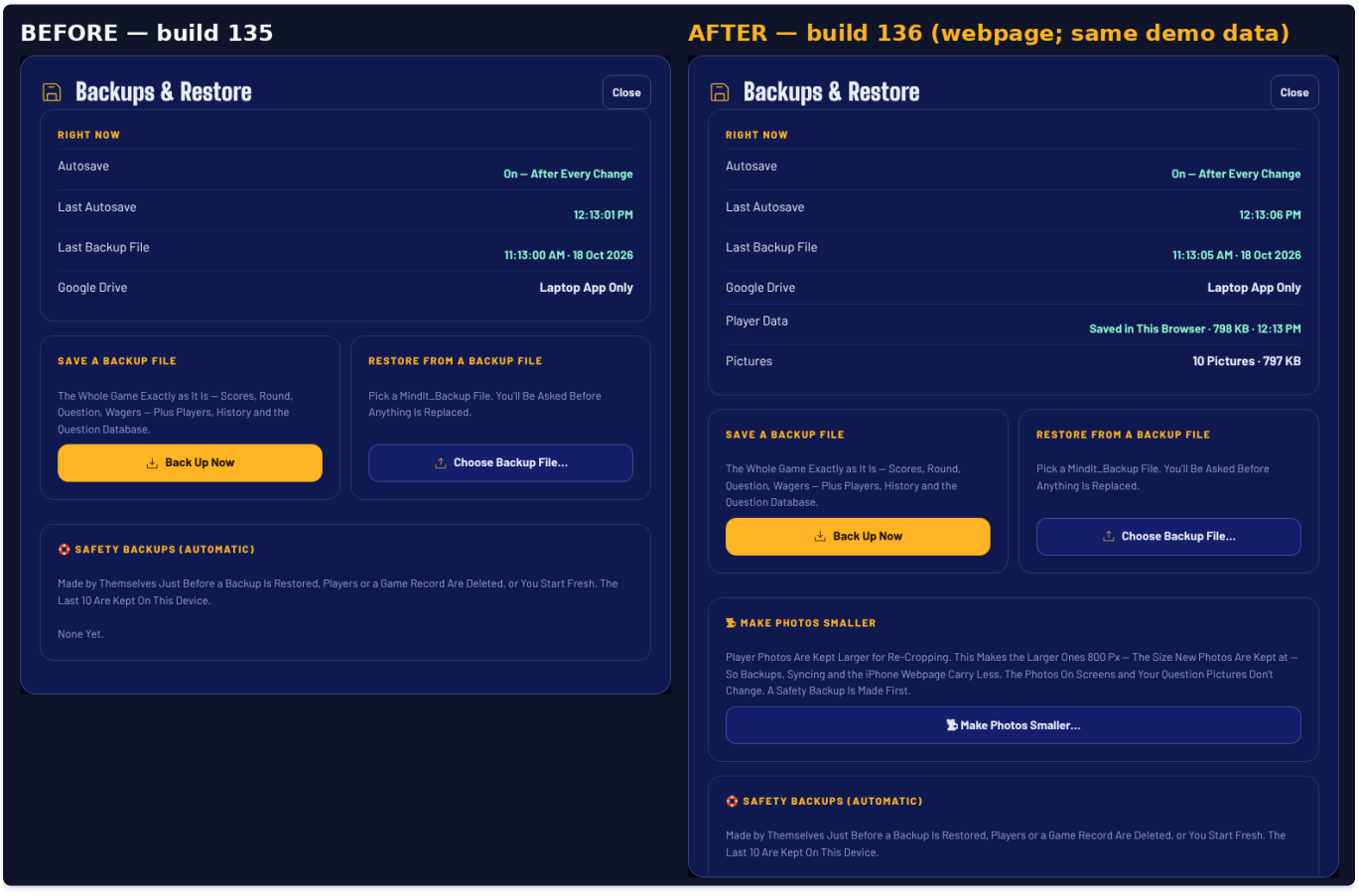
4 · Warnings on builds 129–134 (item 5)

- **How it works:** `releases/notices.json` and a new `notices` step in the release workflow (`mindit_release.py notices`) put "Don't install this release's Galaxy Tab app" at the top of those releases' notes. A release that already has the warning is skipped.
- **Unchanged:** files, tags and dates.
- **Why the workflow does it:** releases can't be edited from the build machine's session.
- Tried first against stand-in releases (all six get the warning once; a second run changes nothing); applied by the build-136 publish and checked afterwards.

5 · MindIt-Downloads public

- Repository settings can't be changed from the build machine's session, so you switch it yourself: Settings → General → Danger Zone → Change visibility.
- What becomes public with it was listed for you before you switched.
- Done by the owner on 18 October; a download link opened without signing in (checked from the build machine).

Tests: Final full run of **127 suites on the page that ships (Online Mode included): all 2,813 checks passed, every suite on its first try** (83 minutes) — the same page that is inside the Windows, Mac and Android apps. Before Online Mode was added, a full run of the 125 suites then in place passed 2,767 of 2,768: the one miss was the build-111 check that backups hold no pointers, which build 136 changes on purpose (that check now asks that every picture is inside the file, a shared one stored once: 20/20), and `fix130a` passed on its automatic retry (it has missed "all three show the same picture" in earlier builds too). The storage check `storq` now opens the page from a site address: from a file:// address the test browser sometimes gave a new page an empty localStorage while the other pages still saw it (seen with a probe), which made the migration step read nothing; over http it passed 5 runs out of 5. New suites: **`fix136o` (26/26)** — the presets, your seven settings, the rest as Family Night had them, 2 rounds + Final with the usual values, the Phoenix switch hidden and off, it stays Online Mode when you adjust, Quick Setup's button, Classic and Full Show unchanged against the build-135 page, remembered, a game starts with the Speed Bonus, Standard answering, no phone QR on the TV, How to Play, the wager card's cursor; **`fix136od` (6/6)** — in the laptop app the Final Round shows the QR code in Full Show and the "Send your wager" box in Online Mode, no join code, and no Phoenix Chip at Round 2 in Online Mode (Full Show gives one); **`fix136` (27/27)** — lighter backups, restored in build 119 (the rollback build) and 132–136 with every picture back byte for byte, Make Photos Smaller, the new rows; **`fix136d` (7/7)** — the same in the laptop app with its player file. **Apps:** Windows EXE 69,529,814 B (page and `main.js` inside identical; joiner tested under Wine 4/4); APK versionCode 136 signed with your original key (v2 + v3), `apk_check.py` and Google's `aapt2` read the name "Mind It!" and `.MainActivity` ; Mac zip 100,806,283 B (arm64, ad-hoc signed, page and `main.js` inside identical).



Build 135 addendum (18 October 2026)

Your requests:

- "Do them all, comics background added" — the eight backdrops, the remote build warning, the guide screenshot swap, steadier tests, tidied docs.
- **Android:** "Critical Android APK Launch Failure & Packaging Fix", with screenshots showing the name "application" and "application closed because this app has a bug".
- **iPhone:** "Fix iPhone Web App Storage Full Error & Player Profile Saving".
- "do all the fixes in this build".

The Comic Book picture you sent was not used. It is a fan artist's signed artwork of Marvel and DC characters, and the app is published on a public GitHub Pages site. The Comic Book backdrop is an original drawing instead.

1 · The Android app (builds 129–134)

- **Root cause, confirmed in the file:**
  - A compiled Android manifest stores each text attribute twice: a *raw* string index and a *typed value* (TYPE_STRING, data = string index). Android reads the typed value.
  - Since build 129, `axml_fgs.py` adds `android:foregroundServiceType` to the compiled manifest. It inserts one string into the string pool and moves every later *raw* index up by one, but it left the *typed* values where they were.
  - On the device every string therefore pointed one entry too early:
    - the app's label read "application", exactly as on your screen;
    - the activity class read "activity", which doesn't exist, so the app closed at once;
    - most permissions shifted to the neighbouring name.
  - `aapt dump badging`, androguard, the build checks and the release workflow all print the raw strings, so all of them showed "Mind It!".
  - **Confirmed independently with Google's `aapt2 dump xmltree`**, which prints the typed values. The build-134 APK reads label "application", activity "activity", service "service". The build-135 APK reads "Mind It!", ".MainActivity", ".HostService".
- **Fix:**
  - `axml_fgs.py` now moves the typed values too, including CDATA.
  - New `apk_check.py` (standard library only) reads the manifest like a device. It checks:
    - every TYPE_STRING value equals its raw string;
    - the name is "Mind It!" and the icon is a resource;
    - the launcher activity is `.MainActivity` (MAIN + LAUNCHER, exported) and is a class in `classes.dex`;
    - the service is `.HostService` with connectedDevice;
    - all permissions, package, versionName, versionCode, min/target SDK;
    - `resources.arsc` is stored and aligned, and the assets are present.
  - It runs at the end of `android/build.sh`, so the build stops on a fault. The GitHub release workflow (`mindit_release.py` validate) also runs it, so a faulty APK can't be published.
  - On the build-134 APK it reports the device-side label "application" and fails. On build 135 it passes.
- **Unchanged:** package `com.mindit.app`, the original signing key `5f4819e4…`, minSdk 24, targetSdk 35. It updates over the broken install and the tablet keeps its data.
- **Not verified on a device:** no Android device, emulator or system image is available on the build machine.

2 · iPhone "Storage Full" and player profile saving

- **Root cause, reproduced in Chromium with Safari's quota simulated:**
  - `profSave()` wrote the whole player database (players, every photo as base64, the Question Database, history) to localStorage as one JSON string. It called **any** exception "storage is full".
  - The iPhone/iPad remote page (`/tablet?remote=`, Connect Device → iPad / iPhone) ran `libSyncNow` on every welcome. That pulled the laptop's complete database: your 10 October backup is about **7.0 million characters**, about 14 MB as Safari counts it, against Safari's 5 MB per site. It then saved it 2–3 times. Nothing on that page ever reads the copy.
  - The laptop page also showed the false message once its database outgrew the browser's quick copy, though its file was written.
  - A stale quick copy on the laptop could, in theory, be saved over the newer file in the first second after opening.
- **Fix (`app126.js`, with small hooks in `app19.js` and `app25.js`):**
  - **1 · Remote page:** no database sync on the phone/iPad browser remote. The Galaxy Tab app (it has a file) still syncs, and re-syncs automatically at most every 10 minutes.
  - **2 · Storage chain:**
    - **Android:** tablet file first.
    - **Laptop:** the laptop file. It is read before anything is written, every write is checked (`{ok:true}`), and an out-of-date quick copy is removed once the file confirms.
    - **Browser:** a large database goes to IndexedDB `mindit-profiles` (`meta` = everything but pictures, `img` = one entry per existing picture id). The write is read back before the old localStorage copy is removed, and a small pointer `mindit.profiles.store` records where the data is. Small databases stay in localStorage exactly as before.
  - **3 · Error kinds:** browser quota (only a real `QuotaExceededError`), storage blocked (Private Browsing), database couldn't open, data couldn't be packed, laptop/tablet file not written, laptop file couldn't be read. Each has its own message, shown once per session. The same classification covers Star Of The Show clips, the rehearsal safety copy and the autosave note.
  - **4 · Nothing is lost:** the data stays in memory, retries run at 5 s, 15 s, 30 s and 60 s, and a bar offers **Try Again** and **⬇ Save Backup File**. It saves when the page is hidden or closed. If a stored copy can't be read, nothing is saved over it.
  - **5 · Fewer writes:** a save with unchanged content writes nothing, and on a large database saves within 150 ms are written once.
  - **6 · New photos:** JPEG unless they really have transparency (Safari can't make WebP), within about 360 KB. Existing pictures and IDs are untouched.
  - **7 · Webpage offline cache (`sw.js`):** the page is kept once under `./index.html` instead of one 3.4 MB copy per address.
- **Not verified in real Safari:** no WebKit is available on the build machine. The iPhone remote and Safari's limit were emulated in Chromium.

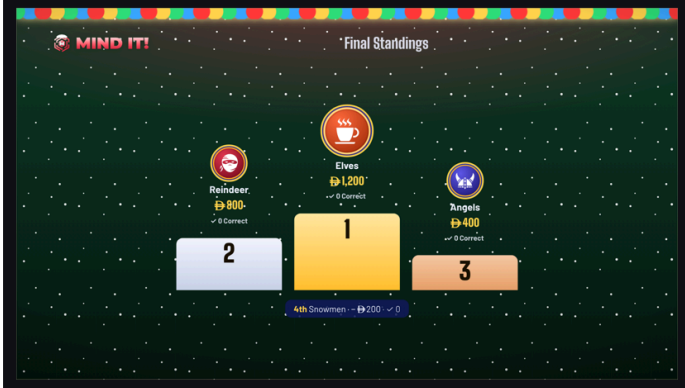
3 · Backdrops, remote warning and the rest

- **`app125.js`:** eight original inline-SVG scenes through `sceneTheme()`: a scene band along the bottom, sky items and movers, keyframes with theme prefixes. Each stays within about 14 KB of new drawing.
  - Each was checked behind the quiz at 1920×1080: still with reduced motion and with Subtle intensity, and every layer in the picture.
  - The host's link state now carries `build` and `hostWord`, so the remote shows ⚠ **Different Builds** at iPhone width too.
- **Guide:** the build-120 screenshot with real first names was replaced by one with the demo cast. The guide also gained text for the tablet fix, the storage messages and the backdrops.
- **Tests:**
  - `imp` / `libx` now wait for each import step.
  - `fix134` / `fix134d` follow the page's own build number.
  - The Intermission loudness comment is corrected to about 7 dB (measured).
- **Docs:** the stale "next task" section and the build-132 publishing note in CURRENT_STATUS are fixed.

Tests: Full run of **123 suites: all 2,732 checks passed** (82 minutes), on the final build-135 page — the same page that is inside the Windows, Mac and Android apps. One older suite passed on its automatic retry: `fix128d` (build 128) reads the 📱 Add Phone Remote QR code from a screenshot, and once it couldn't read it. the check now also tries the picture with a white margin and with OpenCV's second QR detector; it then passed 35/35 five times in a row, first try each time. An earlier full run, on the first build-135 page, stopped in two suites: `heic` (the new photo saving had pushed the iPhone HEIC conversion aside — fixed, 5/5) and `fix130c` (its reference page had moved out of the downloads repo — fixed, 7/7); the 12 photo, storage and sync suites were then re-run on the new page, 134/134. New suites: **`fix135` (27/27)** — the eight backdrops (each within its size budget, still with reduced motion and with Subtle intensity, every layer in the picture), the remote's build warning (also at iPhone width) and the tablet server passing on its build; **`storq` (17/17)** — the webpage with Safari's 5 MB limit simulated: a 6.8-million-character database moves to IndexedDB and comes back identical (same player and picture IDs), an older build's 2.2-million-character copy moves safely, the true messages (browser limit, Private Browsing, database won't open, unexpected error — never "full" unless it is), Try Again saves what was waiting, ten saves become one write, the phone remote page never asks for the database, JPEG within budget; **`fix135d` (7/7)** — the laptop app's server and the warning; **`fix135s` (9/9)** — the real laptop app with a 6.8-million-character database (file written and confirmed, an old quick copy never overwrites it, a failed write keeps the change and Try Again saves it) and an iPhone remote with Safari's limit (no database copy, no warning, also after a reconnect). **Apps:** Windows EXE 69,525,950 B (page and `main.js` inside identical; joiner tested under Wine 4/4); APK versionCode 135 signed with your original key (v2 + v3), `apk_check.py` and Google's `aapt2` read the name "Mind It!" and `.MainActivity`; Mac zip 100,802,327 B (arm64, ad-hoc signed, page and `main.js` inside identical, symlinks kept).

Build 135 · theme backdrops on the TV (Final Standings) · before → after

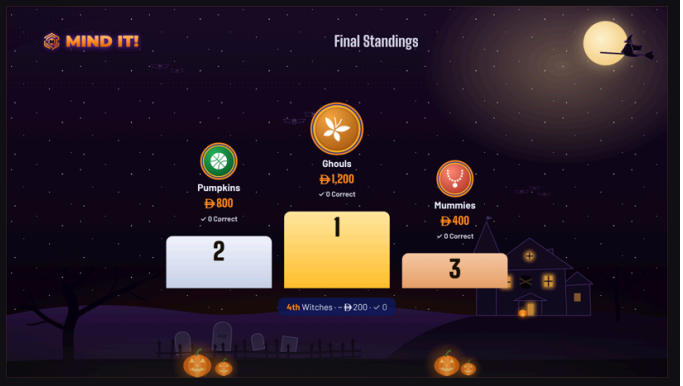
Christmas — BEFORE (build 134)



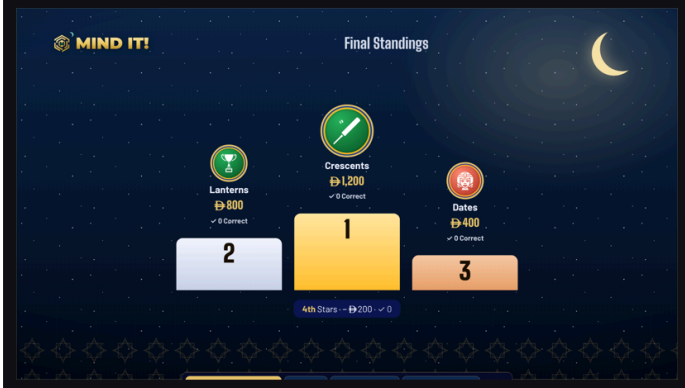
Christmas — AFTER (build 135)

Halloween — BEFORE (build 134)

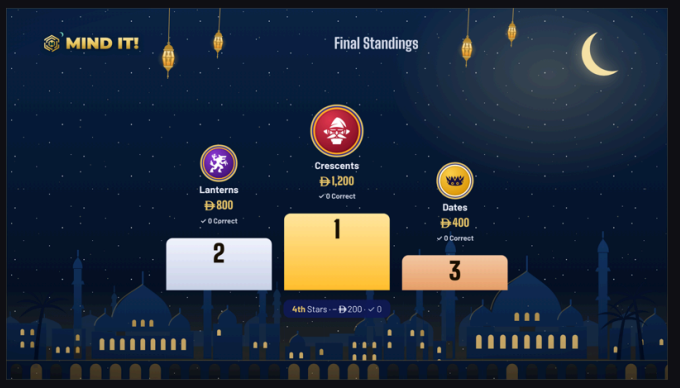
Halloween — AFTER (build 135)



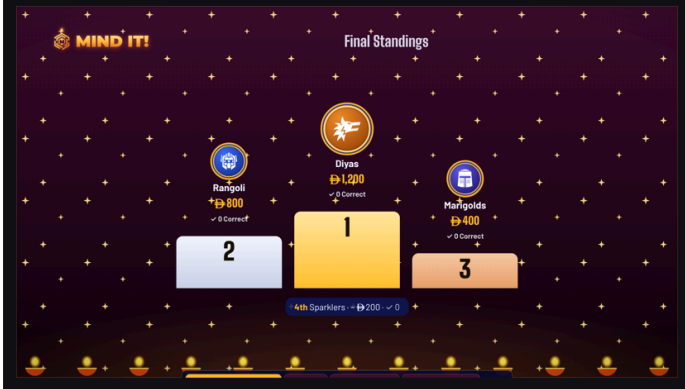
Ramadan / Eid — BEFORE (build 134)



Ramadan / Eid — AFTER (build 135)



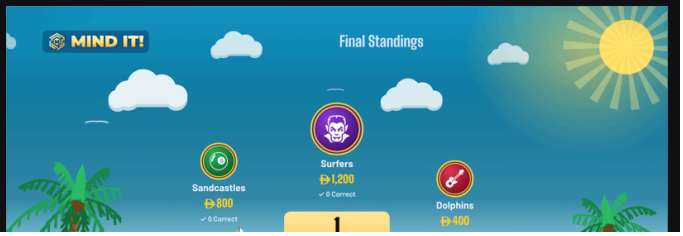
Diwali — BEFORE (build 134)

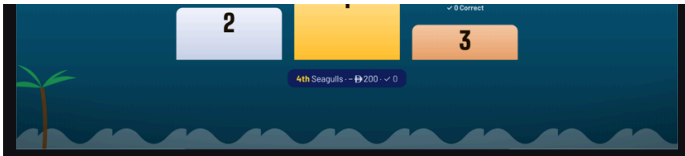


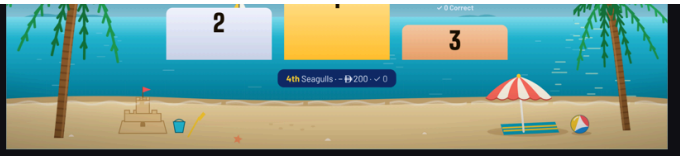
Diwali — AFTER (build 135)

Summer — BEFORE (build 134)

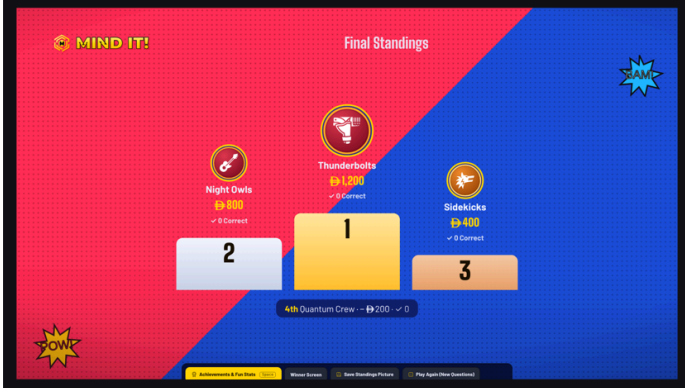
Summer — AFTER (build 135)



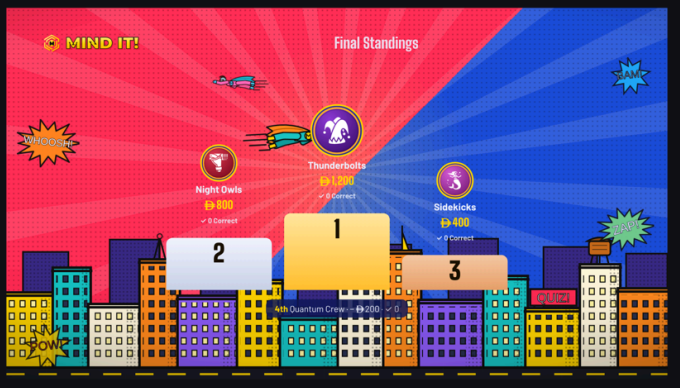




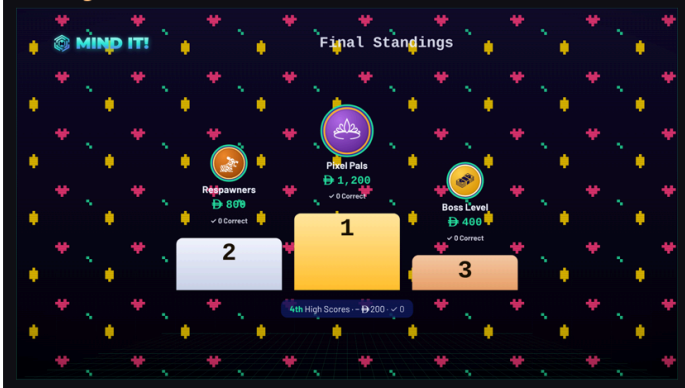
Comic Book — BEFORE (build 134)



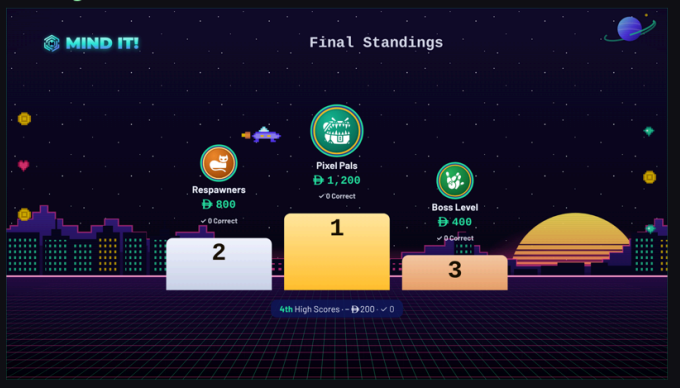
Comic Book — AFTER (build 135)



Gaming — BEFORE (build 134)

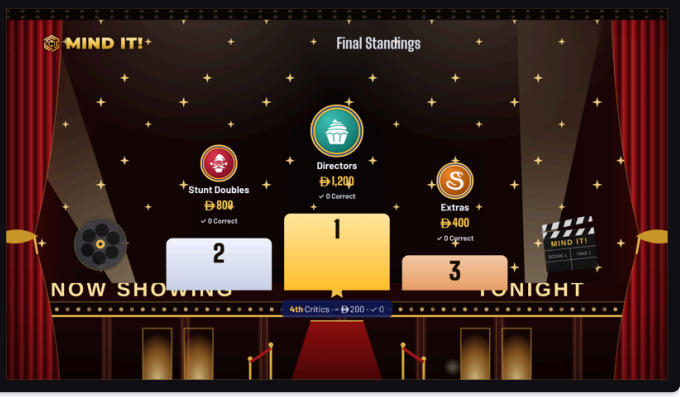


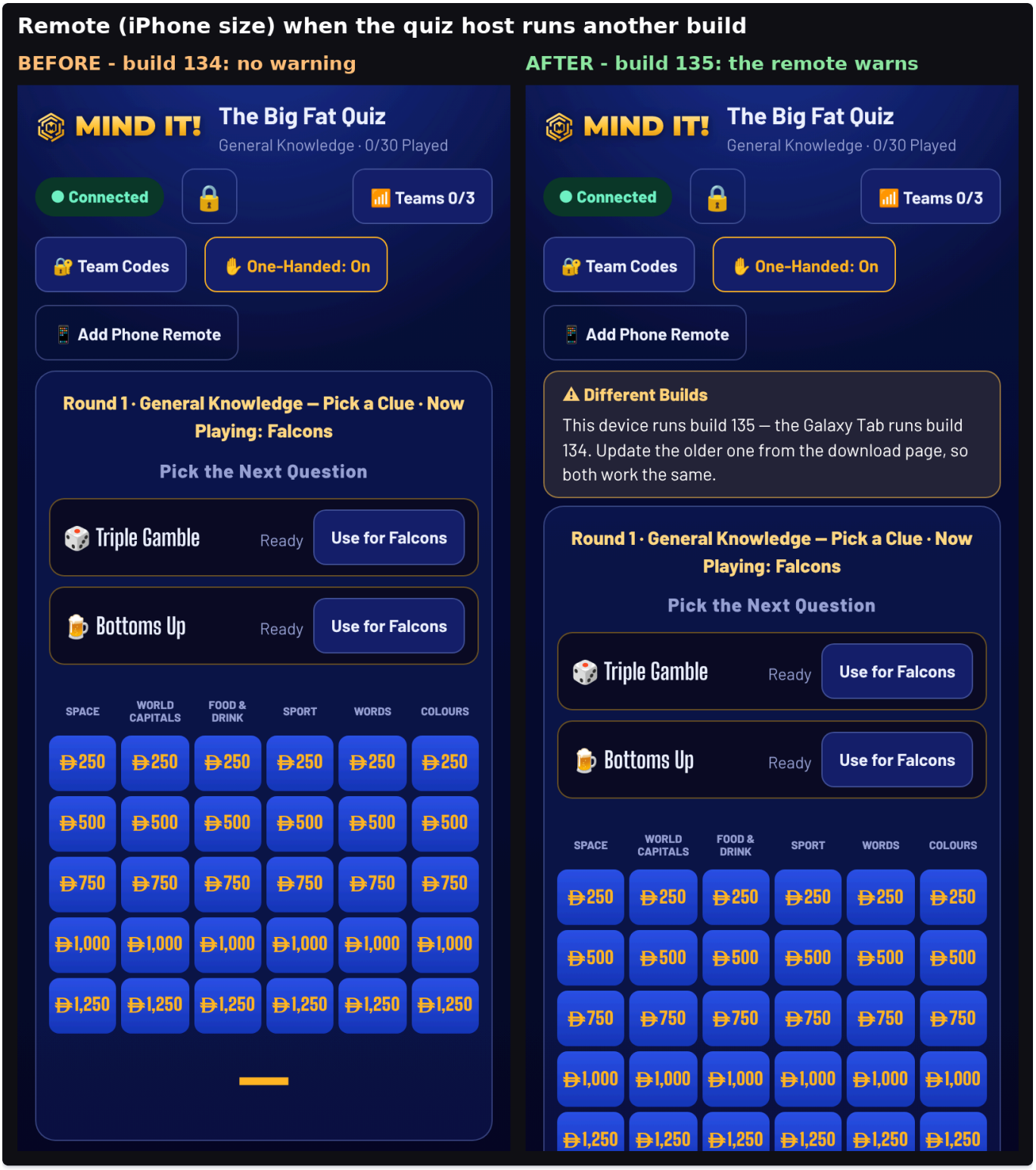
Gaming — AFTER (build 135)



Movies — BEFORE (build 134)

Movies — AFTER (build 135)





Build 134 addendum (17 October 2026)

Your request: "sting 1 is fine · use loop 1 · and do all the below upgrades", then "go ahead with all, dont wait for my approval". The upgrades:

1. the build number in About & Credits;
2. a warning in Connected Devices about mixed builds;
3. a magical castle backdrop for the wizarding theme, with owls and magical creatures;
4. your image as the comic book backdrop — **not done: the image never arrived**;
5. remove the old build-131 app files from the downloads repo;
6. make the downloads repo public;
7. steadier test runs.

What changed — exactly these files

- `app124.js` (new):

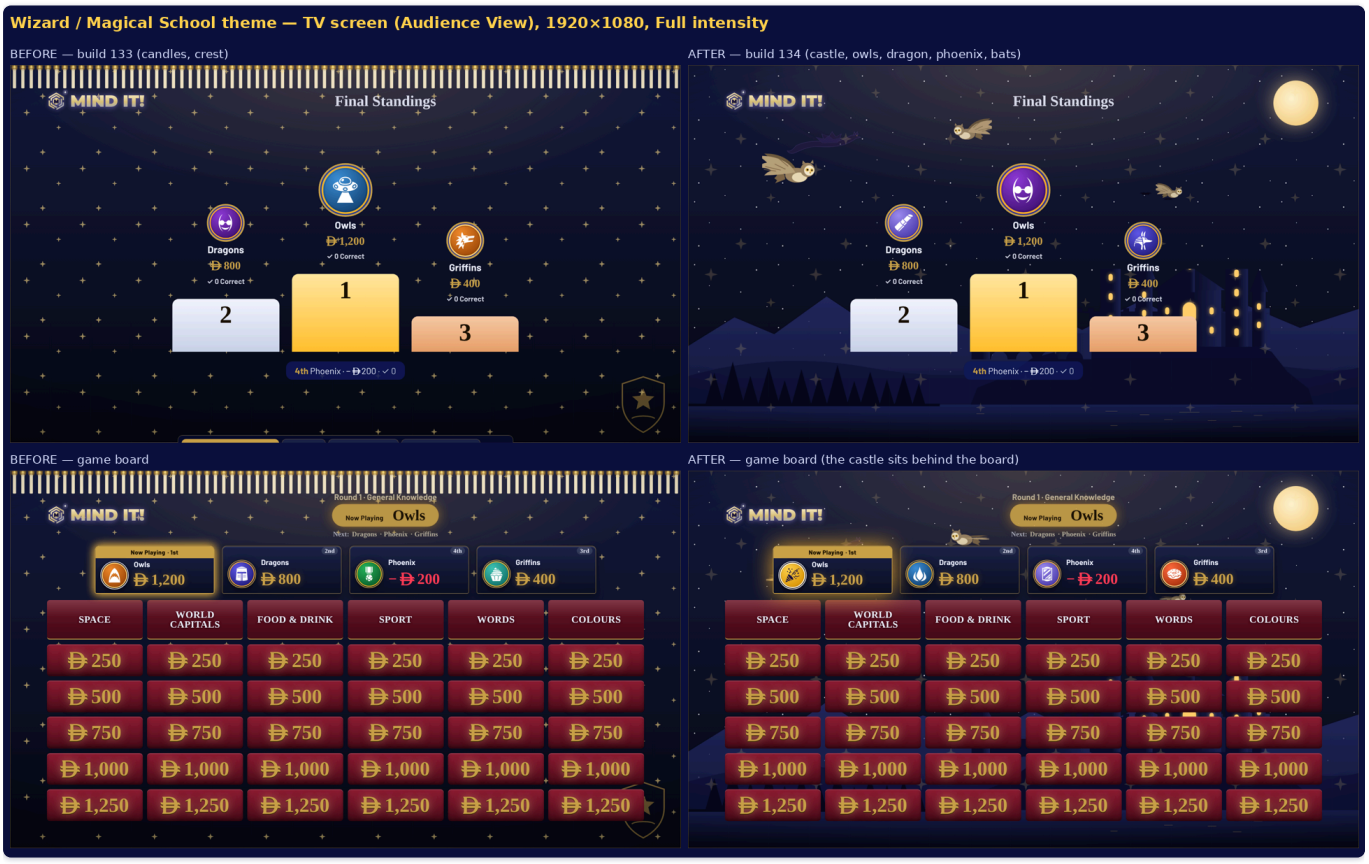
- **Wizard / Magical School backdrop.** An original inline-SVG night scene, replacing the candles and crest: a castle on a cliff with lit, flickering windows, a moon and a lake. Flying across it are three owls (wings flap), a distant dragon, a phoenix that rises off the top before reaching the logo, and a small group of bats. Everything sits behind the quiz.
  - The castle is drawn generically: no film's castle, crest or character.
  - With reduced motion or Subtle intensity, nothing moves and every creature stays in the picture.
- ☕ **Intermission music.** `LOOP_MP3` is the theme's 19.2 s loop (Quiz Room Riff, cut on exact samples 195422–1042125, level evened out). It is 308,079 B, SHA-256 `d9177370…`.
  - It plays while Intermission is on, where the quiz's sounds play. Remotes never play it.
  - Level: volume × 0.337 (about −25 LUFS at the default volume). It fades in over 2 s and fades out over 1.5 s.
  - It follows the volume slider, Sound Off and your own theme music.
- **The build number.** `APP_BUILD=134`, shown in About & Credits ("1.69 · Build 134") and Setup → About.
- **Mixed builds.** Connected Devices shows **⚠ Different Builds** when a connected remote reports another build. A remote that reports none (133 or older) is shown as "an older build".

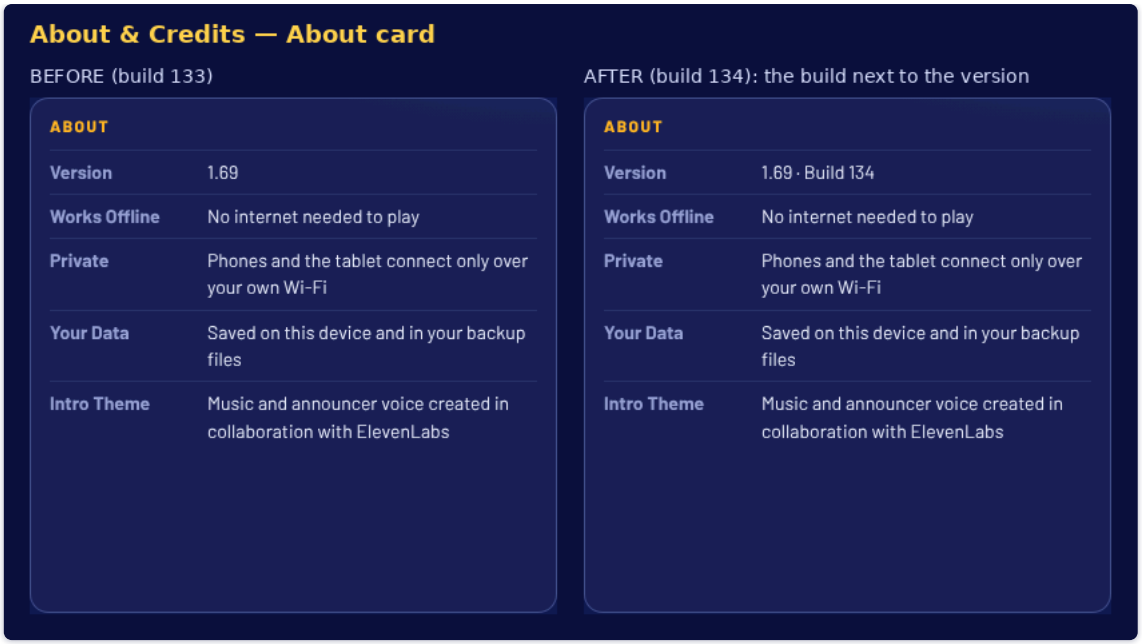
- `app25.js`: a remote now says its build when it connects.
- `app121.js`: the Rehearsal Report uses `APP_BUILD`.
- `style.css`: the warning box.
- **Laptop app `main.js` and tablet `MindItHost.java`:** they pass each remote's build on to the quiz page. Their transfer `BUILD` stays 132, so builds 132, 133 and 134 can still move a quiz between each other.
- `tests/run_all.py`: the laptop-app suites now wait until the app answers on its debugging port with its page open, instead of a fixed 13 s.

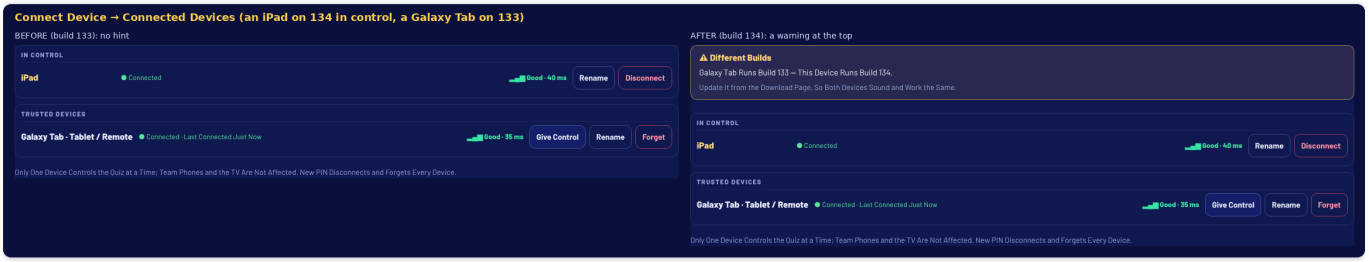
Before/after pictures: `BUILD_134_BEFORE_AFTER_WIZARD.png`, `BUILD_134_BEFORE_AFTER_BUILD.png`, `BUILD_134_BEFORE_AFTER_DEVICES.png`.

Tests: Full run of **119 suites: all 2,678 checks passed** (81 minutes). Two older suites passed on their automatic retry: `imp` (a dialog was re-drawn while its button was being clicked) and `libx` (the import check read the dialog before it had finished). Both were run twice more afterwards and passed 8/8 each time; nothing in build 134 touches them. The new suite **`fix134` (24/24)** checks the loop file byte for byte, starts and stops the Intermission music (fade, volume, Sound Off, own theme music, remotes stay silent), the build number in About & Credits, Setup → About and the Rehearsal Report, the wizarding backdrop (every creature in the picture and behind the quiz, still with reduced motion) and the tablet server passing on each remote's build. **`fix134d` (8/8)** does the same for the laptop app's server and the warning ("this laptop runs build 134"). **Apps:** Windows EXE 69,499,660 B (page and `main.js` inside identical; joiner tested under Wine 4/4); APK versionCode 134 signed with your original key (v2 + v3); Mac zip 100,768,202 B (arm64, ad-hoc signed, page and `main.js` inside identical, symlinks kept).

Not tested: the real laptop, Galaxy Tab, phones and TVs. The Intermission music hasn't been heard on real speakers, the castle backdrop hasn't been seen on a real TV browser, and the warning hasn't been seen with a real Galaxy Tab on another build.







Build 133 addendum (17 October 2026)

Your request: "use 5a and release it" — the intro theme preview you approved (*Option A – mix 5a*) becomes the app's intro, and the release goes out. Nothing else in the app changes.

What 5a is:

- the ElevenLabs music *Quiz Show Riff* (Option A) for the riff and the quieter groove;
- your ElevenLabs *Dark Villain, Cinematic* take of **"MIND IT!"**, with the voice's speed, pitch and wording unchanged; the music dips under it;
- then the music stops for the Mind It! explosion, made louder and more booming;
- one bar later, the last riff.

What changed — exactly these files

- `app12.js` :
  - **The three sounds.** 5a is split into the app's three parts, at the levels the app plays them at the default volume (music ×1.0, voice ×0.704, explosion ×1.0). Playing them at their places rebuilds 5a. The parts are listed in the table below; nothing of the old three sounds is left in the page (1,233 distinctive pieces checked).
  - **Timings:**
    - the voice enters at 8.55 s of the music (`ROCK_VOICE` ; `VOICE_AT` 5.35 for the sting);
    - `VOICE_END` 1.06, so the explosion lands at **9.63 s**, right after "IT!";
    - the loading loop is the riff's two-bar hook, 1.619–4.819 s, cut on exact samples where the join is seamless;
    - `INTRO_END` 13.04: the intro page and the 🎵 sting now last until the last riff has finished (the intro page about 8.7 s instead of 6.8 s; the sting about 9.8 s).
  - **Build 132's dip under the first line is off** (`MUSIC_DIP_DB` 0): the new music dips under "MIND IT!" by itself, as in 5a.
  - **If the music can't be loaded**, the voice comes in at once instead of after 5 s of silence.
- `app123.js` (new): ★ About & Credits → About card → *Intro Theme — Music and announcer voice created in collaboration with ElevenLabs* (the credit ElevenLabs asks for). Before/after: `BUILD_133_BEFORE_AFTER_CREDITS.png` .
- `app121.js` : the 📋 Rehearsal Report says build 133.
- `main.js` and the tablet server are unchanged, so their `BUILD` stays 132 and a 132 device and a 133 device can still move a quiz between them.

Part (in `app12.js`)	Size	Length	SHA-256
`ROCK_MP3` (music)	314,140 B	13.04 s	`c8bfc9a0…`
`VOICE_MP3` ("MIND IT!" + its room tail)	31,808 B	1.91 s	`3913b3cb…`
`BOOM_MP3` (explosion)	33,062 B	2.01 s	`f9951ce4…`

How it was checked (build machine):

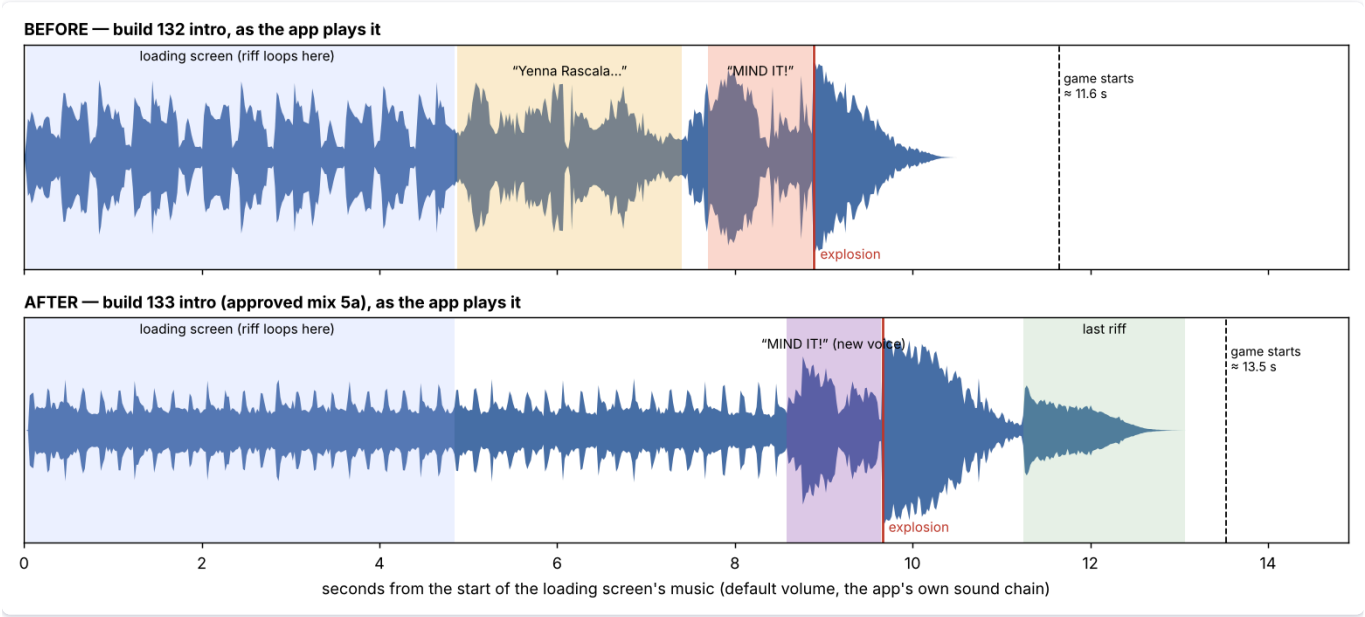
- **The intro rendered offline through the app's own code and sound chain** (Chromium, the real `continueIntro` / `playSting` , default volume), then compared with the 5a file you approved:
  - every part (riff, groove, "MIND IT!", explosion, last riff) lands at the same moment, to the sample, with correlation 0.994–0.998;

- once the levels are matched, what's left is MP3 coding (−20 to −24 dB);
- the app's sound chain plays everything 1.1–1.4 dB louder, and the explosion about 0.6 dB less than that (it was the same for the old intro).

- **Balance in the app:** the explosion is 9.6 LU over the riff (5a: 10.3) and "MIND IT!" 4.2 LU over the riff (5a: 4.3).
- **Loudness in the app at the default volume:** −15.3 LUFS (build 132: −14.2). The music is about 1.7 dB quieter, and the explosion louder, than in build 132.
- **Before/after:** `BUILD_133_BEFORE_AFTER_INTRO.png` .

Tests: Full run of **117 suites: 2,635 of 2,637 checks passed** (83 minutes). The 2 misses were older checks that assumed build 132: `fix132s` waited a fixed 9 s for the 🎵 sting to end on a Wi-Fi TV, and the new sting lasts 9.84 s; `fix132r` expected the Rehearsal Report to say build 132. Both were updated and re-run: 24/24 and 19/19. `desk` passed on its retry (the laptop app was still starting the first time). The new suite **`fix133m` (32/32)** replaces `fix131v` : it checks the files byte for byte, renders the intro and the sting through the app, and covers the loading loop (the repeat is identical and the join has no click), the no-music fallback, loading → intro → game in real time (the intro page stays while the last riff plays), Skip Intro, the winner screen, the Rehearsal Report and the credit at laptop and phone widths. **Apps:** Windows EXE 69,187,013 B (page inside identical; joiner tested under Wine 4/4); APK versionCode 133 signed with your original key (v2 + v3); Mac zip 100,452,011 B (arm64, ad-hoc signed, page inside identical, symlinks kept).

Not tested: the real laptop's speakers, the Galaxy Tab, phones, a TV (HDMI or 🔊 Sound On This TV), and the iPhone webpage — the sound was measured only, not heard on a real device. The ElevenLabs licence for an app and for public or paid events depends on your ElevenLabs plan (Scale or Business; offline commercial use is excluded on every plan).



Build 132 addendum (17 October 2026)

Your request: "do them all and for 2 dip the music slightly" — the six upgrades offered after build 131:

1. 🔊 sound on a Wi-Fi TV;
2. a clearer first line in the intro — **your choice: dip the music slightly** under it;
3. 📋 a Rehearsal Report from the real devices;
4. a shorter TV address;
5. the remote lock as a small icon;
6. handing the quiz back from the laptop to the Galaxy Tab.

What changed

- **Page:**
  - `app120.js` (new): TV sound;
  - `app121.js` (new): the report;
  - `app122.js` (new): the hand-back;
  - `app12.js` : the dip, and the custom sting routed to the TV;
  - `app117.js` : the audience panel — the sound button and the short address;
  - `app119.js` : the lock icon and the short-address check;
  - `app118.js` : the transfer texts for both directions;
  - `app116.js` : the remote's offer message and its name as "Laptop";
  - `app40.js` : the Connect ▾ items;
  - `app63.js` ;
  - `style.css` .
- **Laptop app** `main.js` :
  - port 80;
  - the TV sound relay;
  - `/api/host/diag` ;
  - the hand-back source with its fencing;
  - `BUILD` 132.
- **Android:**
  - `MindItHost.java` : the TV sound relay, the hand-back target, and two fixes;
  - `MainActivity.java` : `openHostPage` , and more in `hostStatus` ;
  - `HostService.java` : the Wi-Fi announcement status;
  - versionCode 132.

The dip (item 2): the music goes down 5 dB from just before the first line to the end of the pause, then comes back for "MIND IT!" and the explosion. Measured on the offline render:

- the first line now stands about **+6.6 dB** over the music (build 131: about +2 dB);
- "MIND IT!" stands about **+7.3 dB** over the music;
- everything outside the dip is identical to build 131.

The two preview MP3s (before / after) are in the chat.

Bugs found while building, and fixed

1. **A Galaxy Tab whose quiz had moved to the laptop could never host again** (builds 130–131). The tablet's server kept its "moved" record forever:
   - it kept sending every phone, remote and TV to the laptop — also for a new quiz days later;
   - the tablet's own quiz page turned itself into the laptop's remote as soon as it opened.

   Now:
   - the record is closed (kept as history) when the quiz comes back, when a quiz is started there, or when a set-up quiz is open there more than 6 hours after the move;
   - only the quiz that actually moved turns a page into a remote.

   The same applies to the laptop after it hands a quiz to the tablet. `fix132t` checks both.
2. **The tablet's server crashed while listing devices if a trusted remote wasn't connected** (builds 130–131). The cause was a Java conditional that unboxed a missing number. The crash cut off:
   - the remote that had just connected;
   - the tablet page's own remote link — again and again.

   Seen in testing: an iPad dropped 13 times in 8 seconds. Fixed; `fix132n` reproduces it — it fails on the build 131 code and passes now. The laptop was never affected.
3. The optional **theme music** (Setup → Theme music) played from the laptop even when a TV had the sound; it now goes to the TV too.
4. The audience panel's buttons stretched when the sound button was added; fixed before the screenshots (see the before/after sheet).

Transfer compatibility: `BUILD` is now 132 on both sides. A 131 device and a 132 device politely refuse to move a quiz between them, so update the laptop and the tablet together. Everything else works across 131 and 132.

📸 *Before and after (all layout changes, plus the new screens):*

Check	Result
`fix132s` (new, laptop app) — **TV sound + short address.** The laptop answers on port 80: `<ip>/tv` opens the TV page, the bare address lands there too, and Quizmaster-only addresses are refused on both doors. The panel shows `<ip>/tv` with the long form underneath. **Sound On This TV:** the TV asks for OK first, while the laptop keeps the sound. OK pressed: the TV plays, and the laptop's speaker gain goes to 0. The sting reaches the TV (66 pieces of sound, up to −11 dB at the TV's speakers); the TV goes quiet after it (silence isn't sent); a single effect arrives within a second; the theme music plays there too. The TV drops: the sound is back on the laptop within seconds, with a note. It reconnects and asks for OK again. Another TV takes the sound (one at a time); Sound Back Here; a TV can't send sound or commands into the quiz	**24/24**
`fix131v` (updated) — the intro: the page carries your clip; music and explosion are identical to build 131 outside the dip; about −5.0 dB inside it; the dip is in the intro and the sting	**22/22**
`fix132r` (new, laptop app) — **Rehearsal Report:** ready in seconds; the file name; this device; the app file's SHA-256; system, port and short address; the Test Hosting rows; the TV with its own browser, size and error list; remotes, team phones, sound, tablet discovery and errors. **No** team codes, PIN, device keys or folder names; a plain page with no scripts; Save Report File; also in Connect ▾	**19/19**
`fix132t` (new, laptop app + simulated Galaxy Tab) — **Hand Quiz to the Galaxy Tab**, after a build-130 take-over: the menu item; the list of Galaxy Tabs; the offer; Not Now; Cancel; accept. The laptop pauses scoring; the tablet takes exactly the same game with the countdown running; both phones move and keep their team, and stay there; the tablet hosts again (its earlier "moved" record closed); the laptop becomes the remote in control, named "Laptop"; the iPhone remote follows and waits; names and the TV follow. The laptop retires and refuses or redirects late phones; host generation 2; the laptop as a remote scores on the tablet. The laptop's start screen says where the quiz went, it doesn't turn itself into a remote, and a new quiz on it hosts normally. **Failure before the hand-over** (the quiz can't be opened on the tablet): the laptop keeps hosting with the same scores and countdown, the phone stays, and the tablet goes back to being the remote with one tap. **Tablet app stopped just after the hand-over:** the laptop accepts no scores meanwhile; the tablet restarts and finishes by itself, with the same scores, and the phone follows	**57/57**
`fix132n` (new, tablet server + Chromium) — a trusted iPhone goes away; an iPad then connects and stays connected (no drops); the tablet's own link stays up; Connected Devices lists both. The same check on the build 131 tablet code: 4/7 (the iPad dropped 13 times in 8 s)	**7/7**
Full regression	**117 suites, 2,618 / 2,621** (83 min). The 3 misses were older checks that expected the build 131 look, and were updated for the changes made on purpose: `fix130t` expected the tablet's TV address with `http://` in front (now shown without it); `fix130c` flagged the one existing style changed on purpose (room for the quiz name beside the 🔒 icon on narrow remotes — shown in the before/after); `fix119` counted the 🔒 icon as part of the tablet's button row (it now sits beside Connected). Re-run: 13/13, 7/7, 12/12. `fix130a` passed on its second try: its Wi-Fi-drop step closed the TV's connection while already offline, which sometimes left it closing for seconds; the step now closes first, then goes offline, and passed first time in 3 runs straight after other suites (41/41). **Layout audits:** host screens 90 views, 0 issues; Audience View 234 screens, 0 issues
Windows EXE	69,077,219 bytes, SHA-256 `96a5f52a…` ; the tested page (md5 `1433e6fd…`) and `main.js` (md5 `e185b083…`) inside; parts 23,025,740 + 23,025,740 + 23,025,739 bytes + `Join Mind It - build 132.bat` ; the joiner's fingerprint test under Wine 4/4

Check	Result
Android APK	versionCode **132**, versionName 1.69, 1,746,714 bytes, SHA-256 `ce65e019…` ; v2+v3, original key `5f4819e4…` ; foreground-service type kept; the tested page and the new tablet code (`openHostPage` , hand-back files) inside
Mac (Apple Silicon)	arm64, ad-hoc signed, the tested page and `main.js` inside; zip 100,344,693 bytes, SHA-256 `712700c7…` ; 5 parts re-join identically; **not run on a real Mac**

Tested here vs. needs real devices

- ✅ **Tested on the build machine:**
  - the laptop app's engine (Electron) with the page and `main.js` ;
  - the tablet's server code on the JVM with the tablet page in Chromium (bridge simulated);
  - TVs, phones and remotes as Chromium pages at real sizes.
- 🟡 **Needs your devices:**
  - **TV sound on a real TV browser** — whether it unlocks with the OK button, its delay, and whether it plays smoothly over your Wi-Fi;
  - Windows letting Mind It! use port 80 (if anything else uses it, the long address still works);
  - the Galaxy Tab app opening its own quiz page on Host It on This Tablet (`openHostPage` in Android's WebView);
  - the hand-back with a real Galaxy Tab;
  - the report's tablet part (`hostStatus` on Android);
  - **listening** to the intro dip on real speakers.
- **Rehearse at home:** Rehearsal Checklist rows 35 (short address), 37 (TV sound), 38 (hand back) and 39 (report).

Live acceptance: **PENDING USER LIVE TEST**.

Build 131 addendum (17 October 2026)

Your spec: "Replace Intro Vocals Only (Global Update)". Your new voice clip replaces the old intro voice everywhere; everything else stays exactly the same. **Your choices:**

- **Use the fitted version** (after listening to the before/after previews).
- **Republish the web link with 131** (outside the alternate-build turn, so every platform gets the new voice; the next turn is build 133).

The limitation (reported before building, as your spec asks): the old voice ran 3.1 s from its first word to the explosion; your clip runs 5.4 s, with a 1.2 s silent pause between its two lines. The music, the loading loop and the explosion had to stay unchanged, so the clip fits only if:

- the pause is shortened to 0.3 s;
- the voice starts at the music's build, about 1 s earlier than the old voice (it can't start any earlier, because the loading music is still looping before that point).

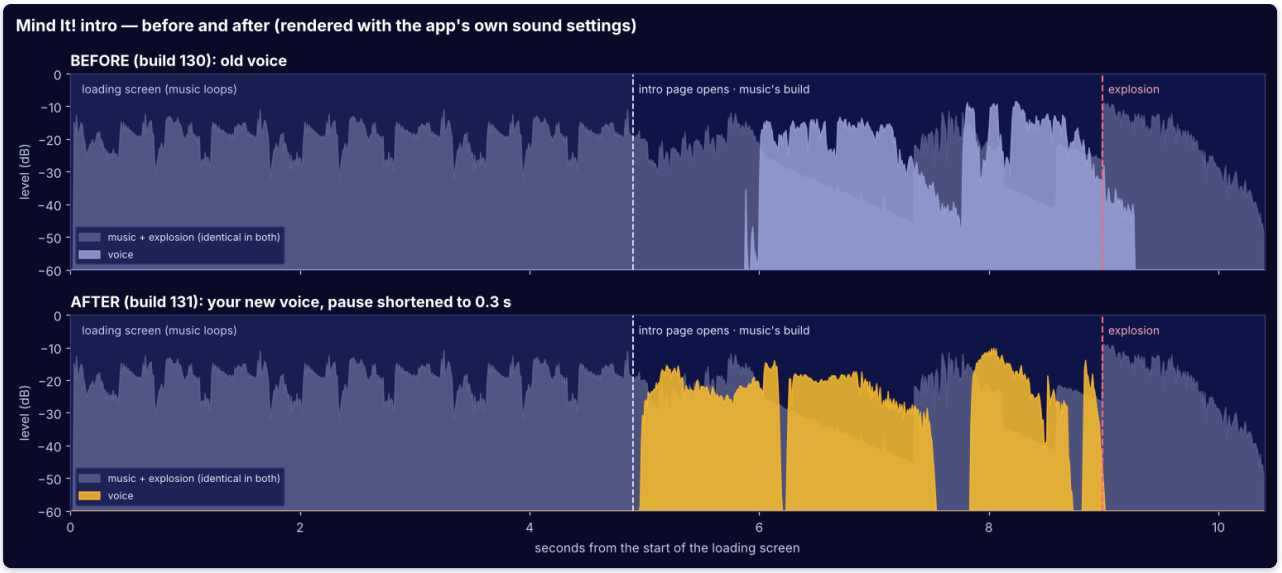
As a result, the first line plays over the build, where it stands about 2 dB above the music; the old first line, in a quiet gap, stood about 14 dB above it. "MIND IT!" stands about 7 dB above the music (old: about 9 dB) and ends right on the explosion.

What changed — exactly these files and references

- `app12.js` , `VOICE_MP3` : the embedded intro voice, the only copy anywhere, replaced by your clip. The clip was edited in two ways only:
  - the silent pause was cut from 1.23 s to 0.30 s (the cut is inside digital silence, so no spoken sound was touched);
  - the level was raised +11 dB with a gentle peak limiter (–1.5 dBFS) to match the old voice.

  The file is 72,351 bytes, SHA-256 `e97315fd…` ; your original is kept in the private source zip.
- `app12.js` , timing:
  - `ROCK_VOICE` 5.75 → 4.8 and `VOICE_AT` 2.55 → 1.6 (where the voice enters);
  - `VOICE_END` 3.09 → 4.04, so the explosion stays at **8.86 s of the music**, as before;
  - `VOICE_PAUSE` 1.46 → 2.57 and `VOICE_MINDIT` 1.89 → 2.87 (used only by the no-music fallback);
  - `continueIntro` now starts the voice at "the build or now, whichever is later", joining the clip at the same point, so it can never drift from the music.
- **Unchanged:**
  - the music `ROCK_MP3` and the explosion `BOOM_MP3` (same bytes);
  - the loop points, the music's start point, the music / explosion / voice volume settings, the loading time and the loading loop's jump to the build;
  - the intro's length; Skip Intro.
- **Every place the voice plays** goes through this one asset:
  - the loading screen → intro;
  - 🎵 Play The Mind It! Sting (Audience View menu, Setup → 5 · Audio & Presentation, the host panel's 🎵 button);
  - the theme sound preview;
  - the winner screen.

  The TVs never play sound (it comes from the laptop or tablet). If someone has uploaded a *Custom "Mind It!" sting* in Setup, the 🎵 buttons and the winner screen play that file instead, as before. Remove it there to hear the new voice on them.
- **No stale copy:**
  - the project, Android, desktop and documentation files were searched for the old recording: only the built page held it, and that page has been rebuilt;
  - the voice is inside the page, not a separate file or address, and is decoded into memory each time the app opens;
  - the Windows, Mac and Android apps carry the page inside them;
  - the hosted webpage's offline copy is replaced the next time it's opened with internet.
- `AndroidManifest.xml` versionCode 131. `main.js` and the tablet server are unchanged, so builds 130 and 131 can still hand over a tablet quiz to each other.



📸 Before and after: the intro rendered with the app's own sound settings (music + explosion in grey, identical in both; the voice in colour). The two preview MP3s are in the chat.

Check	Result
`fix131v` (new). **The page:** carries your clip byte for byte; none of the old voice's 252 distinctive pieces are left; music and explosion are the same files as build 130; loop points, levels and loading time unchanged; the explosion stays at 8.86 s of the music. **Offline render** (Chromium Web Audio, the app's gains): music + explosion identical to build 130 sample for sample (max difference 0); the voice enters at the build; "IT!" ends 0.01 s before the explosion; the decoded voice is 4.45 s with first line, 0.3 s pause, then MIND IT!. **In the app:** loading → intro → game, with the loading music started once and the voice once; the explosion 4.06 s after the build, as in build 130; 🎵 sting with the explosion 5.71 s after the start, as in build 130; played again reuses the same voice; Skip Intro stops music, voice and explosion at once; the winner screen plays it	**20/20**
Full regression	**113 suites, 2,526 / 2,527** (77 min). The one failure was in a test, not the app: `fix130a` 's Wi-Fi-drop step relied on the browser's offline mode to stop the TV reconnecting, and twice in this run (right after `fix129d`) the TV reconnected straight away. The step now blocks new connections itself. Re-run after `fix129d` three times: **41/41** each time (it had also passed 4 out of 4 times on its own). No app file changed. The layout audits were not re-run, because no layout changed
Laptop app (the Windows app's engine, on the build machine)	the real laptop app (Electron 33, the engine inside `Mind It!.exe` and the Mac app): sound running, loading → intro, music started once, voice once, decoded voice 4.45 s, voice at the build, explosion 4.06 s after the build — **7/7**
Windows EXE	69,063,574 bytes, SHA-256 `b4719d0b…` ; tested page inside (md5 `2885a82e…` , no old voice); `main.js` unchanged from build 130; the three parts re-join to the same file; joiner fingerprint test under Wine 4/4
Android APK	versionCode 131, versionName 1.69, 1,722,138 bytes, SHA-256 `99e32a90…` ; v2+v3, original key `5f4819e4…` ; tested page inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page + `main.js` inside; zip 100,326,041 bytes, SHA-256 `e66db8ce…` ; 5 parts re-join identically; **not run on a real Mac**

Tested here vs. needs real devices

- ✅ **Tested on the build machine:**
  - the webpage in Chromium;
  - the Windows app's own engine (Electron) running the page;
  - the tablet's page in Chromium;
  - the offline renders.
- 🟡 **Needs your devices:**
  - **listening** to it on the real laptop, Galaxy Tab, Mac and TV speakers (nothing here was listened to by a person — the checks are measurements);
  - whether the first line is clear enough over the music's build in your room;
  - Android's WebView playing it (the APK carries the same page, checked by its fingerprint).
- **Rehearse at home:** Rehearsal Checklist row 29 now starts with "the intro (the new voice, then the explosion)".

Live acceptance: **PENDING USER LIVE TEST**.

Build 130 addendum (16 October 2026)

Your two specs: "Universal TV Audience Display" and "Live Quiz Host Transfer: Galaxy Tab → Windows Laptop". **Your choices:** both in one build; any TV on the same Wi-Fi may show the audience screen (no pairing code); and all five smaller upgrades.

What changed

- 📡 **Audience Screens (no cable).** Any TV, projector or computer with a web browser on the same Wi-Fi opens `http://<laptop or tablet address>/tv` and shows the audience screen. It's in 📺 Audience View ▾ → Connect Audience Screen, on the setup screen and in 📲 Connect ▾. Several screens can be connected at once, at 720p, 1080p or 4K, with an optional safe margin for TVs that cut the edges. The panel shows each screen's name, status, connection quality and last update, with Test, Refresh, Rename, Margin, Disconnect / Reconnect and Forget. It works whether the laptop or the Galaxy Tab hosts. HDMI and the Audience View window are unchanged.
- 💻 **Take Over Tablet Quiz.** A quiz running on the Galaxy Tab moves to the laptop while it runs, after the tablet's Quizmaster taps Approve Transfer. Scores, the question on screen, Daily Doubles and wagers, chips, the countdown and the Final all move. Team phones keep their team without typing a code again, remotes and TVs follow, and the tablet becomes the Quizmaster remote. It moves from tablet to laptop only, as your spec asked, with no automatic take-back.
- 🧪 **Test Hosting** (Connect Device → Run Test) checks that phones, remotes and audience screens can reach the laptop or tablet. A 🔋 **battery chip** appears below 20% (red at 10%), with one reminder; it never shows on the TV. 🔒 **Lock** on the remote ignores taps until you press and hold. Connected Devices shows a ▂▄▆ **signal** and speed for each remote. The Windows joiner now checks the joined file's **SHA-256 fingerprint**.

How it works (short)

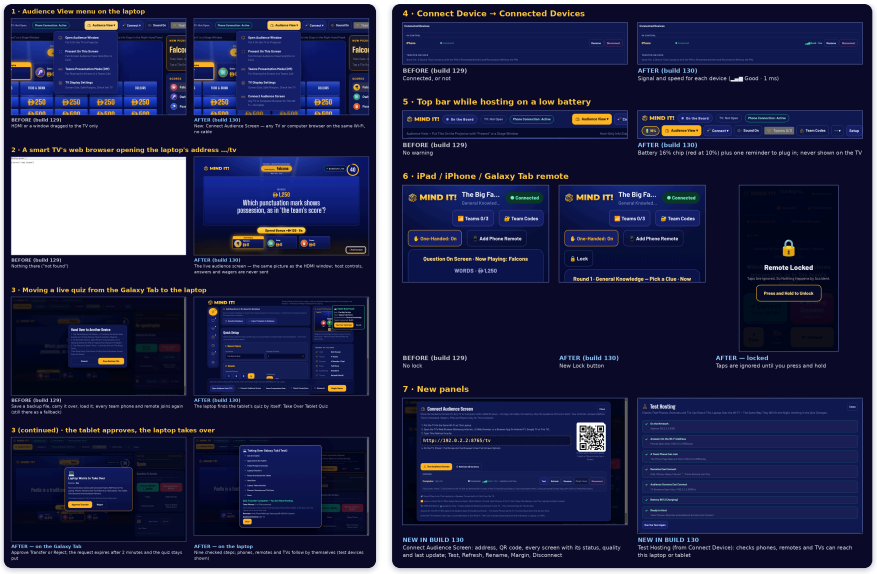
- *Audience screens:* the host draws the audience picture exactly as for the HDMI window (or on a hidden page when no window is open). It sends that finished picture over its own connection, `/audience`, which can only receive; it never sends the game data. The server accepts only hello, screen size and test replies from a TV, limits their size and rate, and refuses any picture that contains script. A TV asking for the Quizmaster's data gets "refused" (403). So hiding is not done with styling: answers, wagers, PINs, team codes and players are simply never sent.
- *Google Cast* was looked at and not used. A Cast receiver needs a paid Google developer registration and a secure (https) web address, which a local quiz on venue Wi-Fi doesn't have. A TV's own web browser does the same job with no extra hardware, and HDMI stays as before.
- *Take over:* nine stages:

  1. ask;
  2. approval on the tablet;
  3. the tablet prepares a snapshot;
  4. the laptop checks it (version, fingerprint, contents);
  5. the tablet pauses scoring (at most 30 seconds);
  6. hand-over (the tablet saves "committed" before it answers);
  7. the laptop takes the quiz;
  8. phones, remotes and TVs move;
  9. done.

  Only one device hosts at any moment. After the commit, the tablet refuses game changes and points every device to the laptop. The laptop's quiz gets a new host generation number, and a command that was already applied is ignored if it arrives again. **Before the commit**, any failure leaves the quiz on the tablet, and it resumes by itself. **After the commit**, the tablet never takes the quiz back by itself. If the laptop can't be sure the hand-over finished, it asks: *Host It Here* or *Discard*. Both devices keep the snapshot, and the laptop saves its own previous game as a Safety Backup first.

Found and fixed during testing (none of these reached a download)

- **How to Play changed look:** the new Test Hosting styles reused three style names the audience *How to Play* screen already uses. The before/after pictures showed it. The new styles were renamed, and How to Play is now **pixel-identical to build 129**. A new check, `fix130c`, compares How to Play with the published build 129 page. It also requires every new build 130 style to use names that only build 130 code uses.
- **Test Hosting's battery line** used the reading from up to a minute earlier; it now reads the battery when the test runs.
- **Moving a quiz:** on the build machine, these went wrong and were fixed:
  - an iPhone remote could grab control before the tablet;
  - remotes lost their names when they moved;
  - a laptop restart could re-apply an old snapshot over newer scores;
  - a restarted laptop was told "busy";
  - the tablet's *Keep Hosting Here* question was hidden behind the pause screen.
- **Title case:** "TVs", "PINs" and "LG" are now written correctly.



Before and after (real screenshots: build 129 and build 130 side by side).

Check	Result
`fix130a` (audience screens on the laptop: a TV opens the address and is listed; same picture as the Audience View; board, question, countdown; 720p + 1080p + 4K at once, each scaled, 16:9 kept; Wi-Fi drop → small *Reconnecting* line, last picture kept, recovers by itself, game never reset; reopening mid-question; the panel: rename (remembered), safe margin, Test Audience Screen 5/5, Refresh, Disconnect / Reconnect / Forget; a TV sending commands changes nothing and gets 403 for wagers, players, info and health; no Quizmaster controls in 816 pictures)	**41/41**
`fix130b` (privacy and a whole game: unplayed questions (57) and pictures (129), unrevealed answers, the Daily Double wager, Final wagers and typed Final answers, team codes and the PIN never reach a TV; every round to the Final reveal and winner; HDMI window and TVs together; a slow TV (slow network, 4× slower processor) keeps up)	**20/20**
`fix130t` (the Galaxy Tab hosting: two TVs show its screen, rename / test / disconnect from the tablet, remembered in the tablet's storage; Test Hosting on the tablet incl. background hosting and battery; battery chip at 9%, not on the TV)	**13/13**
`fix130x` (your success test: live quiz on the tablet with phones, a remote and a TV; Tablet Quiz Found; approve; same scores and question; countdown resumes; phones keep their team; the tablet becomes the remote; the TV follows with its name; the tablet refuses late writes; no double scoring; laptop restart; transfer before the quiz starts)	**32/32**
`fix130y` (during a Daily Double, with a Triple Gamble armed, during the Final with a wager still to come; Reject; repeated and competing requests; a laptop on another build is refused; the laptop's old game kept as a Safety Backup)	**22/22**
`fix130z` (laptop cut off during the pause → the tablet resumes after 30 s; tablet cut off just after the commit → the laptop finishes when it's back; the tablet app stopping mid-pause → *Keep Hosting Here*)	**18/18**
`fix130m` (laptop killed before the commit → the tablet keeps hosting; killed after the commit → the restarted laptop finishes from its saved snapshot; nobody answers → expires after 2 minutes)	**15/15**
`fix130e` (Test Hosting on the laptop; battery chip below 20%, red below 10%, gone when charging, never on the TV; Lock on the tablet/iPhone remote and the Phone Remote — taps ignored, short press doesn't unlock, hold does; signal in Connected Devices)	**17/17**
`fix130c` (new: How to Play identical to build 129; every new style uses new names only) — and it was shown to fail on the page before the fix	**7/7**
Windows joiner under Wine (with a stand-in for Windows' `certutil`): no certutil → joins, says the check was skipped; right fingerprint (new and old certutil formats) → joins, "Fingerprint checked"; wrong fingerprint → no EXE, "a part is damaged"	**4/4**
Full regression	**112 suites, 2,495 / 2,495** (75 min; no suite needed a retry), run on the final page — after both fixes above
Layout audits	host screens at 5 sizes + tablet home: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues** (final page)
Windows EXE	69,038,907 bytes, SHA-256 `03cbf028…` ; tested page inside (md5 `970333f9…`); `main.js` = build 130; the three parts re-join to the same file
Android APK	versionCode 130, versionName 1.69, 1,701,658 bytes, SHA-256 `729633a4…` ; v2+v3, original key `5f4819e4…` ; hosting service type `connectedDevice` ; tested page inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page + `main.js` inside; zip 100,301,418 bytes, SHA-256 `c053a8e0…` ; 5 parts re-join identically; **not run on a real Mac**

The numbers at the start of each check line in the test logs are the numbers of your spec's tests. The transfer checks carry 1–19; the full regression covers the existing features.

Limits of this testing — please read

- 🟡 **No real Galaxy Tab.** The tablet's server ran on the build machine (the same Java code as the APK) with its page in Chromium. The tablet's background hosting and battery reading were simulated. On a real Wi-Fi the laptop finds the tablet through the tablet's network announcement, which wasn't tested; if *Tablet Quiz Found* doesn't appear, use 📲 Connect ▾ → Take Over Tablet Quiz and type the tablet's address.
- 🟡 **No real smart TV.** The "TVs" were Chromium pages at 720p, 1080p and 4K. Some older TV browsers can't open local addresses or run modern pages; then use a cheap streaming stick with a browser, a laptop, or HDMI. Sound plays from the host, not the TV.
- 🟡 **The joiner's fingerprint check** ran under Wine with a stand-in for `certutil`, not on real Windows. If certutil is missing, the joiner still joins and says the check was skipped.
- If the laptop app is killed outright, its last few seconds of browser-side saving can be lost. The transfer itself is written to a file first, so a laptop that crashes after the commit still finishes the transfer from that file (tested).
- **Rehearse at home** (Rehearsal Checklist rows 34–36): Test Hosting; a TV on `/tv` with Full Screen and Test; moving a tablet quiz to the laptop.

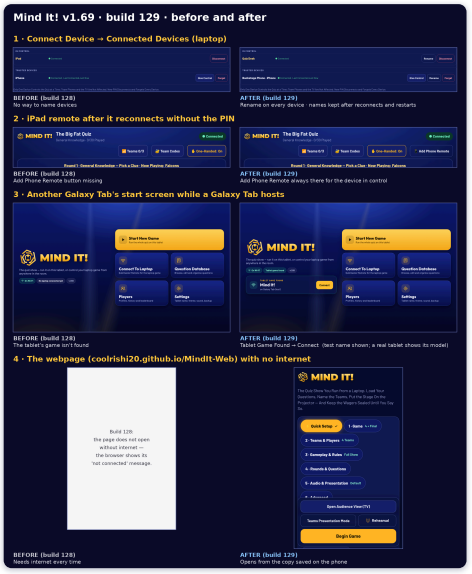
Live acceptance: **PENDING USER LIVE TEST**.

Build 129 addendum (16 October 2026)

Your choice: upgrades **1, 2, 4, 5, 6, 7**. Number 6 (smaller downloads) was dropped: the app's extra language files were already removed, so there was nothing left to trim — I told you at the time.

What changed

- **1 · The Galaxy Tab keeps hosting in the background.** While the tablet hosts, Android is told a quiz is running: a notification *Mind It! is hosting a quiz*, and the tablet keeps its Wi-Fi and server awake if the screen goes off or another app is opened. Swiping Mind It! away stops it. Android 13+ asks once to allow notifications.
- **2 · Other tablets find the tablet's game.** The hosting tablet announces its game on the Wi-Fi; another Galaxy Tab's start screen shows **Tablet Game Found** (and never lists its own game). Laptop games still say *Laptop Game Found*.
- **4 · Rename devices** in Connected Devices (laptop and tablet): the device in control and every trusted device. The name is used on request cards, *Currently in control* and the waiting remote, and is kept through reconnects and restarts. An empty name goes back to the device's own name.
- **5 · The webpage works offline.** The hosted webpage keeps a copy on the phone once opened; with no internet it opens from that copy (with its players and history). With internet it always loads the newest build first.
- **7 · 📱 Add Phone Remote is always there** on the remote in control: the host now sends it the PIN, so the button no longer disappears after a reconnect without the PIN.



Before and after (real screenshots; the offline "before" is a description, because the browser shows only its own "not connected" message).

Check	Result
`fix129d` (laptop: Add Phone Remote after a reconnect without the PIN + its code decoded; Rename / Cancel / Enter / empty name; no redraw while typing; name in the request card and on the waiting iPhone; name saved in the trusted-devices file; kept after reconnect and laptop restart)	**21/21**
`fix129t` (the tablet's own server on the build machine: `kind: tablet`; PIN sent to the controller; rename on the tablet, saved, kept after a server restart; another tablet's start screen shows *Tablet Game Found* + chip; the hosting tablet doesn't list itself; laptop games unchanged)	**16/16**
`fix129w` (the hosted webpage served as on GitHub Pages: offline helper installs; with the site switched off and the browser offline the page opens, with its saved data; a new tab opens offline; icon kept; other requests fail normally; back online a newer upload is used at once) — and a control run without the helper fails to open offline, as expected	**13/13**
Full regression	**103 suites, 2,316 / 2,316** (64 min). `fix127d` passed on retry: its wrong-PIN check reused a browser that has been trusted since build 128, so it sometimes connected without the PIN, which is correct. The check now uses a new device; re-run twice, **22/22** both times
Layout audits	host screens at 5 sizes + tablet home: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues**

Check	Result
Android manifest	`foregroundServiceType=connectedDevice` on the hosting service, checked with `aapt dump xmltree` and with a second, independent reader (androguard); permissions FOREGROUND_SERVICE, FOREGROUND_SERVICE_CONNECTED_DEVICE, POST_NOTIFICATIONS; the Wi-Fi multicast permission Android 14 requires for this service type is declared
Windows EXE	69,016,865 bytes, SHA-256 `5d23dc2b…` ; tested page inside (md5 `ae10643d…`); `main.js` = build 129; parts re-join identically under Wine
Android APK	versionCode 129, SHA-256 `6b26e0b3…` , v2+v3, original key `5f4819e4…` ; tested page and new start screen inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page + `main.js` inside; zip 100,269,100 bytes (95.6 MiB), SHA-256 `1e78e68d…` ; 5 parts re-join identically; **not run on a real Mac**

Limits of this testing — please read

- 🟡 **The background hosting (1) and Tablet Game Found on a real network (2) were not tested at all.** They need a real Galaxy Tab running Android; the build machine has none. What was checked: the app is built as Android 14 and 15 require (manifest), and the code compiles. The tablet's server itself (rename, PIN) ran on the build machine.
- **Rehearse at home:** start a quiz on the tablet, join with a phone, turn the tablet's screen off for a minute and check the phone still works; look for the notification; on a second tablet, check *Tablet Game Found*.
- Offline webpage: tested in Chromium, not real iPhone Safari. An iPhone keeps the home-screen app's copy separately from Safari's. In Safari itself (not on the home screen), iOS can delete a site's saved data after 7 days without a visit, so use the home-screen icon.

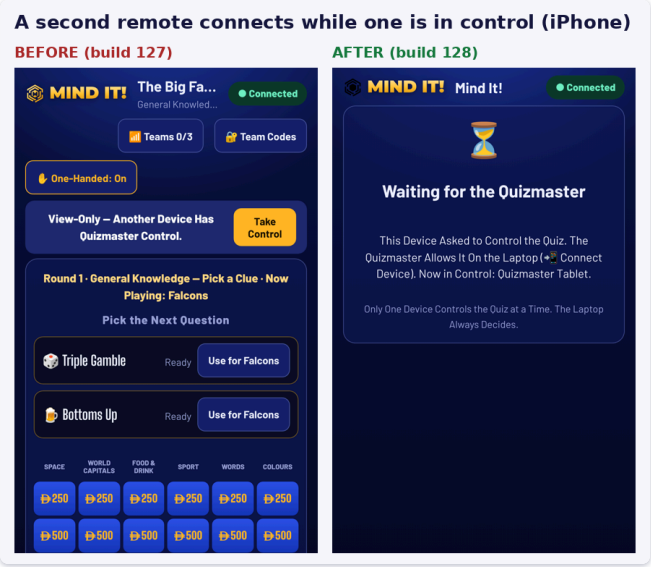
Live acceptance: **PENDING USER LIVE TEST**.

Build 128 addendum (16 October 2026)

Your two specs ("Trusted Device Reconnection & Host-Controlled Device Management" and "Multi-Device Remote Control Hierarchy") with your decisions: the 📱 Phone Remote counts as a remote; a phone joins the laptop directly; requests wait 2 minutes; and "i need the tablet to be the server".

What changed

- **Trusted devices:** PIN once, then a device's own key; reconnects by itself after drops and restarts; Forget / New PIN revoke.
- **One controller:** only one remote (tablet, iPad/iPhone or Phone Remote) controls the quiz; it keeps its place through drops and restarts; only it receives the game (answers stay private).
- **Approval:** a second device waits; the host gets **Allow & Switch / Reject / Ignore** in the right-hand panel (never on the TV, not in Present mode); requests expire after 2 minutes.
- **Connected Devices** panel: in control · waiting · trusted (last connected) · Give Control / Disconnect / Forget.
- 📱 **Add Phone Remote** on a laptop-connected tablet sends a phone straight to the laptop.
- **Galaxy Tab as the server:** the tablet app runs the same server as the laptop (rewritten in Java) when it hosts a quiz alone — team phones, chips, buzzers, Final answers, Phone Remote and remotes.



📸 A second remote — before and after.

Laptop: the second device asks — Allow & Switch / Reject / Ignore

BEFORE (build 127)

AFTER (build 128)

The laptop when a second device asks — before and after.

Connect Device → Connected Devices

BEFORE (build 127)

AFTER (build 128)

Connected Devices — before and after.

Galaxy Tab hosting on its own: now the server (Connect Device)

BEFORE (build 127)

AFTER (build 128)

The Galaxy Tab hosting on its own — before and after.

Your 14 + 15 test scenarios

Scenario	Where	Result
New device connects with the PIN; reconnects without it	`fix128d` 1–2	✅
Trusted device reconnects after the laptop app restarts (data kept)	`fix128e` 3	✅
Second device while one is active; Allow & Switch; Reject	`fix128d` 4–6	✅ the first device keeps working until you decide
A pending request expires without interrupting the quiz	`fix128e` 7	✅ (2 min; scores unchanged)
Manual switching from Connected Devices; Disconnect; Forget	`fix128d` 8	✅ a disconnected device asks before it gets control back
Active device loses Wi-Fi and reconnects	`fix128e` 9	✅ same question still open
The old device reconnects after a switch → asks	`fix128d` 10	✅
Several devices ask at once	`fix128d` 11	✅ exactly one in control
Revoked / wrong-PIN device; commands from a device that lost control	`fix128d` 12	✅ refused
Active device drops during a question and during a Daily Double wager	`fix128e` 13	✅ state preserved
Team phones and the Audience View unaffected	`fix128d` 14, `fix128e`	✅ (request never on the TV or in Present mode)

Scenario	Where	Result
📱 Phone Remote counts as a controller	`fix128e`	✅ waits, Allow & Switch
Phone ↔ tablet: PIN, trusted reconnect, tablet hosts a whole quiz, phone controls it, switching, restart	`fix128t`	✅ on the build machine (JVM)
Tablet ↔ laptop, phone joins the laptop via the tablet's code	`fix128d` , `link`	✅

Check	Result
`fix128d` (laptop + iPad / iPhone / Android-sized pages)	**35/35**
`fix128e` (restart, Wi-Fi drop, expiry, Present mode, Phone Remote, drops mid-question / mid-wager)	**18/18**
`fix128t` (the tablet's own server — same Java source as the APK — on the build machine: phones join, DD wager incl. minimum, chips, buzzers, Final wager + answer, remotes, approval, restart, Check Connections, profiles)	**22/22**
Existing suites updated for the new rules	`link` (second tablet now waits; stale code with wrong PIN refused), `fix121` (Phone Remote approved first), `conn110`
Full regression	**100 suites, 2,269 / 2,269**, every suite first time (57 min)
Layout audits	host screens at 5 sizes + tablet home: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues**
Windows EXE	69,015,668 bytes, SHA-256 `1f38f139…` ; tested page inside (md5 `59efc466…`); `main.js` = build 128; parts re-join identically under Wine
Android APK	versionCode 128, SHA-256 `20ed7e1d…` , v2+v3, original key `5f4819e4…` ; tested page inside; the server classes inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page + `main.js` inside; zip 100,268,686 bytes (95 MiB), SHA-256 `69f902de…` ; 5 parts re-join identically; **not run on a real Mac**

Limits of this testing — please read

- 🟡 **The Galaxy Tab as the server has not run on a real Galaxy Tab.** The same Java code was tested on the build machine with the quiz page loaded as the app loads it and phones joining over the network. Android itself (the WebView, Wi-Fi, the app going to the background, power saving) was not part of it.
- iPhones and iPads were simulated in Chromium, not real Safari.
- "Encrypted communication" from your spec: connections stay on your local Wi-Fi over plain `http://` , as before. Pairing uses the PIN and a per-device key, and the laptop/tablet stores only a fingerprint (hash) of each key. Real encryption would need security certificates on every device, which can't be done offline without extra setup.
- "Google Cast / Android TV audience views" from your spec: Mind It! doesn't have these, so nothing could be affected. The Audience View window works as before.

Live acceptance: **PENDING USER LIVE TEST** — especially a rehearsal with the Galaxy Tab as the server.

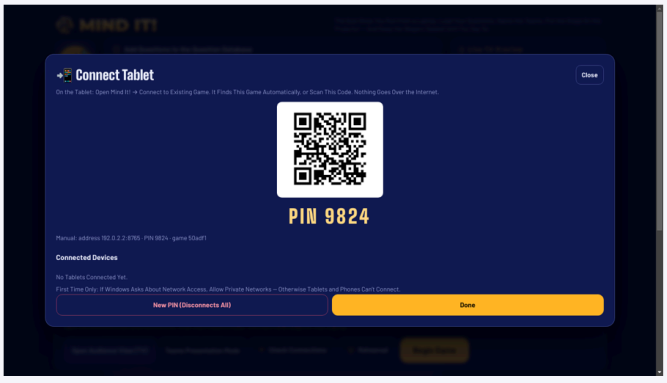
Build 127 addendum (16 October 2026)

Changes (your requests: the iPad/iPhone remote — "do it"; then "change connect tablet into connect device"):

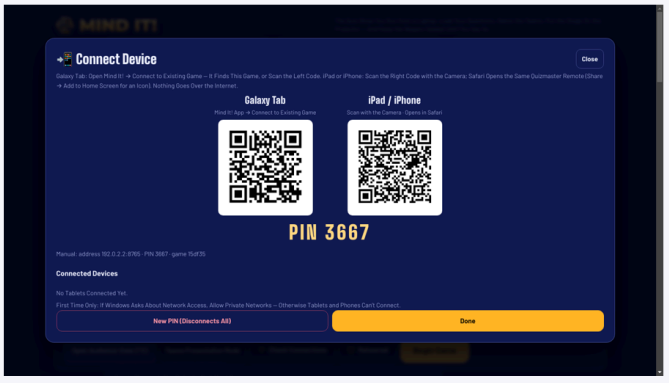
- 📲 **Connect Tablet is now Connect Device** — in the Connect ▾ menu, Setup → 6 · Advanced, the Devices window (*Connect device*, *Show Device QR / PIN*) and the window itself.
- The window shows **two codes**: **Galaxy Tab** (unchanged) and **iPad / iPhone** — a normal web address the Camera opens in Safari, giving the same full Quizmaster remote as the Galaxy Tab. The laptop app serves it at `/tablet` .
- A home-screen icon for that remote keeps working after the laptop app restarts (it picks up the new game number; the PIN is still required).
- On an iPhone the remote's board now fits the screen (all 6 columns).
- **Bug found and fixed:** a restored player could count as still deleted if the deletion and the restore happened in the same millisecond (only reachable by automation — `fix125` failed once on its first try and passed on the retry). A restore is now always dated after its deletion.

Connect Tablet → Connect Device, with a new iPad / iPhone code

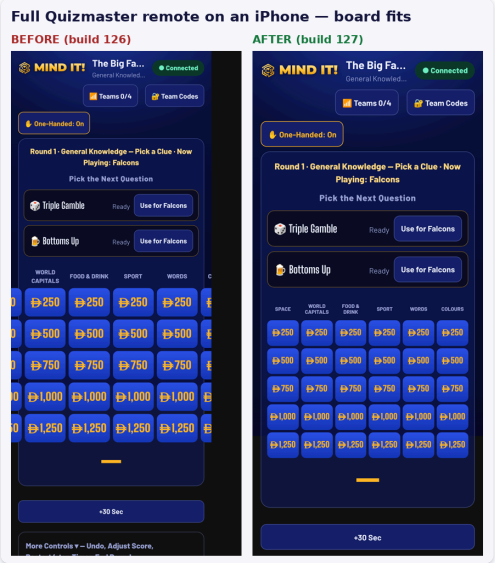
BEFORE (build 126)



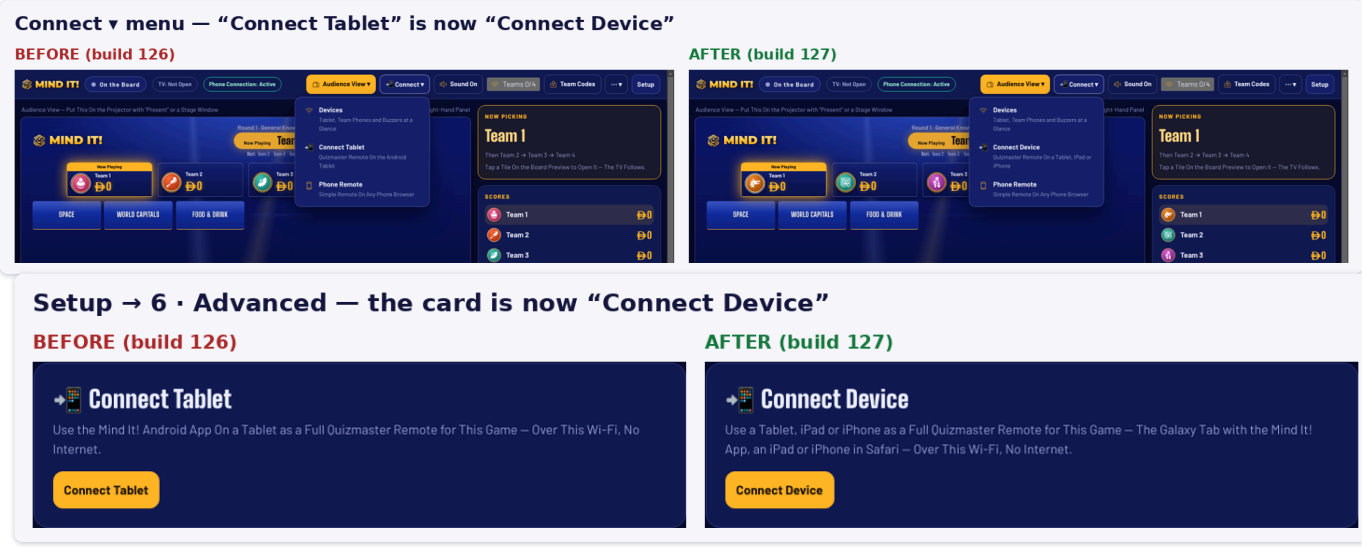
AFTER (build 127)



📸 Connect Tablet → Connect Device, with the new iPad / iPhone code — before and after.



The full remote on an iPhone — before (board runs off the screen) and after.



Connect ▾ menu and the Setup card — before and after.

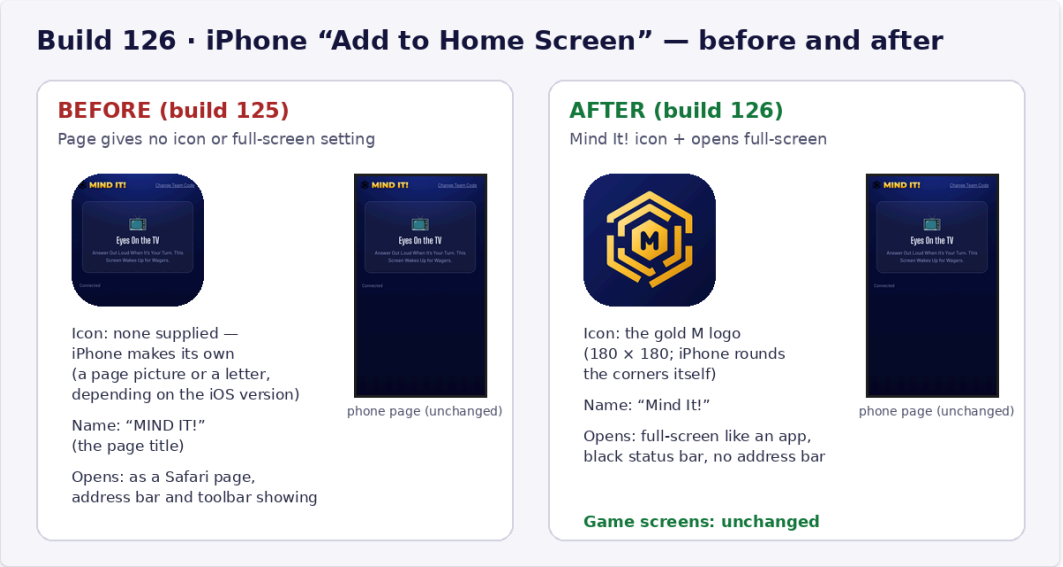
Check	Result
`fix127d` (laptop app + iPhone- and iPad-sized browsers): the window is Connect Device with two equal codes; the right code is the laptop's `/tablet` address with game number and PIN (both codes decoded from a screenshot and checked); the Galaxy Tab code unchanged; the dialog fits the laptop screen; Setup and the Connect ▾ menu say Connect Device; the laptop serves `/tablet` to other devices; iPhone: Connected, board fits (6 columns, nothing off the edges, every value in full), tapping a value opens the question on the laptop, Correct / Wrong appear; an old game number in the address still connects; a wrong PIN can't; iPad: connects, tablet layout unchanged; no errors	**22/22**
Full regression	**97 suites, 2,188 / 2,188** — 96 first time; `fix125` once on retry (the bug above), then fixed; after the fix `fix125` 3 × 27/27, `fix124` 19/19, `psync`, `libsync`, `fix125d`, `fix127d` 48/48
Layout audits	host screens at 5 sizes + tablet home at 3 sizes: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues**
Windows EXE	69,009,786 bytes, SHA-256 `2470d7fa…`; tested page inside (md5 `c9db9f0f…`), `main.js` with the `/tablet` route; parts re-join identically under Wine
Android APK	versionCode 127, SHA-256 `a547d00e…`, v2+v3, original key `5f4819e4…`, tested page inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page and `main.js` inside; zip 100,261,994 bytes (95 MiB), SHA-256 `456f2c4e…`; 5 parts re-join identically; **not run on a real Mac**
Webpage	on GitHub Pages: https://coolrishi20.github.io/MindIt-Web/ (updated to build 127)

Limits of this testing: iPhone and iPad were simulated with Chromium (size, touch, user agent), not real Safari. The GitHub-hosted webpage can't be a remote for the laptop — a secure `https://` page isn't allowed to talk to the laptop's local `http://` address; use the **iPad / iPhone** code instead.

Live acceptance: **PENDING USER LIVE TEST**.

Build 126 addendum (13 October 2026)

Change (your request: "ok do both, include the webpage as a download option in github"): 📲 Mind It! on the home screen — on an iPhone or iPad, Safari's **Share → Add to Home Screen** now gives a **Mind It!** icon (the gold M logo) that opens the team-phone page or the 📱 Phone Remote full-screen. The page's `<head>` only: no game screen, rule or saved-data change. The webpage version `mindit.html` is now also on your GitHub download page.



📸 Home-screen icon and launch — before (build 125) and after (build 126).

Check	Result
`fix126` (the page itself): full-screen setting, name "Mind It!", browser tab title unchanged; 180×180 and 192×192 icons solid (no see-through corners); page starts normally at laptop and iPhone size, no errors	**8/8**
`fix126d` (laptop app + an iPhone-sized phone): the laptop's `/play` and `/remote` pages carry the full-screen setting, name and icon; the icon is the gold M logo; with the "opened from the home screen" flag set, the phone joins its team and shows "Connected"; nothing sticks out sideways; the Phone Remote still connects with its PIN; no errors	**13/13**
Webpage as a downloaded file	opened from disk in Chromium: starts, keeps a saved game and a new player after closing and reopening, Audience View opens; no errors
Full regression	**96 suites, 2,169 / 2,169**, every suite first time (53 min)
Layout audits	host screens at 5 sizes + tablet home at 3 sizes: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues**
Windows EXE	69,008,995 bytes, SHA-256 `6cd8baec…` ; tested page inside (md5 `d9d70e55…`), `main.js` unchanged; parts re-join identically under Wine
Android APK	versionCode 126, SHA-256 `c18ac5de…` , v2+v3, original key `5f4819e4…` , tested page inside
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page inside; zip 100,260,337 bytes (95 MiB — under GitHub's 100 MiB limit), SHA-256 `c04447c1…` ; 5 parts re-join identically; **not run on a real Mac**
Web link	republished (alternate builds: 116, 118, 120, 122, 124, **126**); the published page carries the new head tags and the same app code

Limits of this testing: the phone checks used Chromium pretending to be an iPhone (size, touch, iPhone user agent). Real Safari wasn't available here, so how iOS shows the icon and opens it full-screen is **not verified on a real iPhone or iPad**. Android Chrome opens a plain `http://` address from the home screen as a normal tab (no full-screen).

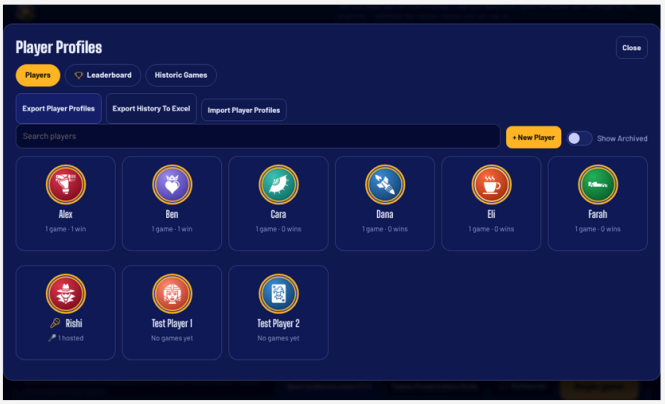
Live acceptance: **PENDING USER LIVE TEST**.

Build 125 addendum (13 October 2026)

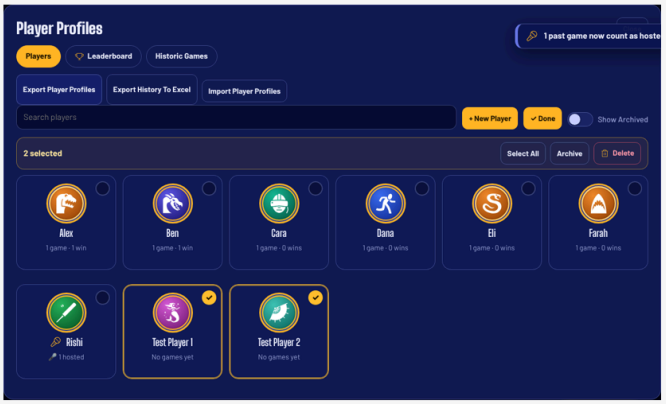
Changes (your request, all five): 🗑 Recently Deleted (restore a deleted player for 30 days) · 🛟 Safety backups (automatic, last 10) · ☑ Select several players · the tablet keeps what you type · 🖼 Results Picture for any Historic Game.

BEFORE — build 124

1+3 · Player Profiles: “Select” (tick several → Archive / Delete) · “Recently Deleted” appears after a deletion



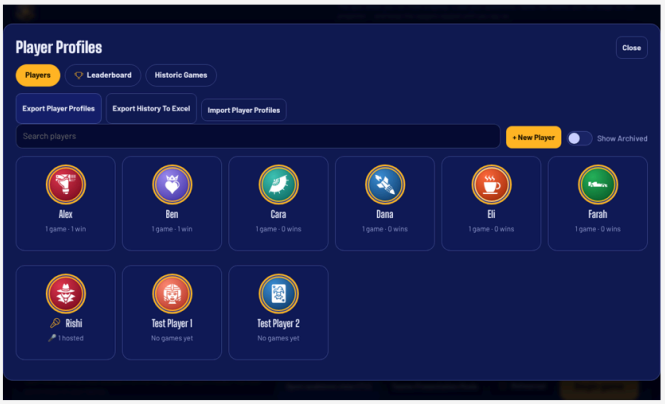
AFTER — build 125



📸 *Player Profiles — before (build 124) and after (build 125, Select mode with two players ticked).*

BEFORE — build 124 (no way back)

1 · Recently Deleted: Restore for 30 days

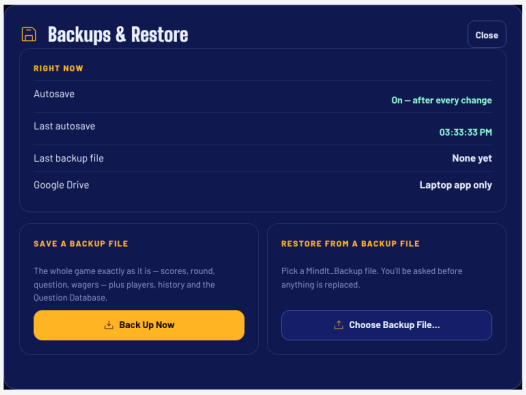


AFTER — build 125

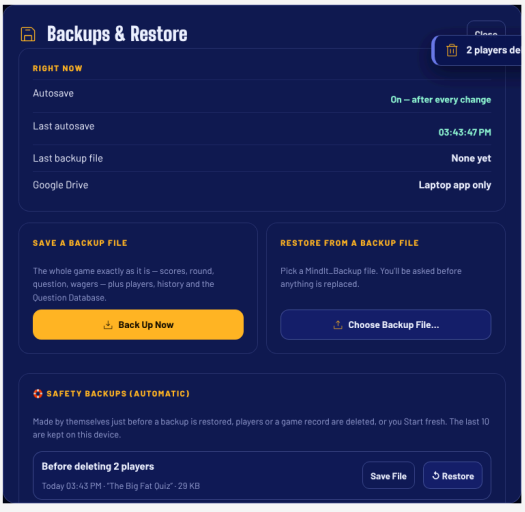
📸 *Recently Deleted (new).*

BEFORE — build 124

2 · Backups & Restore: Safety backups (automatic, last 10)



AFTER — build 125



📸 *Backups & Restore — before and after (🛟 Safety backups card).*

BEFORE — build 124

5 · A Historic Game: “Results Picture” joins the bottom row

AFTER — build 125

📸 *A Historic Game's bottom row — before and after.*

Check	Result
`fix125` (browser): Select / Select All / untick / Delete several with one question / Archive several; Recently Deleted list, Restore, picture and nickname back, 30-day expiry keeps the name, a sync can't re-delete a restored player; safety backups before deleting players, a game record and Start fresh, only 10 kept, listed in Backups & Restore, Save File gives a normal backup, Restore asks first; Results Picture saves a PNG for a past game	**27/27**

Check	Result
`fix125d` (laptop app + tablet): Adjust Score — amount, reason, chosen team and the cursor survive the laptop scoring meanwhile; Cancel starts empty next time; Apply works	**9/9**
Full regression	**94 suites, 2,127 / 2,127**, every suite first time
Layout audits	host screens at 5 sizes: **0 issues**; Audience View 234 screens at 9 sizes: **0 issues**
Windows EXE	68,963,836 bytes, SHA-256 `e995b25f…` ; tested page inside; parts re-join identically under Wine
Android APK	versionCode 125, SHA-256 `e39b61b4…` , v2+v3, original key
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page inside; 5 parts re-join to `5fcb4496…` ; **not run on a real Mac**

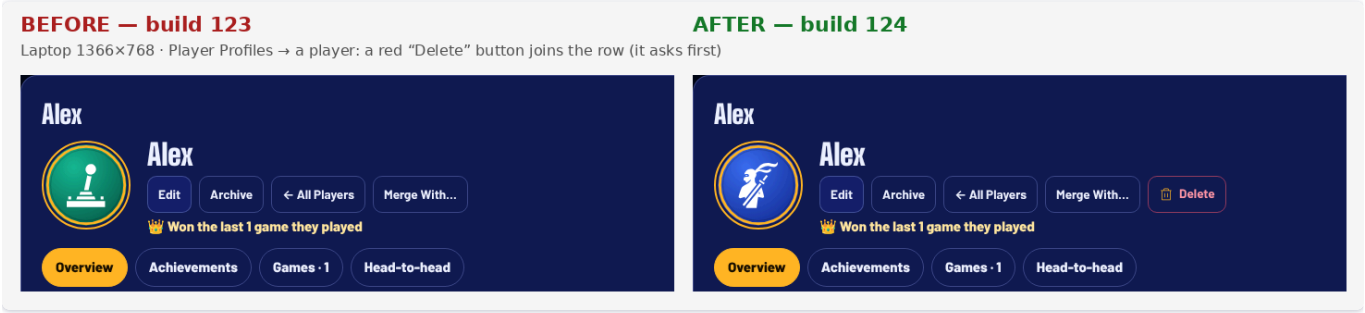
Note: safety backups are stored inside the app on each device (laptop app, or the tablet when it runs a game alone) — not on the linked tablet remote and not in a file you can see; **Back Up Now** files are still the backup to keep elsewhere.

Live acceptance: **PENDING USER LIVE TEST**.

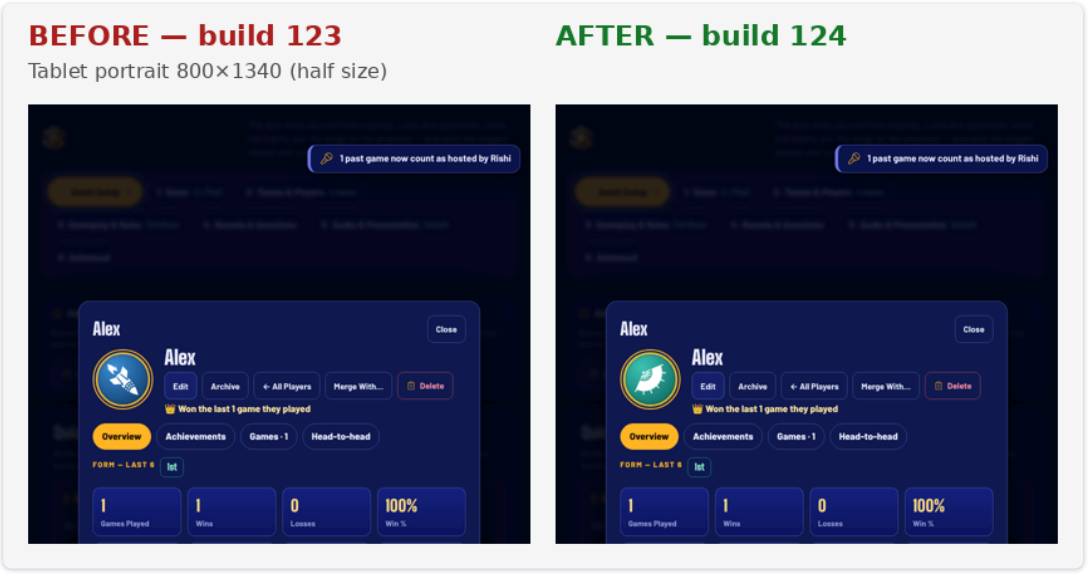
Build 124 addendum (13 October 2026)

Changes (your requests): 🗑 **delete a player** from Player Profiles; the Mac app is now **Apple Silicon only**.

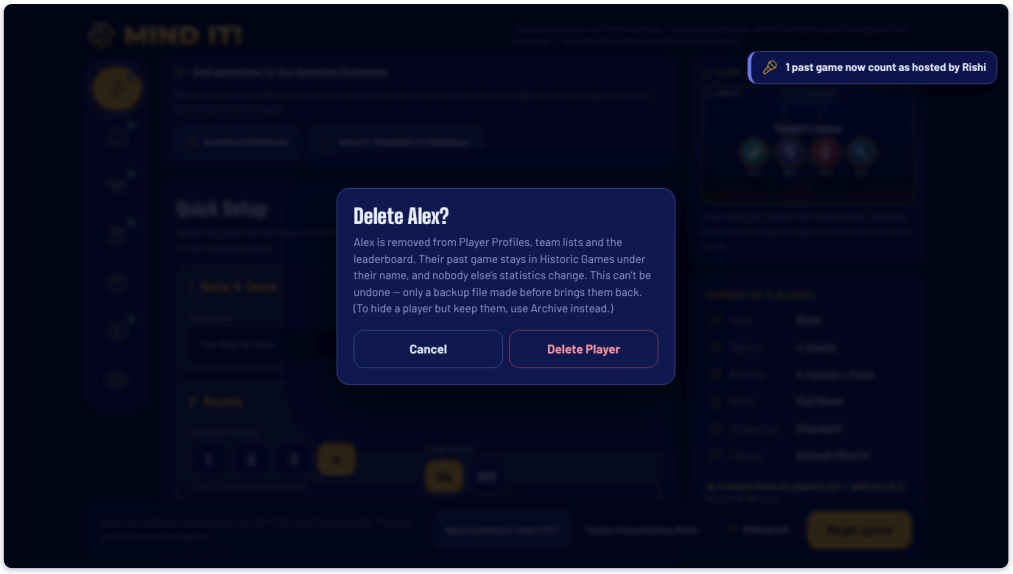
Layout — before / after (the only layout change is one more button on a player's page; nothing else moved):



Laptop 1366×768 — a player's page, build 123 (left) and build 124 (right).



Tablet portrait 800×1340 — the same row fits on one line.



The question before deleting.

Check	Result
New suite `fix124` : button on the same row as Edit/Archive/All Players/Merge; asks first; Cancel keeps; Delete removes from Player Profiles, the list and team lists; on Setup taken off their team; Quizmaster cleared; refused during a game; Historic Games keeps the name; other players' statistics unchanged; survives a reload; sync never brings the player back and passes deletions on; Returning Champion ignores deleted winners; restoring an older backup brings the player back	**19/19**
Full regression	**92 suites, 2,097 / 2,097**, every suite first time
Layout audits	laptop/tablet/phone host screens (incl. the player page) at 5 sizes: **0 issues**; Audience View 234 screens at 9 sizes (FHD, 4K, 16:10, 4:3, portrait): **0 issues**
Windows EXE	68,959,783 bytes, SHA-256 `bd485b3b…` ; tested page inside; parts re-join identically under Wine
Android APK	versionCode 124, SHA-256 `a0ca8c0a…` , v2+v3, original key `5f4819e4…`
Mac (Apple Silicon)	arm64, ad-hoc signed, tested page inside; zip 100,212,557 B (symlinks kept), 5 parts re-join to `fbdf9b69…` ; **not run on a real Mac**
Web link	republished (version 166), live page verified

Live acceptance: **PENDING USER LIVE TEST**.

Build 123 addendum (13 October 2026)

Changes (your decisions): restoring a backup or loading a game file **always asks first** (Restore Backup / Load File / Cancel), even with no game running — K4 settled; team photos stay in the save (K3, "leave it"); a **Mac version** of the laptop app (your request).

Check	Result
New suite `fix123` : asks with no game running; Cancel leaves the quiz untouched; Restore Backup restores game and players; with a game in progress only the usual "A game is paused…" question; game files ask "Load this game file?"; player-profile files merge without asking; Backups & Restore → Choose Backup File… asks; the in-app note is now true	**13/13**
Full regression	**91 suites, 2,081 / 2,081.** `qlib` and `crop` restore a backup as part of their checks; they first failed because the new question appeared, were updated to press Restore Backup, then passed (26/26, 27/27).
Windows EXE	68,958,930 bytes, SHA-256 `463b707e…` ; page md5 `feeb27bd7019…` inside; parts re-join identically under Wine
Android APK	versionCode 123, SHA-256 `b937f559…` , v2+v3, original key `5f4819e4…` ; page md5 `feeb27bd7019…`
Mac app — Apple Silicon	arm64, bundle `com.mindit.quiz` 1.69.0, macOS 11+, camera and local-network permission texts present, ad-hoc signed (app and frameworks), tested page and `main.js` inside; zip 100,211,536 B with the framework symlinks kept; 5 parts re-join to the same SHA-256 (`51b4421c…`)
Mac app — Intel	x86_64, same checks; zip 106,274,619 B (`18767abd…`)

Check	Result
Mac app on a real Mac	**Not run** — no Mac here. It isn't notarized by Apple, so macOS asks once (Open Anyway).
Guides	Complete Guide 4.1b (Mac install), 22.3/22.5 (restore asks), Quick Start Mac line; Cheat Sheet and Player's Guide still exactly 2 pages

Live acceptance (Windows, Galaxy Tab, phones, TV — and now the Mac): **PENDING USER LIVE TEST**.

Build 122 addendum (13 October 2026)

Change: your Returning Champion rule — the crown goes only to a team that has **every player of last game's winning team**; a one-player winner crowns the team that player is on; split winners → no crown (`app111.js`). This settles K1.

Check	Result
New suite `champ122` (split winners, full team, full team + new player, 1 of 4 / 2 of 4, one-player winner, deleted player, teams without players, joint winners, win streak and where it stops, rule switched off)	**15/15**
Existing `champ` (crown, TV intro, Title Defence, Dethroned, next champion)	9/9
Full regression	**90 suites, 2,061 / 2,061** (`desk` passed on retry: its first attempt couldn't connect to the freshly started laptop app)
Windows EXE	68,959,626 bytes, SHA-256 `2158c7c4…` ; page md5 `e2d075a926a6…` inside; parts re-join identically under Wine
Android APK	versionCode 122, SHA-256 `a4481b76…` , v2+v3, original key `5f4819e4…` ; page md5 `e2d075a926a6…`
Web link	republished (version 165); live page verified

Everything else in this report (build 121) still applies. Live acceptance: **PENDING USER LIVE TEST**.

A. Previous task completion

Interrupted task	Build 120 — "Interactive team swapping & Returning Champion logic" (your attachment 1): click-to-select / swap / + move / undo / reset / confirm in the Random Team Generator, with persistence, tests and layout testing; plus a review (not a change) of the Returning Champion logic.
Done before your QC message arrived	The team-swapping code (`app109.js`), its suites and the desktop/tablet layout check were finished; the Returning Champion review was written up.
Remaining when the QC message arrived	The build-120 EXE/APK, the full regression run, the guide update and the deliverables A–F.
Resumed and finished	Build 120 was built, verified and checkpointed (EXE `e93ffe65…` , APK `d65fd147…` vc120, web link v164); full run 2,019/2,019 (two suites passed on retry — the cause was found and fixed in this QC, see C2); the guide's §7.4 got the new team-swapping text and screenshot.
Verification of the build-120 requirements	`rtg120` **52/52** (mouse 1366×768 and portrait-tablet touch 800×1280): select, swap, same-team re-select, cancel, + move (refused when it would empty a team), undo (swaps and moves), reset (asks first), confirm saves the final teams, scroll position kept, 46 px+ touch targets. `fix120` 13/13. In build 121 both pass again (section B).
Returning Champion	Reviewed only — **not changed**, because you asked for approval first and no new rule has been approved. The current rule is now described exactly in the guide (§12.5). See K1.

B. QC results

Full automated regression, build 121 (final page): 89 suites (55 browser + 34 laptop-app), 2,055 / 2,055 checks passed, every suite first time (no retries), 49 minutes. (An earlier full run of build 121, before fix C7 was added, gave 2,025/2,025 in 88 suites + `fix121` 32/32. `qc2` plays random games, so its check count varies from run to run: 276–291.)

Area	Suites (examples)	Result
Game creation, Setup, rounds, Question Database	`qc2` , `zones` , `qlib` , `ext` , `l3` , `l4` , `fix120`	647/647 (7 suites)
Teams, players, Random Team Generator	`rtg` , `rtg120` , `g1` , `g3`	89/89 (4 suites)
Gameplay rules (scoring, Daily Double, Final, chips, buzzer, ties)	`rules35` , `tg` , `tg2` , `ud` , `bu` , `bz` , `bz3` , `r9` , `fix112` – `fix115`	250/250 (12 suites)
Historic Games, statistics, achievements, champion	`champ` , `g2` , `g6` , `g7` , `rh` , `v169`	74/74 (6 suites)

Area	Suites (examples)	Result
Phones, tablet, link, End Game	`link`, `pin`, `psync`, `hostreq`, `hostreq2`, `conn110`, `endgame110`, `fix116`, `fix117`, `fix119`, `fix121`	238/238 (11 suites)
Complete games end to end	`fullmatch` (whole match from the tablet), `onehand`, `desk`	64/64 (3 suites)
Audience View layout	`avaudit` — 234 screens × automatic checks at 9 sizes	**235/235**, 0 layout issues
Laptop host screens	`vaudit` — 18 screens × 5 sizes (90 screenshots)	**0 issues**
Live-game stress and soak	`stress110` phases	**24/24 + soak 2/2:** idle 2 × 10 min with the Quizmaster window hidden (117 samples each, TV + tablet + 10 phones: no false disconnects, no full-screen cover); network drop/return 6/6; 10 phones at once 4/4; phone lock 60 s 2/2; rapid/duplicate tablet taps 4/4; Check Connections 6/6; **60-minute real-time quiz:** 60 questions, 346 tablet actions, Quizmaster window hidden/shown 5×, an intermission — **0 false disconnects, 0 script errors**, laptop server memory 213 → 222 MB, page heap 12 → 13 MB
Rollback (build 121 data → build 119 app)	rollback test, section I	11/11

Skipped / not possible here: real hardware (Windows PC, Galaxy Tab A9+, phones, TV, real Wi-Fi, real sound); the Android rollback; Google Drive backup (needs internet and your script).

End Game (asked specifically): `endgame110` 19/19 — End Game is refused while the game is still being played; the game is recorded **once**; the button locks while saving (no double tap); a repeated End Game after a dropped connection is acknowledged without saving twice; an End Game for an old game is refused; rehearsal End Game records nothing; standalone tablet saves once.

C. Bugs found and fixed (build 121, `app110.js`)

Every item was reproduced or measured first, then fixed, then re-measured. No gameplay rule, layout or saved-data format was changed.

#	Defect (confirmed)	Effect on a quiz night	Fix	Evidence
C1	**Connection Health: phones flickered Responsive ↔ Unstable** although they checked in every 1.5 s. The laptop added the time since its own 3-second health refresh to each phone's age (1.5 s + 3 s > the 4 s limit).	False "Unstable" warnings; the tablet's 📶 chip changed colour for nothing.	Only time beyond the normal refresh counts. A phone that really stops still turns amber (after ~7 s) and red/stale (~15 s).	80/80 samples "ok" over 20 s for each phone (before: flapping every second); dropped phone → Unstable at 7 s, Stale at 19 s.
C2	**Quizmaster tablet redrew its whole screen ~once a second.** The laptop re-sent the whole game because round-trip times inside Connection Health changed at every check-in.	Typed Adjust Score numbers could vanish; a tap could land on a button that was being replaced (this is why `hostreq2` and `fullmatch` needed a retry in the build-120 run).	The tablet's copy of Connection Health uses steady wording (round-trip bands, "just now"); the laptop keeps exact numbers.	Tablet state messages and full redraws: **1.07/s → 0/s** at idle; data sent to the tablet **~14.5 KB/s → 0**; `hostreq2` and `fullmatch` now pass first time.
C3	**Phone Buzzer Mode: BUZZ IN was rebuilt ~6.5 times a second**, so a tap whose press and release straddled a rebuild never arrived.	Lost buzzes — a fairness problem in a buzzer race.	The button is rebuilt only when something on it changes.	30 ordinary taps (120 ms): **14/30 → 30/30** reached the button; rebuilds **6.52/s → 0**. A real buzz still registers; the fast check pauses while the phone is locked.
C4	Phone screen rebuilt at **every check-in during a question** (the seconds left were part of the "has anything changed?" test).	Same lost-tap risk on chip/confirm buttons; wasted battery.	Seconds left no longer count as a change; the countdown runs on the phone and still follows +time / stop from the laptop.	Rebuilds **0.67/s → 0** in 6 s; +15 s reached the phone within 0.1 s of the laptop; stop cleared the phone timer.
C5	**Laptop Phones card (and wager and live-setup cards) rebuilt every 500 ms.**	"Join link & QR code" snapped shut; the network-address list closed while open; clicks during a rebuild were lost.	A card is replaced only when its content changes; open sections stay open; nothing is replaced while you type or choose in it.	Phones card rebuilt **2.0/s → 0/s**; a phone joining still updates it at once.
C6	📱 **Phone remote: every button press started another background check loop**; the page was redrawn every second while the timer ran.	After a few presses the phone hammered the laptop; presses could be lost.	One check loop; a press just makes the next check come sooner; the seconds update in place.	6 presses → 10 checks in 7 s (one loop); no redraw just for the seconds.
C7	**An unfinished game was lost if Mind It! was opened and closed again without choosing** Resume saved game / Start fresh. At start-up the app shows a fresh Setup while offering the saved game; closing saved that fresh Setup over it. **Also in build 119.**	A crash, reopen, close (e.g. to fix the TV), reopen → the game was gone (only a backup file could bring it back).	The unfinished game is kept aside until you choose; it is offered at every start. Save format unchanged; the extra copy (`mindit.resume.pending`) is ignored by older builds.	Restart test on the real laptop app: build 119 — offered once, then **never**; build 121 — offered at the 1st, 2nd and 3rd restart; Resume → right scores; Start fresh → no more offer.

Regression suite added: `fix121` (39 checks) covers C1–C7.

Found, not changed (reported): see K.

D. Connectivity results

Device	What was checked (simulated)	Result
Windows host (laptop app server)	Health endpoint, port fallback, saved-game and settings copies across a port change (`fix115` , `fix116`), Check Connections	Passed (`fix115` 22/22, `fix116` 26/26, stress `health` 6/6: host, tablet, TV and 8/8 teams green in 3.6 s)
Quizmaster tablet	Pairing/PIN, control hand-over, commands with acknowledgements, reconnect after Wi-Fi loss, End Game retry, Connection Health sheet (`link` , `pin` , `hostreq` , `hostreq2` , `conn110` , `endgame110` , `fix121`)	Passed (`link` , `pin` , `hostreq` 28/28, `hostreq2` 29/29 first time, `conn110` 35/35, `endgame110` 19/19, stress `net` : offline 20 s → small status line only, back in 0.3 s with the current state)
Audience View	No full-screen "Reconnecting" cover; heartbeat; the Quizmaster window hidden/minimised (build-110 fix) — `conn110` , stress `idle` / `soak`	Passed — no full-screen cover in 2 × 10 min idle and the 60-min soak with the Quizmaster window hidden half the time
Phones	Join, team codes, wagers, chips, buzzer, lock/unlock, network drop/return, Connection Health status (`psync` , `bz` , `conn110` , stress `lock` / `net` , `fix121`)	Passed (`psync` , `bz` 3 phones, stress `load` 10 phones at once, `lock` 60 s locked → "asleep", back on the same team with no re-pairing; `fix121` C1/C3/C4)
False "Reconnecting" (your live-game report)	The root cause found in build 110 (the Quizmaster window throttled when covered) stays fixed; build 121 removes the remaining false **Unstable** flicker (C1).	0 false disconnects in about 1 h 25 min of stress and soak sampling (2 × 10 min idle, the 60-min quiz and the short phases); Connection Health stays "Connected" for phones checking in normally (80/80 samples)

E. Performance results

Measured on the build machine with the laptop app, one phone and the tablet, 30 s per phase (`perfmeasure.py`), before and after build 121:

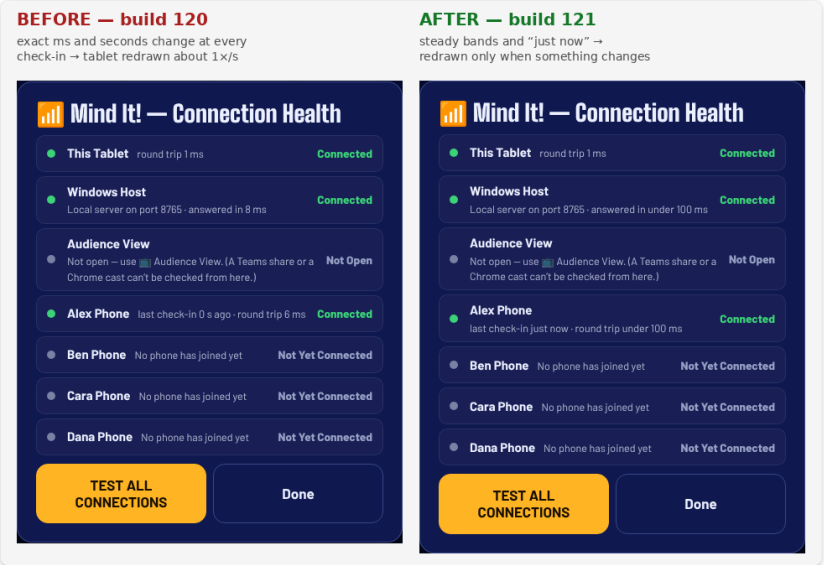
Phase	Measure	Before (build 120)	After (build 121)
Board idle	Tablet full redraws / s	1.07	0
Board idle	Game updates sent to the tablet / s · bytes / s	1.07 · 14,540	0 · 0
Board idle	Laptop Phones card rebuilt / s	2.0	0
Question, timer running	Tablet full redraws / s	1.07	0
Question, timer running	Phone screen rebuilds / s	0.67	0
Phone Buzzer Mode	BUZZ IN rebuilt / s · taps that arrived	6.52 · 14/30	0 · 30/30
Phone remote	Checks / s after 6 presses	grew with every press	~1.4 (one loop)

Requests per second are unchanged (phones check in every 1.5 s, 350 ms in Buzzer Mode); what changed is how often screens are thrown away and rebuilt.

Autosave: with four 300 KB team photos a save is ~1.2 MB and takes ~23 ms here; saves happen 0.7 s after a change (not on a timer — 0 saves in 20 s idle). Not changed (K3).

F. Visual QC

No layout was changed in build 121 (so there is no layout before/after). The only visible text change is in the **tablet's** Connection Health sheet — before/after:



📸 Tablet → 📶 Connection Health, build 120 (left) and build 121 (right), same devices.

Screen	Sizes tested	Result
Audience View (TV)	1920×1080 (FHD), 3840×2160 (4K), 2560×1600 and 1920×1200 (16:10), 1280×800, 1366×768, 1280×720, 1024×768 (4:3), 800×1280 (portrait) — 26 key screens each	234 screens, **0 issues** (text off screen, cut off, overlapping; board tiles equal and centred; smallest text)

Screen	Sizes tested	Result
Laptop host	1920×1080, 1366×768, 1340×800, 800×1340, 390×844 — 18 screens	**0 issues**
Quizmaster tablet (Galaxy Tab A9+ size, 1340×800 / 800×1340)	`fix119` , `rtg120` , `endgame110` , `butab` layout checks	passed (in the full run)
Phones (390×844 @2×)	`fix119` , `fix116` , `psync`	passed (in the full run)

Screenshots are from the sandbox; real-screen colour, brightness and viewing distance still need your rehearsal (Rehearsal Checklist).

G. Documentation update

Document	Updated for build 121	Check
Complete Idiot's Guide (PDF + Markdown)	"New in build 121"; §9.2 tablet Connection Health wording; §12.5 exact Returning Champion rule (current behaviour, 🔵 review pending); §22.3 restore wording corrected (asks only when a game is in progress; players/history merged); crash-recovery note (the offer now stays)	138 pages, rebuilt and text-checked
Quick Start	build stamp	3 pages
Quizmaster Cheat Sheet	build stamp	**exactly 2 A4 pages**, no overflow
Player's Guide	unchanged content (nothing changed for players)	**exactly 2 A4 pages**, no overflow
Rehearsal Checklist	"stay Connected for a minute, no flicker"; reopen twice without choosing; restore asks mid-game	2 pages
Project docs (status, changelog, rules, architecture, CLAUDE.md)	B31–B40, build 121 row, Build-121 module notes, spare-copy key	—
Documentation audit	new section for builds 116–121	—

Every build-121 change is described in the guide's "New in build 121" and in the changelog. Code vs documents: where they disagreed, the code was taken as the truth and the documents corrected (restore wording, Returning Champion detail); one in-app sentence is still too broad (K4).

H. Build verification

	Windows	Android
File	`Mind It!.exe` (portable, x64) — also delivered as `MindIt_v1.69.exe` (same bytes) and as 3 parts + `Join Mind It - build 121.bat`	`Mind It! for Android (Galaxy Tab).apk` — also as `MindIt_v1.69.apk` (same bytes)
Size	68,959,878 bytes; parts 3 × 22,986,626	1,574,609 bytes
SHA-256	`d8d14ca1a12187d44343eb52aa786c13a49aed15c71ee0ff6f0ac3c7e4a2f12e`	`67926597a774e18a8f84e6e43fe63c1ca2e35a1ec0a90a9348427ce68b274b62`
Version	1.69 (package 1.69.0)	`com.mindit.app` , versionName **1.69**, versionCode **121**
App page inside	`app.asar` → `app/mindit.html` md5 `a146a07d0bbc…` = the tested page; `main.js` identical to build 119	`assets/mindit.html` md5 `a146a07d0bbc…` = the tested page
Signing	not code-signed (as before — SmartScreen "More info → Run anyway")	v2 + v3 verified; one signer, certificate SHA-256 `5f4819e4b15e…` = your original key
Checks run	Built with electron-builder (exit 0); app.asar extracted and compared; parts re-joined with the `.bat` under Wine → identical SHA-256	`aapt dump badging` , `apksigner verify -v --print-certs` , page md5
Not run	The EXE on a real Windows PC (the same app shell and page ran all laptop tests here under Linux)	Install/run on a real tablet (no device or emulator here)

Source: `MindIt_Source_v1.69.zip` (7.0 MB, 438 files, extracts cleanly). Rebuilding the app page from the extracted zip gives a file identical to the shipped page except 3 characters — the timestamp inside the compressed HEIC helper (its unpacked content is identical). Git commit `c926b63` , tag `checkpoint-build121` .

I. Rollback verification

- **Preserved:** `MIND_IT_v1.69_PREVIOUS_WORKING_RELEASE.zip` (build 119 as delivered — not rebuilt): 86,577,817 bytes, SHA-256 `a745c41b…` ; 390 files, all checksums verified after unzipping; the EXE parts re-join to `49517ba1…` . Kept separate from the build-121 files and not overwritten. Git tags `checkpoint-build119` , `checkpoint-build120` , `checkpoint-build121` .
- **Rollback test (laptop app, sandbox):** build 121 wrote 5 players, a finished game, an unfinished game and a backup file; the app was closed, the build-119 page put in place and the app started on the **same profile**: players and Historic Games kept, the finished game's scores intact, the unfinished game offered and resumed with the right scores, and the build-121 backup file restored through ⋯ → Backups & Restore — **11/11**. (Run twice: with the build-121 page before and after fix C7.)
- **Android:** not tested (no tablet). Android refuses to install an older versionCode over a newer one; uninstalling deletes the tablet's Mind It! data. The guide gives three options, data-safe first.

J. Live acceptance status

PENDING USER LIVE TEST. Nothing here was run on your Windows laptop, Galaxy Tab A9+, phones or TV. Please play a 60–120-minute game with the real devices (the Rehearsal Checklist covers it) and watch for: false disconnects, full-screen Audience View interruptions, lost scores, duplicate actions, wrong game records, crashes, devices that don't reconnect. Keep the build-119 rollback until it passes.

K. Outstanding issues and risks

#	Issue	Severity	Status
~~K1~~	**Returning Champion (settled in build 122 — see the addendum):** a team with **any one** of last game's winners qualifies, and only the **first** qualifying team (team order) gets the crown — so a team with 1 of 4 winners can beat a team with 2 of 4. The proposed majority rule has open edge cases (tied winners, small rosters, deleted players, teams without players, the "N wins in a row" count).	—	**Done in build 122**
K2	Real-hardware behaviour (Wi-Fi quality, Android power saving, TV scaling, sound levels)	—	PENDING USER LIVE TEST
K3	Autosave includes team photos (~1.2 MB / ~23 ms per save with 4 photos).	Low	**Kept as is — your decision (13 Oct)**
~~K4~~	In-app Backups & Restore note said "You'll be asked before anything is replaced" but it asked only when a game was in progress.	—	**Fixed in build 123: it always asks now**
K5	The Android shell doesn't pause the page when the tablet app goes to the background (keeps the link alive — intended).	Info	Not changed
K6	The tablet still redraws fully when the game really changes (a score, a new question), so typing in Adjust Score while another device scores can still be interrupted.	Low	Not changed (rare; the once-a-second redraw is gone)
K7	Test scripts and `build.py` use the build machine's paths (`/home/claude`, `/tmp`, `/mnt`).	Info	Documented in the build instructions
K8	Build 119 (the rollback) has the C7 problem: press **Resume saved game** the first time it is offered.	Info	In the rollback guide

Release readiness: Ready for live acceptance testing. Not marked ready for unrestricted live use until your real-device game passes (J). No known open defect would stop a game; K1 is a rule decision waiting for you.